

**LAPORAN AKHIR PRAKTIKUM
ALGORITMA & STRUKTUR DATA**



Oleh:

Fathoni Akhmad Kurniawan

NIM. 2510817210013

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
2026**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM ALGORITMA & STRUKTUR
DATA

Laporan Praktikum Algoritma & Struktur Data

Modul 1 : Sturck and Point

Modul 2 : Stack and Queue

Modul3 : Linked List

Modul 4 : Sorting

Modul 5 : Searching

Modul 6 : Graph and tree

ini disusun sebagai syarat lulus mata kuliah Praktikum Praktikum Algoritma & Struktur Data. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Fathoi Akhmad Kurniawan

NIM : 2510817310011

Menyetujui,

Asisten Praktikum

Mengetahui,

Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani

NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom,

Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR.....	7
DAFTAR TABEL.....	8
MODUL 1: STRUCT DAN POINTER.....	1
SOAL 1	1
A. Source Code	1
B. Output Program	4
C. Pembahasan.....	4
SOAL 2.....	6
A. Source Code	6
B. Output Program	7
C. Pembahasan.....	8
Modul 2: STACK And Queue SOAL 1	9
A. Pembahasan.....	9
SOAL 2.....	12
A. Source Code	16
B. Output Program	19
C. Pembahasan.....	19
Soal 3	21
A. Pembahasan.....	21
SOAL 4: Si Palui dan Deretan Angka.....	23
A. Source Code	16
B. Output Program	20
C. Pembahasan.....	20
MODUL 3: Linked List SOAL 1	23
A. Soruce Code	23
B. Pembahasan.....	27

Soal 2: Si Palui dan Relikui Siklus Resonansi.....	29
A. Soruce Code	30
B. Output Program	31
C. Pembahasan.....	31
SOAL 3: Implementasi Senarai Berantai Ganda Mel-ingkar	33
A. Soruce Code	33
B. Pembahasan.....	38
SOAL 4.....	42
A. Soruce Code	42
B. Output Program	44
C. Pembahasan.....	44
MODUL 4: SORTING	47
Buble Sorting	47
A. Source Code	47
B. Pembahasan.....	47
C. Analisis.....	49
2. Krakteristik Algoritma	50
Selection Sort	52
A. Source Code	52
B. Pembahasan.....	52
C. Analisis.....	54
Insertion Sort.....	57
A. Source Code	57
B. Pembahasan.....	57
C. Analisis.....	59
Merge Sort	61
A. Source Code	61
B. Pembahasan.....	62
C. Analisis.....	65
Quick Sort	68
A. Source Code	68
B. Pembahasan.....	68

C. Analisis.....	71
Radix Sort	73
A. Source Code	73
B. Pembahasan.....	74
C. Analisis.....	76
ANALISIS PERFORMA.....	79
MODUL 5: SORTING	81
Linear Searching	81
A. Source Code	81
B. Pembahasan.....	81
C. Analisis.....	82
1. Kompleksitas Waktu	82
2. karakteristik Algoritma	83
3. analisis program	83
Binary Searching.....	84
A. Source Code	84
B. Pembahasan.....	84
C. Analisis.....	86
1. Kompleksitas waktu	86
2. Karakteristik algoritma.....	86
3. analisis program	87
ANALISIS PERFORMA.....	88
MODUL 6: Graph and Tree.....	89
Soal 1: Konversi Adjacency List ke Adjacency Matrix.....	89
Deskripsi Soal.....	89
A. Source Code	91
B. Output Program	92
C. Pembahasan.....	92
D. Analisis.....	94
Soal 2: Konversi Adjacency Matrix ke Adjacency List	95
A. Source Code	97
B. Output Program	98

C. Pembahasan.....	98
D. Analisis.....	101
Soal 3: BFS: Jarak Terpendek Keluar Labirin	102
A. Source Code	104
B. Ouput Program	105
C. Pembahasan.....	105
D. Analisis.....	108
Soal 4 Menghitung Semua Jalur Keluar Labirin	109
A. Source Code	111
B. Ouput Program	112
C. Pembahasan.....	112
D. Analisis.....	116
Soal 5: Tree Traversal.....	118
Deskripsi Soal.....	118
Format Input.....	118
Keterangan:	118
Format Output	118
Contoh Kasus.....	119
Penjelasan Contoh	119
A. Source Code	120
B. Output Program	122
C. Pembahasan.....	122
D. Analisis.....	126
TAUTAN GITHUB.....	127

DAFTAR GAMBAR

Gambar 1 . <i>ScreenShot Hasil Jawaban Soal 1A Modul 1</i>	4
Gambar 2. <i>ScreenShot Hasil Jawaban Soal 1B Modul 1</i>	4
Gambar 3 <i>Screenshot Hasil Jawaban Soal 1C Modul 1</i>	4
Gambar 4. <i>Screenshot Hasil Jawaban Soal 2A Modul 1</i>	7
Gambar 5 <i>Screenshot Hasil Jawaban Soal 2B Modul 1</i>	7
Gambar 6. <i>Screenshot Hasil Jawaban Soal 2C Modul 1</i>	7
Gambar 7. <i>Screenshot Hasil Jawaban Soal 2D Modul 1</i>	8
Gambar 8 <i>Screenshot soal 1A Modul 2</i>	19
Gambar 9. <i>Screenshot soal 1B Modul 2</i>	19
Gambar 10 <i>Screenshot soal 1C Modul 2</i>	19
Gambar 11. <i>Screenshot soal 2A Modul 2</i>	20
Gambar 12. <i>Screenshot soal 2B Modul 2</i>	20
Gambar 13. <i>Screenshot soal 2C Modul 2</i>	20
Gambar 14. <i>ScreenShot Ouput Program Soal 1A Modul 3</i>	31
Gambar 15. <i>ScreenShot Ouput Program Soal 1B Modul 3</i>	31
Gambar 16. <i>ScreenShot Ouput Program Soal 2A Modul 3</i>	44
Gambar 17. <i>ScreenShot Output Program Soal 2B Modul 3</i>	44
Gambar 18. <i>Output main.cpp (Bubble Sort) Modul 4</i>	49
Gambar 19. <i>Output main.cpp (Selection Sort) Modul 4</i>	54
Gambar 20. <i>Output main.cpp (Insertion Sort) Modul 4</i>	59
Gambar 21. <i>Output main.cpp (Merge Sort) Modul 4</i>	65
Gambar 22. <i>Output main.cpp (Quick Sort) Modul 4</i>	71
Gambar 23 <i>Output main.cpp (Radix Sort) Modul 4</i>	76
Gambar 24. <i>Ouput Main.cpp(linear Search) Modul 5</i>	82
Gambar 25. <i>Output main.cpp(Binary Searching) Modul 5</i>	86
Gambar 26 <i>Output Soal1.cpp Modul 6</i>	92
Gambar 27. <i>Ouput Soal2.cpp Modul 6</i>	98
Gambar 28. <i>Ouput Soal3.cpp Modul 6</i>	105
Gambar 29. <i>Ouput Soal4.cpp Modul 6</i>	112
Gambar 30. <i>Ouput Soal5.cpp Modul 6</i>	122

DAFTAR TABEL

<i>Table 1 Source Code Line.h</i>	<i>2</i>
<i>Table 2. Source Code Point.h Modul 1</i>	<i>3</i>
<i>Table 3. Source Code Prob1.cpp Modul 1</i>	<i>3</i>
<i>Table 4. Source Code circle.h Modul 1.....</i>	<i>6</i>
<i>Table 5. Source Code prob2.cpp Modul 1</i>	<i>7</i>
<i>Table 6. Source Code Prob1.cpp Modul 2.....</i>	<i>16</i>
<i>Table 7. Source Code Stack.cpp Modul 2</i>	<i>17</i>
<i>Table 8. Source Code prob2.cpp Modul 2</i>	<i>16</i>
<i>Table 9. Source Code queue.cpp Modul 2</i>	<i>16</i>
<i>Table 10. Source Code Queue.h Modul 2</i>	<i>20</i>
<i>Table 11. Source Code Single_Linked_list.h Modul 3.....</i>	<i>23</i>
<i>Table 12. Source Code Single_LInked_list.cpp Modul 3.....</i>	<i>24</i>
<i>Table 13. prob_a.cpp</i>	<i>30</i>
<i>Table 14. Source Code Double_Linked_list.h Modul 3</i>	<i>33</i>
<i>Table 15. Source Code prob_b.cpp Modul 3</i>	<i>42</i>
<i>Table 16. Source Code sorting_algorithms.cpp (Buble Sort) Modul 4.....</i>	<i>47</i>
<i>Table 17. Source Code sorting_algorithms.cpp (Selection Sort) Modul 4</i>	<i>52</i>
<i>Table 18. Source Code sorting_algorithms.cpp (Insertion Sort) Modul 4.....</i>	<i>57</i>
<i>Table 19. Source Code sorting_algorithms.cpp (Merge Sort) Modul 4.....</i>	<i>61</i>
<i>Table 20. Source Code sorting_algorithms.cpp (Quick Sort) Modul 4</i>	<i>68</i>
<i>Table 21. Source Code sorting_algorithms.cpp (Radix Sort) Modul 4.....</i>	<i>73</i>
<i>Table 22. Source Code searching_algorithms (Linear Searching) Modul 5</i>	<i>81</i>
<i>Table 23. Source Code searching_algorithms (Binary Searching) Modul 5</i>	<i>84</i>
<i>Table 24 Source Code Soal1.cpp Modul 6.....</i>	<i>91</i>
<i>Table 25. Source Code Soal2.cpp Modul 6.....</i>	<i>97</i>
<i>Table 26. Source Code Soal3.cpp Modul 6.....</i>	<i>104</i>
<i>Table 27. Source Code Soal4.cpp Modul 6.....</i>	<i>111</i>
<i>Table 28. Source Code Soal5.cpp Modul 6.....</i>	<i>120</i>

MODUL 1: STRUCT DAN POINTER

SOAL 1

Diberikan sebuah garis l yang direpresentasikan dalam bentuk persamaan umum:

$$ax + by + c = 0$$

dan sebuah titik $P = (x_P, y_P)$. Tentukan posisi titik P terhadap garis l dengan cara mengimplementasikan fungsi `gradient` dan `CheckPointPosition` pada file header yang telah diberikan.

Input	Output
1 0 -3 5 0	Left
1 0 -3 1 7	Right
2 -1 -1 1 1	On Line

A. Source Code

Table 1 Source Code Line.h

```
1  #ifndef      LINE_H
2  #define LINE_H
3
4  #include     "point.h"
5  #include <string>
6
7  struct Line {
8      double a, b, c;
9  };
10
11 double Equality(const Line *l, const Point *p){
12     double hasil = (l->a * p->x) + (l->b * p->y) + l->c;
13     return hasil;
14 }
15 std::string CheckPointPosition(double hasil){
16     if(hasil < 0){
17         return "right";
18     }
19     else if(hasil == 0){
20         return "On Line";
21     }
22     else{
23         return "left";
24     }
25 }
26 #endif
```

Table 2. Source Code Point.h Modul 1

1.	#ifndef POINT_H
2.	#define POINT_H
3.	
4.	struct Point {
5.	double x, y;
6.	};
7.	
8.	#endif

Table 3. Source Code Probl.cpp Modul 1

1.	#include "line.h"
2.	#include <iostream>
3.	using namespace std;
4.	
5.	int main(){
6.	Line l;
7.	Point p;
8.	
9.	cout << "Masukkan nilai a, b, dan c: ";
10.	cin >> l.a >> l.b >> l.c;
11.	
12.	cout << "Masukkan nilai xp dan yp: ";
13.	cin >> p.x >> p.y;
14.	
15.	double hitungan = Equality(&l, &p);
16.	string hasil = CheckPointPosition(hitungan);
17.	cout << hasil << endl;
18.	return 0;
19.	}

B. Output Program

```
PS D:\programming stuff\tugas> cd "d:\programming stuff\tugas\ALGO\PRAK\" ; if ($?) { g++ prob1.cpp -o prob1 } ; if ($?) { .\prob1 }  
Masukkan nilai a, b, dan c: 1 0 -3 5 0  
Masukkan nilai xp dan yp: left
```

Gambar 1 . *ScreenShot Hasil Jawaban Soal 1A Modul 1*

```
PS D:\programming stuff\tugas\ALGO\PRAK> cd "d:\programming stuff\tugas\ALGO\PRAK\" ; if ($?) { g++ prob1.cpp -o prob1 } ; if ($?) { .\prob1 }  
Masukkan nilai a, b, dan c: 1 0 -3 1 7  
Masukkan nilai xp dan yp: right
```

Gambar 2. *ScreenShot Hasil Jawaban Soal 1B Modul 1*

```
PS D:\programming stuff\tugas\ALGO\PRAK> cd "d:\programming stuff\tugas\ALGO\PRAK\" ; if ($?) { g++ prob1.cpp -o prob1 } ; if ($?) { .\prob1 }  
Masukkan nilai a, b, dan c: 2 -1 -1 1 1  
Masukkan nilai xp dan yp: On Line
```

Gambar 3 *Screenshot Hasil Jawaban Soal 1C Modul 1*

C. Pembahasan

Program ini dirancang untuk menentukan letak suatu titik terhadap sebuah garis lurus berdasarkan persamaan umum ($ax + by + c = 0$). Dalam pembuatannya, program dibagi menjadi beberapa file agar setiap bagian memiliki fungsi yang jelas. File `point.h` digunakan untuk mendefinisikan struktur data titik, `line.h` berisi struktur data garis beserta fungsi-fungsi yang diperlukan, sedangkan `prob1.cpp` berperan sebagai program utama yang mengatur jalannya proses.

Pada saat program dijalankan, pengguna diminta memasukkan nilai koefisien garis, yaitu (a), (b), dan (c), kemudian memasukkan koordinat titik yang akan diuji. Data tersebut selanjutnya diproses menggunakan fungsi `Equality`, yang menghitung hasil substitusi koordinat titik ke dalam persamaan garis. Nilai yang diperoleh dari proses ini menjadi dasar dalam menentukan posisi titik.

Selanjutnya, hasil perhitungan dikirim ke fungsi `CheckPointPosition` untuk dianalisis. Fungsi ini membandingkan nilai yang diperoleh dengan nol. Jika hasilnya negatif, program menyatakan bahwa titik berada di sisi kanan garis. Jika hasilnya sama dengan nol, berarti titik tepat berada pada garis. Sementara itu, apabila hasilnya positif, titik dinyatakan berada di sisi kiri garis.

Setelah proses penentuan posisi selesai dilakukan, program menampilkan hasilnya kepada pengguna melalui layar. Dengan demikian, program mampu menentukan posisi relatif suatu titik terhadap garis berdasarkan perhitungan matematis yang dilakukan. Selain menghasilkan keluaran yang sesuai, program ini juga menerapkan konsep pemrograman modular, penggunaan struktur (*struct*), fungsi, pointer, dan

percabangan sehingga kode menjadi lebih terstruktur serta mudah dipahami dan dikembangkan.

Dari sisi efisiensi, program ini memiliki kompleksitas waktu (Time Complexity) sebesar $O(1)$ karena seluruh proses yang dilakukan, mulai dari menerima masukan, menghitung nilai substitusi pada persamaan garis, hingga menentukan posisi titik menggunakan percabangan, hanya terdiri dari operasi-operasi dengan jumlah tetap yang tidak bergantung pada banyaknya data masukan. Sementara itu, kompleksitas ruang (Space Complexity) juga sebesar $O(1)$ karena program hanya menggunakan sejumlah variabel dan objek dengan ukuran tetap, yaitu satu objek Line, satu objek Point, serta beberapa variabel bantu, tanpa memerlukan alokasi memori tambahan yang bertambah seiring ukuran input.

SOAL 2

Diberikan sebuah lingkaran C dengan pusat (c_x, c_y) dan jari-jari $r > 0$, serta sebuah titik $P = (p_x, p_y)$. Tentukan posisi titik P terhadap lingkaran C dan mengimplementasikan fungsi `distance` dan `CheckPointInCircle` pada file header yang diberikan.

Input	Output
0 0 5 3 4	On Circle
0 0 5 1 1	Inside
0 0 5 4 4	Outside
-2 -2 4 2 -2	On Circle

A. Source Code

Table 4. Source Code circle.h Modul 1

1.	<code>#ifndef CIRCLE_H</code>
2.	<code>#define CIRCLE_H</code>
3.	
4.	<code>#include <cmath></code>
5.	<code>#include "point.h"</code>
6.	<code>#include <string></code>
7.	
8.	<code>struct Circle {</code>
9.	<code> Point center;</code>
10.	<code> double radius;</code>
11.	<code>};</code>
12.	
13.	<code>double distance(const Circle * c, const Point * p){</code>
14.	<code> return std::pow(p->x - c->center.x, 2) +</code>
	<code>std::pow(p->y - c->center.y, 2);</code>
15.	<code>}</code>
16.	<code>std::string CheckPointInCircle(double distance, double</code>
	<code>r2){</code>
17.	<code> double epsilon = 1e-6;</code>
18.	<code> if(distance < r2){</code>
19.	<code> return "Inside";</code>

20.	}
21.	else if(std::abs(distance - r2) < epsilon){
22.	}
23.	else{
24.	return "Outside";
25.	}
26.	}
27.	
28.	#endif

Table 5. Source Code prob2.cpp Modul 1

1.	#include "circle.h"
2.	#include <iostream>
3.	using namespace std;
4.	
5.	int main(){
6.	Circle c;
7.	Point p;
8.	cout << "Masukkan nilai Cx dan Cy: ";
9.	cin >> c.center.x >> c.center.y;
10.	cout << "Masukkan nilai radius: ";
11.	cin >> c.radius;
12.	cout << "Masukkan nilai Xp dan Yp: ";
13.	cin >> p.x >> p.y;
14.	double Jari2 = c.radius * c.radius;
15.	double jarak = distance(&c, &p);
16.	string hasil = CheckPointInCircle(jarak, Jari2);
17.	cout << hasil << endl;
18.	return 0;
19.	
20.	}

B. Output Program

```
PS D:\programming stuff\tugas\ALGO\PRAK> cd "d:\programming stuff\tugas\ALGO\New folder\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) { .\prob2 }
Masukkan nilai Cx dan Cy: 0 0 5 3 4
Masukkan nilai radius: Masukkan nilai Xp dan Yp: On Circle
```

Gambar 4. Screenshot Hasil Jawaban Soal 2A Modul 1

```
PS D:\programming stuff\tugas\ALGO\New folder> cd "d:\programming stuff\tugas\ALGO\New folder\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) { .\pr
ob2 }
Masukkan nilai Cx dan Cy: 0 0 5 1 1
Masukkan nilai radius: Masukkan nilai Xp dan Yp: Inside
```

Gambar 5 Screenshot Hasil Jawaban Soal 2B Modul 1

```
PS D:\programming stuff\tugas\ALGO\New folder> cd "d:\programming stuff\tugas\ALGO\New folder\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) { .\pr
ob2 }
Masukkan nilai Cx dan Cy: 0 0 5 4 4
Masukkan nilai radius: Masukkan nilai Xp dan Yp: Outside
```

Gambar 6. Screenshot Hasil Jawaban Soal 2C Modul 1

```
PS D:\programming_stuff\tugas\ALGO\New folder> cd "d:\programming_stuff\tugas\ALGO\New folder\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) { .\pr
ob2 }
Masukkan nilai Cx dan Cy: -2 -2 4 2 -2
Masukkan nilai radius: Masukkan nilai Xp dan Yp: On Circle
```

Gambar 7. Screenshot Hasil Jawaban Soal 2D Modul 1

C. Pembahasan

Program ini dibuat untuk menentukan posisi suatu titik terhadap sebuah lingkaran berdasarkan koordinat pusat dan panjang jari-jari yang diberikan oleh pengguna. Program disusun secara modular dengan memisahkan definisi struktur data dan fungsi ke dalam file `circle.h`, sedangkan proses utama dijalankan pada file `prob2.cpp`. Pembagian ini membuat kode menjadi lebih terorganisasi, mudah dipelajari, serta memudahkan pengembangan program di kemudian hari.

Pada saat program dijalankan, pengguna diminta memasukkan koordinat pusat lingkaran, nilai jari-jari, serta koordinat titik yang akan diperiksa. Data tersebut kemudian digunakan untuk menghitung jarak antara titik dan pusat lingkaran menggunakan fungsi `distance`. Perhitungan dilakukan tanpa menggunakan akar kuadrat sehingga yang dibandingkan adalah kuadrat jarak dengan kuadrat jari-jari. Cara ini menghasilkan proses yang lebih efisien tanpa mengubah hasil perbandingan.

Nilai yang diperoleh dari fungsi `distance` selanjutnya diproses oleh fungsi `CheckPointInCircle` untuk menentukan posisi titik terhadap lingkaran. Apabila nilai kuadrat jarak lebih kecil daripada kuadrat jari-jari, maka titik berada di dalam lingkaran (*Inside*). Jika nilainya sama atau sangat mendekati kuadrat jari-jari, titik dinyatakan berada tepat pada keliling lingkaran (*On Circle*). Sebaliknya, apabila kuadrat jarak lebih besar daripada kuadrat jari-jari, maka titik berada di luar lingkaran (*Outside*).

Setelah proses pemeriksaan selesai, program menampilkan hasil penentuan posisi titik kepada pengguna melalui terminal. Secara keseluruhan, program telah menerapkan konsep pemrograman modular, penggunaan struktur (*struct*), fungsi, pointer, percabangan, serta operasi matematika dalam bahasa C++. Melalui implementasi tersebut, program mampu menentukan posisi relatif suatu titik terhadap lingkaran secara tepat dan efisien.

Berdasarkan analisis efisiensi, program ini memiliki kompleksitas waktu (Time Complexity) sebesar $O(1)$. Hal ini disebabkan seluruh proses, seperti membaca masukan, menghitung kuadrat jarak titik terhadap pusat lingkaran, membandingkannya dengan kuadrat jari-jari, serta menentukan posisi titik, dilakukan dalam jumlah operasi yang tetap dan tidak dipengaruhi oleh ukuran data masukan. Adapun kompleksitas ruang (Space Complexity) juga sebesar $O(1)$ karena program hanya menggunakan beberapa variabel, yaitu satu objek `Circle`, satu objek `Point`, dan beberapa variabel pendukung, sehingga kebutuhan memorinya tetap dan tidak bertambah selama program dijalankan.

Modul 2: STACK And Queue

SOAL 1

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Pembahasan

Fungsi `init` bertujuan untuk menginisialisasi stack ke kondisi awal, yaitu kosong. Algoritmanya sangat sederhana, hanya dengan mengatur pointer `top` menjadi `nullptr`. Hal ini menandakan bahwa belum ada elemen dalam stack. Kebenaran algoritma ini didasarkan pada invariant bahwa stack kosong selalu direpresentasikan dengan `top == nullptr`, sehingga fungsi lain seperti `isEmpty` dapat bekerja dengan konsisten. Kompleksitas waktu dan ruang dari fungsi ini adalah $O(1)$ karena hanya melakukan satu operasi sederhana tanpa alokasi tambahan.

Fungsi `isEmpty` digunakan untuk mengecek apakah stack dalam kondisi kosong. Algoritmanya hanya membandingkan apakah `top` bernilai `nullptr`. Jika iya, maka stack kosong. jika tidak, berarti terdapat elemen di dalamnya. Kebenaran fungsi ini mengikuti definisi struktur stack yang menggunakan pointer sebagai penanda elemen teratas. Karena hanya melakukan satu perbandingan, kompleksitas waktu fungsi ini adalah $O(1)$ dan tidak membutuhkan ruang tambahan sehingga kompleksitas ruangnya juga $O(1)$.

Fungsi `isFull` bertujuan untuk mengecek apakah stack sudah penuh. Karena stack diimplementasikan menggunakan array dengan ukuran tetap, kondisi penuh terjadi ketika pointer `top` sudah menunjuk ke elemen terakhir dalam array. Algoritmanya membandingkan `top` dengan alamat elemen terakhir `data[MAX-1]`. Jika sama, berarti stack tidak dapat menerima elemen baru. Kebenaran algoritma ini berasal dari sifat array statis yang memiliki batas kapasitas tetap. Kompleksitas waktu dan ruang fungsi ini adalah $O(1)$ karena hanya melakukan satu operasi perbandingan.

Fungsi push digunakan untuk menambahkan elemen baru ke dalam stack. Algoritma dimulai dengan mengecek apakah stack penuh menggunakan `isFull`. jika penuh, maka akan melempar exception. Jika tidak penuh, maka terdapat dua kondisi:

jika stack kosong, top akan diarahkan ke elemen pertama array

jika tidak kosong, pointer top akan dinaikkan satu posisi.

Setelah itu, nilai baru disimpan pada posisi top. Kebenaran algoritma ini didasarkan pada prinsip LIFO (Last In First Out), di mana elemen terakhir yang dimasukkan selalu berada di posisi teratas. Kompleksitas waktu fungsi ini adalah $O(1)$ karena hanya melibatkan operasi pointer dan assignment, serta kompleksitas ruangnya $O(1)$.

Fungsi pop berfungsi untuk menghapus dan mengembalikan elemen teratas dari stack. Algoritmanya dimulai dengan mengecek apakah stack kosong; jika kosong, maka akan melempar exception. Jika tidak kosong, nilai pada top disimpan terlebih dahulu. Kemudian, jika elemen tersebut adalah satu-satunya elemen, maka top diatur kembali menjadi `nullptr`. Jika masih ada elemen lain, pointer top akan diturunkan satu posisi. Nilai yang disimpan kemudian dikembalikan. Kebenaran algoritma ini terletak pada penerapan prinsip LIFO, di mana elemen terakhir yang masuk adalah yang pertama keluar. Kompleksitas waktu fungsi ini adalah $O(1)$ dan kompleksitas ruangnya juga $O(1)$.

Fungsi peek digunakan untuk melihat elemen teratas stack tanpa menghapusnya. Algoritmanya adalah dengan mengecek apakah stack kosong; jika kosong, akan melempar exception. Jika tidak, fungsi langsung mengembalikan nilai yang ditunjuk oleh top. Kebenaran algoritma ini jelas karena tidak mengubah struktur stack dan hanya membaca nilai teratas. Kompleksitas waktu dan ruang fungsi ini adalah $O(1)$ karena hanya melakukan satu operasi akses.

Fungsi HasilOperasi digunakan untuk menghitung hasil dari ekspresi dalam bentuk Reverse Polish Notation (RPN). Algoritmanya melakukan iterasi terhadap setiap elemen input. Jika elemen berupa operator, maka dua elemen teratas di-pop dari stack, kemudian dilakukan operasi sesuai operator tersebut, dan hasilnya di-push kembali ke stack. Jika elemen berupa angka, maka dikonversi ke integer menggunakan `stoi`, divalidasi agar berada dalam rentang tertentu, lalu dimasukkan ke dalam stack. Setelah seluruh input diproses, hasil akhir diambil dari stack. Kebenaran algoritma ini didasarkan pada metode standar evaluasi postfix yang memanfaatkan struktur stack untuk memastikan urutan operasi tetap benar.

Kompleksitas waktu fungsi ini adalah $O(n)$ karena memproses setiap elemen sekali, sedangkan kompleksitas ruangnya adalah $O(n)$ karena stack dapat menyimpan hingga n elemen

SOAL 2

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya: ▸ bilangan bulat
▸ operator: +, -, *, /

Format Input

n A_1 A_2
... A_n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol

- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Table 6. Source Code Prob1.cpp Modul 2

1.	<code>#include <iostream></code>
2.	<code>#include <string></code>
3.	<code>#include "stack.h"</code>
4.	<code>using namespace std;</code>
5.	
6.	<code>int main() {</code>
7.	<code> Stack s;</code>
8.	<code> init(&s);</code>
9.	<code> string inputData[MAX];</code>
10.	
11.	<code> int n;</code>
12.	
13.	<code> if (!(cin >> n)) {</code>
14.	<code> return 0;</code>

15.	}
16.	for (int i = 0; i < n && i < MAX; i++) {
17.	cin >> inputData[i];
18.	}
19.	int hasil = HasilOperasi(&s, n, inputData);
20.	cout << hasil << endl;
21.	return 0;
22.	}

Table 7. Source Code Stack.cpp Modul 2

1.	#include "stack.h"
2.	#include <iostream>
3.	#include <string>
4.	
5.	using namespace std;
6.	
7.	void init(Stack* s) {
8.	s->top = nullptr;
9.	}
10.	
11.	bool isEmpty(const Stack* s) {
12.	return s->top == nullptr;
13.	}
14.	
15.	bool isFull(const Stack* s) {
16.	return s->top == &(s->data[MAX - 1]);
17.	}
18.	
19.	void push(Stack* s, int value) {
20.	if (isFull(s))
21.	throw "Stack penuh";
22.	
23.	if (isEmpty(s)) {
24.	s->top = s->data;
25.	} else {
26.	s->top++;
27.	}
28.	
29.	*s->top = value;
30.	}
31.	

```

32. int pop(Stack* s) {
33.     if (isEmpty(s))
34.         throw "Stack kosong";
35.
36.     int value = *s->top;
37.
38.     if (s->top == s->data) {
39.         s->top = nullptr;
40.     } else {
41.         s->top--;
42.     }
43.
44.     return value;
45. }
46.
47. int peek(const Stack* s) {
48.     if (isEmpty(s))
49.         throw "Stack kosong";
50.
51.     return *s->top;
52. }
53.
54. int HasilOperasi(Stack* s, int n, string input[]) {
55.     for (int i = 0; i < n; i++) {
56.         string val = input[i];
57.
58.         if (val == "+" || val == "-" || val == "*" ||
val == "/") {
59.             int bil2 = pop(s);
60.             int bil1 = pop(s);
61.             int hasil;
62.
63.             if (val == "+") {
64.                 hasil = bil1 + bil2;
65.             } else if (val == "-") {
66.                 hasil = bil1 - bil2;
67.             } else if (val == "*") {
68.                 hasil = bil1 * bil2;
69.             } else if (val == "/") {
70.                 if (bil2 == 0) {
71.                     hasil = 0;
72.                 } else {
73.                     hasil = bil1 / bil2;
74.                 }
75.             }
76.
77.             push(s, hasil);

```

```

78.         } else {
79.             int bil = stoi(val);
80.
81.             if (bil < -1000 || bil > 1000) {
82.                 throw "Bilangan diluar jangkauan";
83.             }
84.
85.             push(s, bil);
86.         }
87.     }
88.
89.     return isEmpty(s) ? 0 : pop(s);
90. }

```

B. Output Program

```

PS D:\programming_stuff\tugas\ALGO\PRAK\modul 2> cd "d:\programming_stuff\tugas\ALGO\PRAK\modul 2\" ; if ($?) { g++ prob1.cpp -o prob1 } ; if ($?) {
.\prob1 }
3
2 3 +
5

```

Gambar 8 Screenshot soal 1A Modul 2

```

PS D:\programming_stuff\tugas\ALGO\PRAK\modul 2> cd "d:\programming_stuff\tugas\ALGO\PRAK\modul 2\" ; if ($?) { g++ prob1.cpp -o prob1 } ; if ($?) {
.\prob1 }
9
5 1 2 + 4 *
+ 3 -
14

```

Gambar 9. Screenshot soal 1B Modul 2

```

PS D:\programming_stuff\tugas\ALGO\PRAK\modul 2> cd "d:\programming_stuff\tugas\ALGO\PRAK\modul 2\" ; if ($?) { g++ prob1.cpp -o prob1 } ; if ($?) {
.\prob1 }
5
10 2 / 3 *
15

```

Gambar 10 Screenshot soal 1C Modul 2

C. Pembahasan

Program ini dibuat untuk menghitung ekspresi matematika yang ditulis dalam bentuk Reverse Polish Notation (RPN) dengan bantuan struktur data stack. Di awal program ada beberapa library yang dipakai, yaitu `#include <iostream>`, `#include <string>`, dan `#include "stack.h"`. Library `<iostream>` dipakai untuk input dan output lewat `cin` dan `cout`, sedangkan `<string>` dipakai supaya program bisa mengolah data berbentuk teks. File `stack.h` ikut dimasukkan karena di dalamnya ada struktur `Stack` dan fungsi-fungsi yang dibutuhkan untuk menjalankan program. Selain itu, `using namespace std;` dipakai supaya penulisan kode jadi lebih singkat, jadi tidak perlu menulis `std::` setiap kali memakai fitur dari library standar.

Saat program dijalankan, fungsi `main()` akan membuat objek `Stack` lalu mengaktifkannya dengan fungsi `init()`. Fungsi ini membuat `top` bernilai `nullptr`, artinya stack masih kosong dan siap dipakai. Setelah itu, program membaca jumlah token yang akan diproses, lalu menyimpan semua token itu ke dalam array `inputData`. Token ini bisa berupa angka atau operator seperti `+`, `-`, `*`, dan `/`. Kalau proses membaca jumlah token gagal, program langsung dihentikan supaya tidak terjadi error di langkah berikutnya.

Semua pengaturan stack dilakukan lewat fungsi-fungsi yang ada di `stack.cpp`. Fungsi `isEmpty()` dipakai untuk mengecek apakah stack masih kosong dengan melihat nilai `top`. Fungsi `isFull()` dipakai untuk memastikan apakah stack sudah penuh atau belum. Lalu ada `push()` yang berfungsi menambahkan data baru ke bagian paling atas stack, dan `pop()` yang berfungsi mengambil sekaligus menghapus data paling atas dari stack. Sebelum `push()` atau `pop()` dijalankan, program akan mengecek dulu apakah kondisi stack memang memungkinkan. Selain itu, ada juga fungsi `peek()` yang dipakai untuk melihat isi paling atas stack tanpa menghapusnya.

Bagian utama program ada di fungsi `HasilOperasi()`. Di sini, ekspresi postfix dibaca satu per satu. Kalau token yang dibaca berupa angka, maka angka itu diubah dulu dari string ke integer dengan `stoi`, lalu dicek apakah nilainya masih sesuai batas yang ditentukan. Kalau sudah aman, angka tersebut dimasukkan ke stack dengan `push()`. Kalau token yang dibaca berupa operator, program akan mengambil dua angka paling atas dari stack dengan `pop()`. Angka yang diambil pertama dipakai sebagai operand kedua, sedangkan angka yang diambil berikutnya dipakai sebagai operand pertama. Setelah itu, kedua angka tersebut dihitung sesuai operatornya, misalnya dijumlahkan, dikurangkan, dikalikan, atau dibagi. Hasilnya lalu dimasukkan lagi ke stack supaya bisa dipakai kalau masih ada operasi berikutnya. Untuk pembagian, program juga mengecek dulu supaya tidak ada pembagian dengan nol.

Setelah semua token selesai diproses, angka yang tersisa di stack adalah hasil akhir dari ekspresi tersebut. Nilai ini kemudian dikembalikan ke fungsi `main()` dan ditampilkan ke pengguna lewat `cout`. Dengan cara kerja stack yang bersifat Last In First Out (LIFO), program bisa menghitung ekspresi postfix dengan benar tanpa perlu tanda kurung atau aturan prioritas operator.

Kalau dilihat dari cara kerjanya, program ini butuh waktu $O(n)$ karena setiap token hanya diproses satu kali. Untuk penggunaan memori, program juga butuh ruang $O(n)$ karena stack dan array penyimpanan token mengikuti banyaknya data yang diproses. Sementara itu, fungsi-fungsi dasar seperti `init()`, `isEmpty()`, `isFull()`, `push()`, `pop()`, dan `peek()` bekerja sangat cepat karena masing-masing hanya melakukan satu tugas sederhana, jadi waktunya tetap $O(1)$.

Soal 3

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Pembahasan

`init(Queue* q)` Fungsi ini menginisialisasi queue dengan mengatur pointer `front` dan `rear` ke `nullptr`, yang menandakan queue kosong. Algoritmanya sederhana dan selalu benar karena kondisi kosong didefinisikan dengan `front == nullptr`. Kompleksitas waktunya $O(1)$ dan ruangnya $O(1)$.

`isEmpty(const Queue* q)` Fungsi ini mengecek apakah queue kosong dengan membandingkan `front` terhadap `nullptr`. Jika ya, maka queue kosong. Ini benar karena tidak ada elemen yang ditunjuk. Kompleksitas waktu dan ruang adalah $O(1)$.

`isFull(const Queue* q)` Fungsi ini menentukan apakah queue penuh dengan konsep circular queue, yaitu saat posisi berikutnya dari `rear` sama dengan `front`. Ini memastikan tidak terjadi overflow. Algoritma ini benar karena memanfaatkan sifat melingkar array. Kompleksitas waktu dan ruang adalah $O(1)$.

`enqueue(Queue* q, int value)` Fungsi ini menambahkan elemen ke belakang queue. Jika penuh, akan melempar exception. Jika kosong, `front` dan `rear` diarahkan ke elemen pertama; jika tidak, `rear` digeser secara melingkar lalu nilai dimasukkan. Ini menjaga urutan FIFO. Kompleksitas waktu $O(1)$ dan ruang $O(1)$.

`dequeue(Queue* q)` Fungsi ini menghapus elemen dari depan queue. Jika kosong, akan melempar exception. Jika hanya satu elemen, queue dikosongkan kembali; jika lebih, `front` digeser maju. Ini benar karena selalu menghapus elemen pertama (FIFO). Kompleksitas waktu $O(1)$ dan ruang $O(1)$.

`front(const Queue* q)` Fungsi ini mengembalikan nilai elemen paling depan tanpa menghapusnya. Jika queue kosong, akan melempar exception. Ini benar karena `front` selalu menunjuk elemen pertama. Kompleksitas waktu dan ruang adalah $O(1)$.

`back(const Queue* q)` Fungsi ini mengembalikan nilai elemen paling belakang. Jika queue kosong, akan melempar exception. Ini benar karena `rear` selalu menunjuk elemen terakhir yang dimasukkan. Kompleksitas waktu dan ruang adalah $O(1)$.

`hasilQueue(int n, int k, int inputData[])` Fungsi ini menghitung jumlah setiap subarray berukuran k menggunakan queue. Untuk setiap posisi, queue diisi k elemen lalu dijumlahkan dengan proses dequeue. Algoritma ini benar karena setiap window dihitung secara lengkap. Kompleksitas waktunya $O(n \cdot k)$ dan ruangnya $O(k)$.

SOAL 4: Si Palui dan Deretan Angka

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n k

A_1 A_2 ... A_n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah Format Output Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 8. Source Code prob2.cpp Modul 2

1	#include <iostream>
2	#include "queue.h"
3	
4	using namespace std;
5	
6	int main() {
7	int n, k;
8	if (!(cin >> n >> k)) return 0;
9	int inputData[MAX];
10	for (int i = 0; i < n && i < MAX; i++) {
11	cin >> inputData[i];
12	}
13	hasilQueue(n, k, inputData);
14	return 0;
15	}

Table 9. Source Code queue.cpp Modul 2

1	#include "queue.h"
2	#include <iostream>
3	using namespace std;
4	
5	void init(Queue* q) {
6	q->front = nullptr;
7	q->rear = nullptr;
8	}
9	
10	bool isEmpty(const Queue* q) {
11	return q->front == nullptr;
12	}

```

13 bool isFull(const Queue* q) {
14     if (isEmpty(q)) return false;
15     int nextRearIdx = (q->rear - q->data + 1) % MAX;
16     return (q->data + nextRearIdx) == q->front;
17 }
18
19 void enqueue(Queue* q, int value) {
20     if (isFull(q)) throw "Queue penuh";
21     if (isEmpty(q)) {
22         q->front = q->data;
23         q->rear = q->data;
24     } else {
25         int nextRearIdx = (q->rear - q->data + 1) % MAX;
26         q->rear = q->data + nextRearIdx;
27     }
28     *(q->rear) = value;
29 }
30 void dequeue(Queue* q) {
31     if (isEmpty(q)) throw "Queue kosong";
32     if (q->front == q->rear) {
33         q->front = nullptr;
34         q->rear = nullptr;
35     } else {
36         int nextFrontIdx = (q->front - q->data + 1) %
37 MAX;
38         q->front = q->data + nextFrontIdx;
39     }
40 }
41
42 int front(const Queue* q) {
43     if (isEmpty(q)) throw "Queue kosong";
44     return *(q->front);
45 }
46 int back(const Queue* q) {
47     if (isEmpty(q)) throw "Queue kosong";
48     return *(q->rear);
49 }
50
51 void hasilQueue(int n, int k, int inputData[]){
52     Queue q;
53     for (int i = 0; i <= n - k; i++) {
54         init(&q);
55         for (int j = i; j < i + k; j++) {
56             enqueue(&q, inputData[j]);
57         }
58     }
59
60     long long currentSum = 0;
61     while (!isEmpty(&q)) {
62         currentSum += front(&q);
63         dequeue(&q);
64     }
65 }
66 cout << currentSum << (i == n - k ? "" : " ");

```

```
67 }  
68 cout << endl;  
69 }  
70  
71
```


Table 10. Source Code Queue.h Modul 2

1	#ifndef QUEUE_H
2	#define QUEUE_H
3	
4	#define MAX 100
5	
6	struct Queue {
7	int data[MAX];
8	int* front;
9	int* rear;
10	};
11	
12	void init(Queue* q);
13	bool isEmpty(const Queue* q);
14	bool isFull(const Queue* q);
15	void enqueue(Queue* q, int value);
16	void dequeue(Queue* q);
17	int front(const Queue* q);
18	int back(const Queue* q);
19	void hasilQueue(int n, int k, int inputData[]);
20	
21	#endif

B. Output Program

```
PS D:\programming stuff\tugas\ALGO\PRAK\modul 2> cd "d:\programming stuff\tugas\ALGO\PRAK\modul 2\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) {
.\prob2 }
5 3
1 2 3 4 5
6 9 12
```

Gambar 11. Screenshot soal 2A Modul 2

```
PS C:\Users\LOQ\OneDrive\Documents\Materi kuliah semester 2\algoritma\module-2-stack-and-queue-hafimuhammad-main> cd "c:\Users\LOQ\OneDrive\Documents\Materi
kuliah semester 2\algoritma\module-2-stack-and-queue-hafimuhammad-main\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) { .\prob2 }
5 5
1 2 3 4 5
15
```

Gambar 12. Screenshot soal 2B Modul 2

```
PS D:\programming stuff\tugas\ALGO\PRAK\modul 2> cd "d:\programming stuff\tugas\ALGO\PRAK\modul 2\" ; if ($?) { g++ prob2.cpp -o prob2 } ; if ($?) {
.\prob2 }
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 13. Screenshot soal 2C Modul 2

C. Pembahasan

Program ini dibuat untuk menghitung jumlah setiap subarray berukuran ****k**** dengan memanfaatkan struktur data ****queue**** yang diimplementasikan menggunakan ****circular queue****. Pada bagian awal program digunakan `#include <iostream>` untuk mendukung proses input dan output, serta `#include "queue.h"` yang berisi deklarasi struktur `Queue` beserta fungsi-fungsi yang diperlukan. Selain itu, `using namespace std;` digunakan agar penulisan kode menjadi lebih ringkas

tanpa perlu menambahkan awalan ``std::``.

Saat program dijalankan, fungsi ``main()`` mendeklarasikan variabel ``n`` dan ``k``, di mana ``n`` menyatakan jumlah data yang akan diproses, sedangkan ``k`` menyatakan ukuran setiap kelompok data atau subarray. Program kemudian membaca kedua nilai tersebut menggunakan ``cin``. Apabila proses pembacaan gagal, program akan langsung dihentikan menggunakan ``return 0`` agar tidak melanjutkan proses dengan data yang tidak valid. Selanjutnya dibuat array ``inputData`` untuk menyimpan seluruh data masukan. Melalui sebuah perulangan, setiap nilai dibaca dan disimpan ke dalam array hingga seluruh data berhasil dimasukkan atau mencapai kapasitas maksimum yang telah ditentukan.

Seluruh proses pengelolaan queue dilakukan melalui fungsi-fungsi yang terdapat pada ``queue.cpp``. Fungsi ``init()`` digunakan untuk menginisialisasi queue dengan mengatur pointer ``front`` dan ``rear`` menjadi ``nullptr``, sehingga queue berada pada kondisi kosong. Fungsi ``isEmpty()`` digunakan untuk memeriksa apakah queue masih kosong, sedangkan fungsi ``isFull()`` digunakan untuk memastikan apakah queue telah mencapai kapasitas maksimum. Pada implementasi ini digunakan konsep ****circular queue****, sehingga kondisi penuh terjadi ketika posisi berikutnya dari ``rear`` sama dengan posisi ``front``. Fungsi ``enqueue()`` digunakan untuk menambahkan elemen ke bagian belakang queue, sedangkan fungsi ``dequeue()`` digunakan untuk mengambil sekaligus menghapus elemen yang berada di bagian depan queue. Selain itu, fungsi ``front()`` digunakan untuk melihat nilai elemen paling depan tanpa menghapusnya, sedangkan fungsi ``back()`` digunakan untuk melihat nilai elemen paling belakang yang tersimpan di dalam queue.

Bagian utama program terdapat pada fungsi ``hasilQueue()``. Fungsi ini terlebih dahulu membuat sebuah objek ``Queue``, kemudian melakukan perulangan untuk setiap kemungkinan subarray berukuran ``k``. Pada setiap iterasi, queue diinisialisasi kembali agar berada pada kondisi kosong, kemudian sebanyak ``k`` elemen dari array ``inputData`` dimasukkan ke dalam queue menggunakan fungsi ``enqueue()``. Setelah queue terisi, program menghitung jumlah seluruh elemen di dalam queue dengan melakukan perulangan selama queue belum kosong. Pada setiap langkah, nilai elemen paling depan diambil menggunakan fungsi ``front()``, ditambahkan ke variabel ``currentSum``, kemudian elemen tersebut dihapus menggunakan fungsi ``dequeue()``. Setelah semua elemen pada queue selesai diproses, nilai ``currentSum`` ditampilkan ke layar. Langkah yang sama diulangi hingga seluruh subarray berukuran ``k`` selesai dihitung.

Berdasarkan analisis algoritma, fungsi-fungsi dasar seperti ``init()``, ``isEmpty()``, ``isFull()``, ``enqueue()``, ``dequeue()``, ``front()``, dan ``back()`` masing-masing memiliki ****kompleksitas waktu**

$O(1)$ karena hanya melakukan operasi dasar yang tidak bergantung pada banyaknya elemen di dalam queue. Kompleksitas ruang dari fungsi-fungsi tersebut juga $O(1)$ karena tidak memerlukan alokasi memori tambahan. Sementara itu, fungsi `hasilQueue()` memiliki kompleksitas waktu $O(n \times k)$ karena untuk setiap posisi subarray dilakukan proses terhadap sebanyak k elemen. Adapun kompleksitas ruangnya $O(k)$ karena queue hanya menyimpan maksimal k elemen pada satu waktu selama proses perhitungan berlangsung.

MODUL 3: Linked List

SOAL 1

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`. Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap. Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar

A. Source Code

Table 11. Source Code `Single_Linked_list.h` Modul 3

1.	<code>#ifndef SINGLE_LINKED_LIST</code>
2.	<code>#define SINGLE_LINKED_LIST</code>
3.	
4.	<code>struct Node {</code>
5.	<code> int data;</code>
6.	<code> Node * next;</code>
7.	<code>};</code>
8.	
9.	<code>struct SingleLinkedList {</code>
10.	<code> Node *head = nullptr, *tail = nullptr;</code>
11.	<code> int size = 0;</code>
12.	
13.	<code> void init();</code>
14.	<code> bool is_empty();</code>
15.	
16.	<code> void add_front(int val);</code>
17.	<code> void add_back(int val);</code>
18.	<code> void add_idx(int val, int idx);</code>
19.	
20.	<code> void delete_front();</code>
21.	<code> void delete_back();</code>
22.	<code> void delete_idx(int idx);</code>
23.	
24.	<code> void display();</code>
25.	<code> int get(int idx);</code>
26.	<code> void set(int val, int idx);</code>
27.	
28.	<code> void clear();</code>
29.	<code>};</code>
30.	

31.	#endif
-----	--------

Table 12. Source Code Single_Linked_list.cpp Modul 3

1.	#include "single_linked_list.h"
2.	#include <iostream>
3.	#include <stdexcept>
4.	
5.	void SingleLinkedList::init() {
6.	head = nullptr;
7.	tail = nullptr;
8.	size = 0;
9.	}
10.	
11.	bool SingleLinkedList::is_empty() {
12.	return size == 0;
13.	}
14.	
15.	void SingleLinkedList::add_front(int val) {
16.	Node* newNode = new Node();
17.	newNode->data = val;
18.	newNode->next = nullptr;
19.	if (is_empty()) {
20.	head = tail = newNode;
21.	newNode->next = head;
22.	} else {
23.	newNode->next = head;
24.	head = newNode;
25.	tail->next = head;
26.	}
27.	size++;
28.	}
29.	
30.	void SingleLinkedList::add_back(int val) {
31.	Node* newNode = new Node();
32.	newNode->data = val;
33.	newNode->next = nullptr;
34.	if (is_empty()) {
35.	head = tail = newNode;
36.	newNode->next = head;
37.	} else {
38.	tail->next = newNode;
39.	tail = newNode;
40.	tail->next = head;
41.	}
42.	size++;

```

43.     }
44.
45. void SingleLinkedList::add_idx(int val, int idx) {
46.     if (idx < 0 || idx > size) throw
std::out_of_range("Index out of range");
47.     if (idx == 0) {
48.         add_front(val);
49.     } else if (idx == size) {
50.         add_back(val);
51.     } else {
52.         Node* newNode = new Node();
53.         newNode->data = val;
54.         Node* curr = head;
55.         for (int i = 0; i < idx - 1; i++) {
56.             curr = curr->next;
57.         }
58.         newNode->next = curr->next;
59.         curr->next = newNode;
60.         size++;
61.     }
62. }
63.
64. void SingleLinkedList::delete_front() {
65.     if (is_empty()) throw std::out_of_range("List is
empty");
66.     if (size == 1) {
67.         delete head;
68.         head = tail = nullptr;
69.     } else {
70.         Node* temp = head;
71.         head = head->next;
72.         tail->next = head;
73.         delete temp;
74.     }
75.     size--;
76. }
77.
78. void SingleLinkedList::delete_back() {
79.     if (is_empty()) throw std::out_of_range("List is
empty");
80.     if (size == 1) {
81.         delete head;
82.         head = tail = nullptr;
83.     } else {
84.         Node* curr = head;
85.         while (curr->next != tail) curr = curr->next;
delete tail;

```

```

86.         tail = curr;
87.         tail->next = head;
88.     }
89.     size--;
90. }
91.
92. void SingleLinkedList::delete_idx(int idx) {
93.     if (idx < 0 || idx >= size) throw
std::out_of_range("Index out of range");
94.     if (idx == 0) delete_front();
95.     else if (idx == size - 1) delete_back();
96.     else {
97.         Node* curr = head;
98.         for (int i = 0; i < idx - 1; i++) curr = curr-
>next;
99.         Node* target = curr->next;
100.        curr->next = target->next;
101.        delete target;
102.        size--;
103.    }
104. }
105.
106. void SingleLinkedList::display() {
107.     if (is_empty()) return;
108.     Node* curr = head;
109.     for (int i = 0; i < size; i++) {
110.         std::cout << curr->data << (i == size - 1 ? "" : "
");
111.         curr = curr->next;
112.     }
113.     std::cout << std::endl;
114. }
115.
116. int SingleLinkedList::get(int idx) {
117.     if (idx < 0 || idx >= size) throw
std::out_of_range("Index out of range");
118.     Node* curr = head;
119.     for (int i = 0; i < idx; i++) curr = curr->next;
120.     return curr->data;
121. }
122.
123. void SingleLinkedList::set(int val, int idx) {
124.     if (idx < 0 || idx >= size) throw
std::out_of_range("Index out of range");
125.     Node* curr = head;
126.     for (int i = 0; i < idx; i++) curr = curr->next;
127.     curr->data = val;

```

```

128. }
129.
130. void SingleLinkedList::clear() {
131.     while (!is_empty()) {
132.         delete_front();
133.     }
134. }

```

B. Pembahasan

Program ini mengimplementasikan struktur data **Circular Singly Linked List** untuk menyimpan dan mengelola sekumpulan data bertipe integer secara dinamis. Pada bagian awal digunakan file `single_linked_list.h` yang berisi deklarasi struktur dan fungsi-fungsi yang diperlukan. Di dalam file tersebut terdapat preprocessor directive `#ifndef SINGLE_LINKED_LIST`, `#define SINGLE_LINKED_LIST`, dan `#endif` yang berfungsi sebagai *header guard* agar file header tidak diproses lebih dari satu kali saat kompilasi.

Struktur `Node` digunakan sebagai elemen dasar pada linked list. Setiap node memiliki dua anggota, yaitu `data` yang menyimpan nilai bertipe integer dan pointer `next` yang menunjuk ke node berikutnya. Selanjutnya terdapat struktur `SingleLinkedList` yang merepresentasikan keseluruhan linked list. Struktur ini memiliki pointer `head` yang menunjuk node pertama, pointer `tail` yang menunjuk node terakhir, serta variabel `size` yang menyimpan jumlah node yang terdapat di dalam list. Selain itu, pada struktur ini juga dideklarasikan berbagai fungsi yang digunakan untuk melakukan operasi terhadap linked list, seperti inisialisasi, penambahan, penghapusan, pencarian, perubahan data, penampilan isi list, hingga penghapusan seluruh node.

Implementasi seluruh fungsi terdapat pada file `single_linked_list.cpp`. Fungsi `init()` digunakan untuk menginisialisasi linked list dengan mengatur `head` dan `tail` menjadi `nullptr` serta `size` menjadi 0 sehingga list berada pada kondisi kosong. Fungsi `is_empty()` digunakan untuk memeriksa apakah linked list masih kosong berdasarkan nilai `size`.

Penambahan data dapat dilakukan melalui fungsi `add_front()`, `add_back()`, dan `add_idx()`. Fungsi `add_front()` menambahkan node baru pada bagian depan linked list, sedangkan `add_back()` menambahkan node di bagian belakang. Karena struktur yang digunakan adalah **circular singly linked list**, setiap kali terjadi penambahan node, pointer `tail->next` selalu diarahkan kembali ke `head` sehingga hubungan antar node tetap membentuk lingkaran. Fungsi `add_idx()` digunakan untuk menyisipkan node pada indeks tertentu. Sebelum penyisipan dilakukan, program memeriksa apakah indeks yang diberikan valid. Jika indeks berada di awal

atau akhir list, fungsi akan memanggil ``add_front()`` atau ``add_back()``, sedangkan untuk indeks lainnya program melakukan traversal hingga posisi yang ditentukan sebelum menyisipkan node baru.

Penghapusan data dilakukan melalui fungsi ``delete_front()``, ``delete_back()``, dan ``delete_idx()``. Fungsi ``delete_front()`` menghapus node pertama, sedangkan ``delete_back()`` menghapus node terakhir dengan terlebih dahulu mencari node sebelum ``tail``. Fungsi ``delete_idx()`` digunakan untuk menghapus node pada indeks tertentu setelah memastikan indeks yang diberikan valid. Jika indeks berada pada posisi awal atau akhir, fungsi akan memanfaatkan ``delete_front()`` atau ``delete_back()``, sedangkan pada posisi lainnya program melakukan traversal hingga node sebelum target, kemudian menghubungkan node tersebut dengan node berikutnya sebelum menghapus node yang diinginkan.

Selain operasi penambahan dan penghapusan, program juga menyediakan fungsi ``display()`` untuk menampilkan seluruh isi linked list melalui proses traversal mulai dari ``head`` sebanyak ``size`` kali. Fungsi ``get()`` digunakan untuk mengambil nilai pada indeks tertentu, sedangkan fungsi ``set()`` digunakan untuk mengubah nilai data pada node yang berada pada indeks tertentu setelah dilakukan pemeriksaan validitas indeks. Untuk menghapus seluruh isi linked list, digunakan fungsi ``clear()`` yang bekerja dengan memanggil ``delete_front()`` secara berulang hingga tidak ada lagi node yang tersisa.

Berdasarkan analisis algoritma, fungsi ``init()`` dan ``is_empty()`` memiliki ***kompleksitas waktu $O(1)$*** dan ***kompleksitas ruang $O(1)$*** karena hanya melakukan operasi sederhana pada beberapa variabel. Fungsi ``add_front()``, ``add_back()``, ``delete_front()``, dan ``delete_back()`` memiliki ***kompleksitas waktu $O(1)$*** karena proses penambahan maupun penghapusan dilakukan langsung melalui pointer ``head`` dan ``tail``, sedangkan kompleksitas ruangnya juga ***$O(1)$*** . Sementara itu, fungsi ``add_idx()``, ``delete_idx()``, ``get()``, dan ``set()`` memiliki ***kompleksitas waktu $O(n)$*** karena memerlukan proses traversal hingga mencapai indeks yang dituju, dengan kompleksitas ruang ***$O(1)$*** . Fungsi ``display()`` juga memiliki ***kompleksitas waktu $O(n)$*** karena harus mengunjungi seluruh node dalam linked list, sedangkan fungsi ``clear()`` memiliki ***kompleksitas waktu $O(n)$*** karena menghapus seluruh node satu per satu. Kompleksitas ruang untuk kedua fungsi tersebut tetap ***$O(1)$*** karena tidak menggunakan struktur data tambahan selama proses berlangsung..

Soal 2: Si Palui dan Relikui Siklus Resonansi

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K = K + 1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K = K - 1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami **reset** dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 10^4$
- $1 \leq K \leq 10^9$
- $1 \leq A_i \leq 10^6$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1.

Format Input

N K
 A_1 A_2 ... A_N

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

Penjelasan Contoh Kasus

Pada contoh pertama, susunan batu awal adalah $[1, 4, 3, 6, 2]$ dengan $K = 2$. Dari batu pertama, lompatan 2 langkah mendarat di batu 4 sehingga batu tersebut dihancurkan. Karena 4 genap,

nilai K bertambah menjadi 3 dan susunan berubah menjadi [1, 3, 6, 2].

Selanjutnya, dengan $K = 3$, lompatan mendarat di batu 2. Batu 2 dihancurkan dan karena genap, K kembali bertambah menjadi 4. Susunan kini menjadi [1, 3, 6].

Pada fase berikutnya dengan $K = 4$, lompatan mendarat di batu 1. Karena 1 ganjil, setelah dihancurkan nilai K berkurang menjadi 3, menyisakan [3, 6].

Terakhir, dengan $K = 3$, lompatan kembali mendarat di batu 3 sehingga batu tersebut dihancurkan. Struktur kini hanya menyisakan batu bernilai 6, sehingga keluaran akhirnya adalah 6.

A. Source Code

Table 13. *prob_a.cpp*

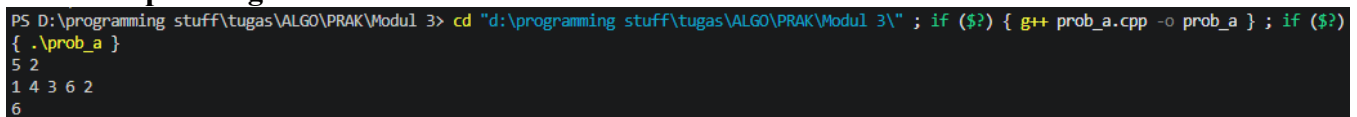
1.	<code>#include "single_linked_list.h"</code>
2.	<code>#include <iostream></code>
3.	<code>using namespace std;</code>
4.	
5.	<code>int main() {</code>
6.	<code> int N, K;</code>
7.	<code> if (!(cin >> N >> K)) return 0;</code>
8.	
9.	<code> int initial_K = K;</code>
10.	<code> SingleLinkedList stones;</code>
11.	<code> stones.init();</code>
12.	
13.	<code> for (int i = 0; i < N; i++) {</code>
14.	<code> int val;</code>
15.	<code> cin >> val;</code>
16.	<code> stones.add_back(val);</code>
17.	<code> }</code>
18.	
19.	<code> int current_idx = 0;</code>
20.	<code> while (stones.size > 1) {</code>
21.	<code> int target_idx = (int)((current_idx + (K - 1) %</code>
22.	<code>stones.size) % stones.size);</code>
23.	<code> int stone_val = stones.get(target_idx);</code>
24.	
25.	<code> if (stone_val % 2 == 0) {</code>
26.	<code> K++;</code>
27.	<code> } else {</code>
28.	<code> K--;</code>
29.	<code> }</code>

```

30.
31.         if (K <= 0) {
32.             K = initial_K;
33.         }
34.
35.         stones.delete_idx(target_idx);
36.         current_idx = target_idx % stones.size;
37.     }
38.     cout << stones.get(0) << endl;
39.
40.     return 0;
41. }

```

B. Output Program

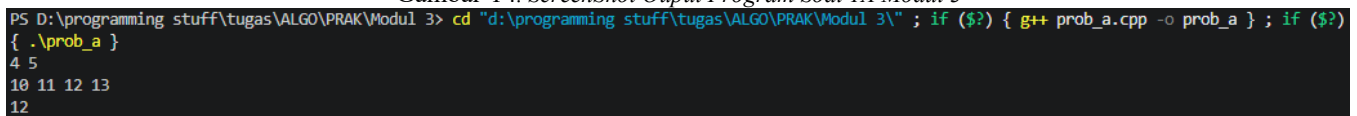


```

PS D:\programming stuff\tugas\ALGO\PRAK\Modul 3> cd "d:\programming stuff\tugas\ALGO\PRAK\Modul 3\" ; if ($?) { g++ prob_a.cpp -o prob_a } ; if ($?)
{ .\prob_a }
5 2
1 4 3 6 2
6

```

Gambar 14. ScreenShot Ouput Program Soal 1A Modul 3



```

PS D:\programming stuff\tugas\ALGO\PRAK\Modul 3> cd "d:\programming stuff\tugas\ALGO\PRAK\Modul 3\" ; if ($?) { g++ prob_a.cpp -o prob_a } ; if ($?)
{ .\prob_a }
4 5
10 11 12 13
12

```

Gambar 15. ScreenShot Ouput Program Soal 1B Modul 3

C. Pembahasan

Program ini dibuat untuk mensimulasikan proses eliminasi batu menggunakan struktur data **Circular Singly Linked List**. Pada awal program digunakan `#include "single_linked_list.h"` untuk memanggil struktur `Node`, `SingleLinkedList`, beserta seluruh fungsi yang telah didefinisikan sebelumnya. Selain itu, digunakan `#include <iostream>` untuk mendukung proses input dan output melalui `cin` dan `cout`, serta `using namespace std;` agar penulisan kode menjadi lebih sederhana tanpa perlu menambahkan awalan `std::`.

Ketika program dijalankan, terlebih dahulu dideklarasikan variabel `N` yang menyimpan jumlah batu dan `K` yang menyimpan nilai langkah lompatan awal. Nilai `K` juga disalin ke dalam variabel `initial_K` sebagai cadangan apabila selama proses eliminasi nilai `K` berubah menjadi nol atau bernilai negatif. Selanjutnya dibuat sebuah objek `SingleLinkedList` bernama `stones` yang kemudian diinisialisasi menggunakan fungsi `init()`, sehingga linked list berada pada kondisi kosong dan siap digunakan.

Program kemudian membaca sebanyak `N` buah nilai yang merepresentasikan batu. Setiap nilai yang

dibaca dimasukkan ke bagian belakang linked list menggunakan fungsi `add_back()`, sehingga urutan data di dalam linked list tetap sama dengan urutan input yang diberikan pengguna.

Setelah seluruh data berhasil dimasukkan, program memulai proses eliminasi dengan mendeklarasikan variabel `current_idx` yang menunjukkan posisi awal perhitungan. Selama jumlah batu yang tersisa masih lebih dari satu, program akan terus melakukan perulangan. Pada setiap iterasi dihitung indeks batu yang akan dihapus menggunakan rumus $(current_idx + K - 1) \% stones.size$. Penggunaan operator modulo memastikan bahwa proses perpindahan indeks berlangsung secara melingkar sesuai karakteristik **circular linked list**.

Sebelum batu dihapus, program mengambil nilai batu pada indeks yang telah ditentukan menggunakan fungsi `get()`. Nilai tersebut kemudian digunakan untuk menentukan perubahan langkah lompatan berikutnya. Apabila nilai batu yang dihapus merupakan bilangan genap, maka nilai K akan ditambah satu. Sebaliknya, apabila nilainya ganjil, maka K akan dikurangi satu. Jika hasil pengurangan menyebabkan nilai K menjadi nol atau negatif, maka K akan dikembalikan ke nilai awal yang tersimpan pada `initial_K` agar proses eliminasi tetap dapat berlangsung.

Setelah nilai K diperbarui, batu pada indeks yang telah dihitung dihapus menggunakan fungsi `delete_idx()`. Karena jumlah elemen di dalam linked list berkurang setelah penghapusan, posisi `current_idx` juga diperbarui agar tetap menunjuk ke posisi yang benar pada iterasi berikutnya. Proses ini terus dilakukan hingga hanya tersisa satu buah batu di dalam linked list. Nilai batu terakhir tersebut kemudian ditampilkan ke layar menggunakan fungsi `get(0)` sebagai hasil akhir program.

Berdasarkan analisis algoritma, proses pembentukan linked list menggunakan fungsi `add_back()` sebanyak N kali memiliki **kompleksitas waktu $O(n)$** . Selanjutnya, proses eliminasi dilakukan sebanyak $N - 1$ kali. Pada setiap iterasi, fungsi `get()` dan `delete_idx()` memerlukan proses traversal dengan kompleksitas **$O(n)$** pada kondisi terburuk. Oleh karena itu, **kompleksitas waktu keseluruhan program adalah $O(n^2)$** . Adapun **kompleksitas ruang (Space Complexity)** adalah **$O(n)$** karena linked list menyimpan seluruh data batu sebanyak N node selama proses berlangsung.

SOAL 3: Implementasi Senarai Berantai Ganda Melingkar

General Technical Requirements:

Standard: C++23

Memory Management: Manual (`new` / `delete`)

Constraints: Strictly NO STL, NO Global Arrays.

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList`. beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari **memory leak** (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan **Big-O Notation**. Seluruh manajemen memori harus dialokasikan secara manual di dalam **heap**.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Table 14. Source Code `Double_Linked_list.h` Modul 3

1.	<code>#ifndef DOUBLE_LINKED_LIST</code>
2.	<code>#define DOUBLE_LINKED_LIST</code>
3.	
4.	<code>struct Node {</code>
5.	<code>char data;</code>
6.	<code>Node * next;</code>
7.	<code>Node * prev;</code>
8.	<code>};</code>
9.	
10.	<code>struct DoubleLinkedList {</code>
11.	<code>Node *head = nullptr, *tail = nullptr;</code>
12.	<code>int size = 0;</code>
13.	
14.	<code>void init();</code>
15.	<code>bool is_empty();</code>
16.	
17.	<code>void add_front(char val);</code>
18.	<code>void add_back(char val);</code>

19.	void add_idx(char val, int idx);
20.	
21.	void delete_front();
22.	void delete_back();
23.	void delete_idx(int idx);
24.	
25.	void display();
26.	char get(int idx);
27.	void set(char val, int idx);
28.	
29.	void clear();
30.	};
31.	
32.	#endif

Table 16. Source Code Double_Linked_list.cpp Modul 3

1.	#include "double_linked_list.h"
2.	#include <iostream>
3.	#include <stdexcept>
4.	using namespace std;
5.	
6.	void DoubleLinkedList::init() {
7.	head = nullptr;
8.	tail = nullptr;
9.	size = 0;
10.	}
11.	
12.	bool DoubleLinkedList::is_empty() {
13.	return size == 0;
14.	}
15.	
16.	
17.	void DoubleLinkedList::add_front(char val) {
18.	Node* newNode = new Node();
19.	newNode->data = val;
20.	if (is_empty()) {
21.	newNode->next = newNode;
22.	newNode->prev = newNode;
23.	head = tail = newNode;
24.	} else {

```

25.         newNode->next = head;
26.         newNode->prev = tail;
27.         head->prev = newNode;
28.         tail->next = newNode;
29.         head = newNode;
30.     }
31.     size++;
32. }
33.
34. void DoubleLinkedList::add_back(char val) {
35.     Node* newNode = new Node();
36.     newNode->data = val;
37.     if (is_empty()) {
38.         newNode->next = newNode;
39.         newNode->prev = newNode;
40.         head = tail = newNode;
41.     } else {
42.         newNode->next = head;
43.         newNode->prev = tail;
44.         tail->next = newNode;
45.         head->prev = newNode;
46.         tail = newNode;
47.     }
48.     size++;
49. }
50.
51. void DoubleLinkedList::add_idx(char val, int idx) {
52.     if (idx < 0 || idx > size) return;
53.     if (idx == 0) { add_front(val); return; }
54.     if (idx == size) { add_back(val); return; }
55.
56.     Node* curr;
57.     if (idx < size / 2) {
58.         curr = head;
59.         for (int i = 0; i < idx; i++) curr = curr->next;
60.     } else {
61.         curr = tail;
62.         for (int i = size - 1; i > idx; i--) curr = curr-
>prev;
63.     }
64.

```



```

65.     Node* newNode = new Node();
66.     newNode->data = val;
67.     newNode->next = curr;
68.     newNode->prev = curr->prev;
69.     curr->prev->next = newNode;
70.     curr->prev = newNode;
71.     size++;
72. }
73.
74. void DoubleLinkedList::delete_front() {
75.     if (is_empty()) return;
76.     Node* temp = head;
77.     if (size == 1) {
78.         head = tail = nullptr;
79.     } else {
80.         head = head->next;
81.         head->prev = tail;
82.         tail->next = head;
83.     }
84.     delete temp;
85.     size--;
86. }
87.
88. void DoubleLinkedList::delete_back() {
89.     if (is_empty()) return;
90.     Node* temp = tail;
91.     if (size == 1) {
92.         head = tail = nullptr;
93.     } else {
94.         tail = tail->prev;
95.         tail->next = head;
96.         head->prev = tail;
97.     }
98.     delete temp;
99.     size--;
100. }
101.
102. void DoubleLinkedList::delete_idx(int idx) {
103.     if (idx < 0 || idx >= size) return;
104.     if (idx == 0) { delete_front(); return; }
105.     if (idx == size - 1) { delete_back(); return; }

```

```

106.
107.     Node* curr;
108.     if (idx < size / 2) {
109.         curr = head;
110.         for (int i = 0; i < idx; i++) curr = curr->next;
111.     } else {
112.         curr = tail;
113.         for (int i = size - 1; i > idx; i--) curr = curr-
114. >prev;
115.     }
116.
117.     curr->prev->next = curr->next;
118.     curr->next->prev = curr->prev;
119.     delete curr;
120.     size--;
121. }
122. void DoubleLinkedList::display() {
123.     if (is_empty()) return;
124.     Node* curr = head;
125.     for (int i = 0; i < size; i++) {
126.         cout << curr->data;
127.         if (i != size - 1) cout << " ";
128.         curr = curr->next;
129.     }
130.     cout << endl;
131. }
132.
133. char DoubleLinkedList::get(int idx) {
134.     if (idx < 0 || idx >= size) throw out_of_range("Index
135. out of range");
136.     Node* curr;
137.     if (idx < size / 2) {
138.         curr = head;
139.         for (int i = 0; i < idx; i++) curr = curr->next;
140.     } else {
141.         curr = tail;
142.         for (int i = size - 1; i > idx; i--) curr = curr-
143. >prev;
144.     }
145.     return curr->data;

```

```

144. }
145.
146. void DoubleLinkedList::set(char val, int idx) {
147.     if (idx < 0 || idx >= size) throw out_of_range("Index
out of range");
148.     Node* curr;
149.     if (idx < size / 2) {
150.         curr = head;
151.         for (int i = 0; i < idx; i++) curr = curr->next;
152.     } else {
153.         curr = tail;
154.         for (int i = size - 1; i > idx; i--) curr = curr-
>prev;
155.     }
156.     curr->data = val;
157. }
158.
159. void DoubleLinkedList::clear() {
160.     if (is_empty()) return;
161.     Node* curr = head;
162.     for (int i = 0; i < size; i++) {
163.         Node* nextNode = curr->next;
164.         delete curr;
165.         curr = nextNode;
166.     }
167.     head = tail = nullptr;
168.     size = 0;
169. }

```

B. Pembahasan

Double Linked List.h

double_linked_list.h terdapat preprocessor directive #ifndef DOUBLE_LINKED_LIST, #define DOUBLE_LINKED_LIST, dan #endif. Ketiga perintah ini berfungsi sebagai header guard untuk mencegah file header dipanggil lebih dari satu kali saat proses kompilasi berlangsung.

Struktur `Node` digunakan sebagai elemen dasar pada double linked list. Di dalam struktur ini terdapat variabel `data` yang bertipe `char` untuk menyimpan nilai karakter. Selain itu terdapat pointer `next` yang digunakan untuk menunjuk node berikutnya dan pointer `prev` yang digunakan untuk menunjuk node sebelumnya.

Selanjutnya terdapat struktur `DoubleLinkedList` yang digunakan untuk merepresentasikan keseluruhan linked list. Struktur ini memiliki pointer `head` yang menunjuk node pertama dan pointer `tail` yang menunjuk node terakhir. Selain itu terdapat variabel `size` yang digunakan untuk menyimpan jumlah node yang terdapat di dalam list.

Di dalam struktur `DoubleLinkedList` juga dideklarasikan berbagai fungsi yang digunakan untuk melakukan operasi pada linked list, seperti menambahkan node, menghapus node, menampilkan isi list, mengambil data berdasarkan indeks, mengubah data, dan menghapus seluruh isi list.

Double Linked List.cpp

```
#include "double_linked_list.h", #include <iostream>, dan #include <stdexcept>. File header digunakan untuk memanggil deklarasi struktur data dan fungsi yang diperlukan. Library iostream digunakan untuk proses input dan output, sedangkan stdexcept digunakan untuk menangani exception seperti out_of_range.
```

Fungsi `init()` digunakan untuk menginisialisasi linked list sebelum digunakan. Pada fungsi ini, pointer `head` dan `tail` diatur menjadi `nullptr`, sedangkan variabel `size` diatur menjadi 0. Kondisi ini menandakan bahwa linked list masih kosong dan belum memiliki node.

Fungsi `is_empty()` digunakan untuk mengecek apakah linked list kosong atau tidak. Fungsi akan mengembalikan nilai `true` apabila jumlah node (`size`) bernilai 0 dan mengembalikan `false` apabila masih terdapat node di dalam list.

Fungsi `add_front(char val)` digunakan untuk menambahkan node baru di bagian depan linked list. Program terlebih dahulu membuat node baru dan menyimpan nilai karakter yang diterima ke dalam atribut `data`. Jika list masih kosong, node baru akan menjadi `head` dan `tail`

sekaligus. Pointer `next` dan `prev` dari node tersebut diarahkan ke dirinya sendiri sehingga membentuk circular double linked list. Jika list sudah memiliki isi, node baru ditempatkan di depan `head`, kemudian hubungan antar node diperbarui agar struktur circular tetap terjaga.

Fungsi `add_back(char val)` digunakan untuk menambahkan node baru di bagian belakang linked list. Jika list masih kosong, prosesnya sama seperti pada fungsi `add_front()`. Namun jika list sudah memiliki isi, node baru ditempatkan setelah `tail`, kemudian pointer `tail` diperbarui agar menunjuk node yang baru ditambahkan. Hubungan antara `head`, `tail`, dan node baru juga diperbarui agar bentuk circular tetap dipertahankan.

Fungsi `add_idx(char val, int idx)` digunakan untuk menambahkan node pada indeks tertentu. Program terlebih dahulu memeriksa apakah indeks yang diberikan valid. Jika indeks bernilai 0, fungsi akan memanggil `add_front()`, sedangkan jika indeks sama dengan jumlah elemen saat ini, fungsi akan memanggil `add_back()`.

Untuk indeks lainnya, program melakukan traversal menuju posisi yang dituju. Jika indeks berada di bagian awal list, traversal dilakukan dari `head`. Namun jika indeks berada lebih dekat ke akhir list, traversal dilakukan dari `tail`. Pendekatan ini membuat proses pencarian posisi menjadi lebih efisien. Setelah posisi ditemukan, node baru disisipkan di antara dua node yang sudah ada.

Fungsi `delete_front()` digunakan untuk menghapus node yang berada di bagian depan linked list. Jika list kosong, fungsi langsung dihentikan. Jika hanya terdapat satu node, maka pointer `head` dan `tail` dikembalikan menjadi `nullptr`. Jika terdapat lebih dari satu node, pointer `head` dipindahkan ke node berikutnya dan hubungan circular antara `head` dan `tail` diperbarui sebelum node lama dihapus dari memori.

Fungsi `delete_back()` digunakan untuk menghapus node yang berada di bagian belakang linked list. Jika list hanya memiliki satu node, proses penghapusan dilakukan dengan mengosongkan `head` dan `tail`. Jika terdapat lebih dari satu node, pointer `tail` dipindahkan ke node sebelumnya, kemudian hubungan circular diperbarui sebelum node lama dihapus.

Fungsi `delete_idx(int idx)` digunakan untuk menghapus node pada indeks tertentu. Program terlebih dahulu memeriksa apakah indeks yang diberikan valid. Jika indeks menunjuk elemen pertama, fungsi akan memanggil `delete_front()`. Jika indeks menunjuk elemen terakhir, fungsi akan memanggil `delete_back()`.

Untuk indeks lainnya, program mencari node yang akan dihapus menggunakan traversal dari `head` atau `tail` tergantung posisi indeks. Setelah node ditemukan, hubungan antara node sebelumnya dan node berikutnya diperbarui sehingga node target dapat dilepas dari linked list dan dihapus dari memori.

Fungsi `display()` digunakan untuk menampilkan seluruh isi linked list ke layar. Jika list kosong, fungsi langsung dihentikan. Jika terdapat data, program melakukan traversal mulai dari `head` sebanyak `size` kali dan menampilkan nilai setiap node secara berurutan.

Fungsi `get(int idx)` digunakan untuk mengambil nilai pada indeks tertentu. Sebelum proses dilakukan, program memeriksa apakah indeks yang diberikan valid. Jika valid, program akan melakukan traversal menuju node yang dituju dan mengembalikan nilai yang tersimpan pada node tersebut.

Fungsi `set(char val, int idx)` digunakan untuk mengubah nilai data pada indeks tertentu. Setelah memastikan indeks valid, program mencari node yang sesuai kemudian mengganti nilai pada atribut `data` dengan nilai baru yang diberikan melalui parameter `val`.

Fungsi `clear()` digunakan untuk menghapus seluruh node yang ada di dalam linked list. Program melakukan traversal sebanyak jumlah node yang tersimpan, kemudian menghapus setiap node satu per satu dari memori. Setelah seluruh node terhapus, pointer `head` dan `tail` dikembalikan menjadi `nullptr` dan nilai `size` diatur menjadi 0.

SOAL 4

A. Source Code

Table 15. Source Code prob_b.cpp Modul 3

1.	#include "double_linked_list.h"
2.	#include <iostream>
3.	using namespace std;
4.	
5.	int main() {
6.	int n, r;
7.	if (!(cin >> n >> r)) return 0;
8.	
9.	DoubleLinkedList list;
10.	list.init();
11.	
12.	for (int i = 0; i < n; i++) {
13.	char karakter;
14.	cin >> karakter;
15.	list.add_back(karakter);
16.	}
17.	
18.	if (list.is_empty()) {
19.	cout << "EMPTY" << endl;
20.	return 0;
21.	}
22.	
23.	Node* p1 = list.head;
24.	Node* p2 = list.tail;
25.	
26.	for (int i = 0; i < r; i++) {
27.	long long x, y;
28.	if (!(cin >> x >> y)) break;
29.	if (list.is_empty()) break;
30.	
31.	x = ((x % list.size) + list.size) % list.size;
32.	for (int j = 0; j < x; j++) p1 = p1->next;
33.	
34.	y = ((y % list.size) + list.size) % list.size;
35.	for (int j = 0; j < y; j++) p2 = p2->prev;

```

36.
37.         if (p1 == p2) {
38.             Node* next_p1 = p1->next;
39.             Node* next_p2 = p2->prev;
40.
41.             int idx = 0;
42.             Node* curr = list.head;
43.             for (int j = 0; j < list.size; j++) {
44.                 if (curr == p1) break;
45.                 curr = curr->next;
46.                 idx++;
47.             }
48.
49.             list.delete_idx(idx);
50.
51.             if (list.is_empty()) {
52.                 p1 = p2 = nullptr;
53.             } else {
54.                 p1 = next_p1;
55.                 p2 = next_p2;
56.                 if (p1 == p2 && list.size > 1) p2 = p1-
>prev;
57.             }
58.         }
59.         else {
60.             bool adjacent = (p1->next == p2 || p2->next ==
p1);
61.
62.             if (adjacent) {
63.                 char temp = p1->data;
64.                 p1->data = p2->data;
65.                 p2->data = temp;
66.             }
67.         }
68.     }
69.
70.     if (list.is_empty()) {
71.         cout << "EMPTY" << endl;
72.     } else {
73.         Node* curr = list.head;
74.         for (int i = 0; i < list.size; i++) {

```


75.	cout << curr->data;
76.	curr = curr->next;
77.	}
78.	cout << endl;
79.	}
80.	
81.	return 0;
82.	}

B. Output Program

```

prob_b.cpp:120:17: note: 'std::endl' is defined in header '<ostream>'; this is probably fixable by adding '#include <ostream>'
PS D:\programming stuff\tugas\ALGO\PRAK\Modul 3> cd "d:\programming stuff\tugas\ALGO\PRAK\Modul 3\" ; if ($?) { g++ prob_b.cpp -o prob_b } ;
if ($?) { .\prob_b }
4 2
A B C D
1 1
2 3
ACB

```

Gambar 16. ScreenShot Output Program Soal 2A Modul 3

```

PS D:\programming stuff\tugas\ALGO\PRAK\Modul 3> cd "d:\programming stuff\tugas\ALGO\PRAK\Modul 3\" ; if ($?) { g++ prob_b.cpp -o prob_b } ;
if ($?) { .\prob_b }
5 3
V W X Y Z
100000 100000
2 2
5 5
VWYZ

```

Gambar 17. ScreenShot Output Program Soal 2B Modul 3

C. Pembahasan

Program ini dibuat untuk memproses sekumpulan karakter menggunakan struktur data Circular Doubly Linked List. Pada bagian awal program digunakan `#include "double_linked_list.h"` untuk menyertakan deklarasi struktur Node, DoubleLinkedList, beserta seluruh fungsi yang diperlukan. Selain itu, `#include <iostream>` digunakan untuk mendukung proses input dan output melalui cin dan cout, sedangkan using namespace std; digunakan agar penulisan kode menjadi lebih ringkas tanpa perlu menambahkan awalan std::.

Ketika program dijalankan, pengguna diminta memasukkan nilai n yang menyatakan jumlah karakter dan r yang menyatakan banyaknya ronde operasi. Selanjutnya dibuat sebuah objek DoubleLinkedList yang kemudian diinisialisasi menggunakan fungsi init() sehingga linked list berada pada kondisi kosong dan siap digunakan. Setelah itu, program membaca sebanyak n karakter dari masukan, kemudian setiap karakter dimasukkan ke bagian belakang linked list menggunakan fungsi add_back(). Dengan cara ini, urutan karakter di dalam linked list akan tetap sama seperti urutan yang diberikan oleh pengguna. Setelah seluruh karakter dimasukkan, program melakukan pengecekan apakah linked list kosong.

Apabila tidak terdapat elemen di dalam list, program langsung menampilkan tulisan "EMPTY" dan mengakhiri eksekusi. Jika list berisi data, program mendeklarasikan dua buah pointer, yaitu p1 yang diarahkan ke node pertama (head) dan p2 yang diarahkan ke node terakhir (tail). Kedua pointer tersebut akan digunakan sebagai penunjuk posisi selama proses berlangsung.

Program kemudian melakukan perulangan sebanyak r ronde. Pada setiap ronde, pengguna memasukkan dua buah nilai, yaitu x dan y. Nilai x digunakan untuk menggeser pointer p1 ke arah depan, sedangkan nilai y digunakan untuk menggeser pointer p2 ke arah belakang. Sebelum perpindahan dilakukan, kedua nilai tersebut dinormalisasi menggunakan operasi modulo sehingga perpindahan tetap berada dalam batas ukuran linked list dan dapat menangani nilai yang besar maupun bernilai negatif. Karena struktur yang digunakan merupakan circular doubly linked list, perpindahan pointer dapat dilakukan secara melingkar tanpa keluar dari batas list.

Setelah kedua pointer mencapai posisinya masing-masing, program memeriksa apakah p1 dan p2 menunjuk node yang sama. Jika kedua pointer bertemu pada node yang sama, node tersebut akan dihapus dari linked list. Sebelum proses penghapusan dilakukan, alamat node berikutnya yang akan menjadi tujuan p1 dan p2 disimpan terlebih dahulu agar kedua pointer tetap dapat digunakan setelah node dihapus. Program kemudian mencari indeks node tersebut melalui proses traversal dari head, lalu memanggil fungsi `delete_idx()` untuk menghapusnya. Setelah penghapusan selesai, apabila linked list menjadi kosong maka kedua pointer diatur menjadi `nullptr`. Sebaliknya, jika masih terdapat elemen di dalam list, kedua pointer dipindahkan ke posisi yang telah disimpan sebelumnya. Program juga melakukan pengecekan tambahan apabila kedua pointer kembali menunjuk node yang sama, sehingga salah satu pointer digeser untuk menghindari konflik pada iterasi berikutnya.

Apabila p1 dan p2 tidak berada pada node yang sama, program akan memeriksa apakah kedua node tersebut saling bersebelahan. Jika kondisi tersebut terpenuhi, maka nilai data pada kedua node akan ditukar. Namun apabila kedua node tidak bersebelahan, tidak ada perubahan yang dilakukan pada linked list dan program langsung melanjutkan ke ronde berikutnya.

Setelah seluruh ronde selesai diproses, program menampilkan hasil akhir. Jika linked list telah kosong, program mencetak tulisan "EMPTY". Sebaliknya, apabila masih terdapat karakter di dalam linked list, program melakukan traversal mulai dari head sebanyak size kali dan mencetak setiap karakter secara berurutan tanpa spasi. Setelah seluruh karakter ditampilkan, program mencetak baris baru dan diakhiri dengan `return 0` sebagai tanda bahwa seluruh proses telah selesai dijalankan.

Berdasarkan analisis algoritma, proses pembentukan linked list menggunakan fungsi `add_back()` sebanyak n kali memiliki kompleksitas waktu $O(n)$. Selama proses utama, setiap ronde dapat melakukan

perpindahan pointer, pencarian indeks melalui traversal, serta penghapusan node yang pada kondisi terburuk memerlukan waktu $O(n)$. Dengan jumlah ronde sebanyak r , maka kompleksitas waktu keseluruhan program adalah $O(r \times n)$. Adapun kompleksitas ruang (Space Complexity) sebesar $O(n)$ karena linked list menyimpan seluruh karakter yang dimasukkan pengguna, sedangkan penggunaan pointer dan variabel tambahan hanya membutuhkan ruang konstan.

MODUL 4: SORTING

Buble Sorting

A. Source Code

Table 16. Source Code *sorting_algorithms.cpp* (Buble Sort) Modul 4

1.	<code>void bubble_sort(std::vector<int>& data, Metrics& m) {</code>
2.	<code> int n = data.size();</code>
3.	<code> auto start = Clock::now();</code>
4.	<code> for (int i = 0; i < n - 1; i++) {</code>
5.	<code> bool swapped = false;</code>
6.	<code> for (int j = 0; j < n - i - 1; j++) {</code>
7.	<code> m.comparisons++;</code>
8.	<code> if (data[j] > data[j + 1]) {</code>
9.	<code> std::swap(data[j], data[j + 1]);</code>
10.	<code> m.swaps++;</code>
11.	<code> swapped = true;</code>
12.	<code> }</code>
13.	<code> }</code>
14.	<code> if (!swapped) break;</code>
15.	<code> }</code>
16.	<code> m.time_ms = std::chrono::duration<double,</code>
	<code>std::milli>(Clock::now() - start).count();</code>
17.	<code>}</code>

B. Pembahasan

Fungsi `bubble_sort(std::vector<int>& data, Metrics& m)` digunakan untuk mengurutkan data menggunakan algoritma **Bubble Sort** sekaligus mencatat performa algoritma melalui objek `Metrics`. Parameter `data` diterima secara *reference* agar proses pengurutan dilakukan langsung pada vektor asli tanpa membuat salinan data, sedangkan parameter `m` digunakan untuk menyimpan informasi seperti jumlah perbandingan, jumlah pertukaran data, dan waktu eksekusi algoritma.

Pada awal fungsi, program mengambil jumlah elemen yang terdapat di dalam vektor menggunakan `data.size()` kemudian menyimpannya ke dalam variabel `n`. Nilai ini digunakan sebagai batas perulangan selama proses pengurutan berlangsung. Setelah itu, program mencatat waktu mulai eksekusi menggunakan `Clock::now()`, sehingga durasi keseluruhan proses pengurutan dapat dihitung setelah algoritma selesai dijalankan.

Proses pengurutan dilakukan menggunakan dua buah perulangan bersarang. Perulangan luar mengatur jumlah *pass* yang diperlukan, sedangkan perulangan dalam membandingkan setiap pasangan elemen yang berdekatan. Pada setiap iterasi perulangan luar, variabel *swapped* diinisialisasi dengan nilai *false*. Variabel ini berfungsi sebagai penanda apakah pada *pass* tersebut terjadi pertukaran elemen atau tidak.

Selama perulangan dalam berlangsung, setiap pasangan elemen yang bersebelahan dibandingkan satu per satu. Sebelum proses perbandingan dilakukan, nilai *m.comparisons* ditambah satu sebagai pencatatan jumlah perbandingan yang dilakukan algoritma. Jika elemen pada indeks saat ini lebih besar daripada elemen di sebelahnya, kedua elemen akan ditukar menggunakan fungsi *std::swap()*. Setelah proses pertukaran selesai, nilai *m.swaps* juga ditambah satu untuk mencatat jumlah pertukaran data, kemudian variabel *swapped* diubah menjadi *true* sebagai penanda bahwa masih terdapat data yang belum berada pada posisi yang benar.

Setelah satu kali *pass* selesai dilakukan, program memeriksa nilai *swapped*. Jika nilainya masih *false*, berarti tidak ada lagi pertukaran elemen selama proses tersebut sehingga seluruh data telah berada dalam keadaan terurut. Oleh karena itu, perulangan dihentikan lebih awal menggunakan perintah *break*. Mekanisme ini merupakan bentuk optimasi pada algoritma Bubble Sort karena menghindari perulangan yang tidak diperlukan ketika data telah terurut sebelum seluruh *pass* selesai dijalankan.

Setelah proses pengurutan selesai, program menghitung waktu eksekusi dengan mencari selisih antara waktu saat ini dan waktu yang telah dicatat di awal fungsi. Selisih tersebut kemudian dikonversi ke satuan milidetik dan disimpan ke dalam atribut *m.time_ms*. Dengan demikian, fungsi ini tidak hanya menghasilkan data yang telah terurut, tetapi juga memberikan informasi mengenai jumlah perbandingan, jumlah pertukaran, serta lama waktu yang dibutuhkan selama proses pengurutan.

Berdasarkan analisis algoritma, **kompleksitas waktu (Time Complexity)** Bubble Sort pada kondisi terbaik adalah **$O(n)$** karena optimasi menggunakan variabel *swapped* memungkinkan algoritma berhenti ketika data sudah terurut. Pada kondisi rata-rata maupun terburuk, kompleksitas waktunya adalah **$O(n^2)$** karena setiap elemen harus dibandingkan dengan elemen lainnya melalui perulangan bersarang. Sementara itu, **kompleksitas ruang (Space Complexity)** adalah **$O(1)$** karena proses pengurutan dilakukan secara *in-place* dan hanya menggunakan beberapa variabel tambahan tanpa memerlukan struktur data baru

C. Analisis

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4935	2564	0	0	0	0.046900
Bubble Sort	Sorted	100	99	0	0	0	0	0.000600
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.050800
Bubble Sort	Nearly Sorted	100	4719	296	0	0	0	0.027000
Bubble Sort	Many Duplicates	100	4845	2315	0	0	0	0.044000
Bubble Sort	Random	1000	497154	243190	0	0	0	4.445300
Bubble Sort	Sorted	1000	999	0	0	0	0	0.004800
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	5.438000
Bubble Sort	Nearly Sorted	1000	481922	25778	0	0	0	2.599100
Bubble Sort	Many Duplicates	1000	496340	240808	0	0	0	5.485500
Bubble Sort	Random	10000	49993404	25118148	0	0	0	475.964800
Bubble Sort	Sorted	10000	9999	0	0	0	0	0.046500
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	524.963300
Bubble Sort	Nearly Sorted	10000	49937030	3119352	0	0	0	275.833400
Bubble Sort	Many Duplicates	10000	49993404	25093086	0	0	0	483.859500

Gambar 18. Output main.cpp (Bubble Sort) Modul 4

1. kompleksitas Waktu

- Best Case: Karena input data sudah dalam keadaan terurut, algoritma cukup melakukan perbandingan linear $n-1$ tanpa perlu melakukan satu pertukaran, maka waktu eksekusinya itu singkat jadi singkat, eksekusinya itu hanya 0.0006 ms di $n=100$, 0.0044 ms di $n=1000$, dan 0.043 ms di $n=10000$ yang mana menghasilkan kompleksitas $O(n)$

- Average Case: karena inputnya berupa data acak maupun data yang sama, jumlah perbandingan yang dilakukan tetap tinggi dan hampir menyentuh batas maksimal, jadi pertukarannya itu lebih sedikit dibanding worst case, jadi waktu eksekusinya itu juga ikut naik cukup signifikan seperti data random $n=10000$ yang mencapai 457.3997 ms yang mana menghasilkan kompleksitas $O(n^2)$
- Worst Case: karena input data terurut terbalik, algoritmanya harus melakukan perbandingan dan pertukaran secara maksimal pada setiap pasang elemen(angka), sehingga waktu eksekusinya itu melonjak sampai 533.4661 ms pada $n=10000$ yang mana ini menghasilkan kompleksitas $O(n^2)$

2. Karakteristik Algoritma

- Karena operasi perbandingan yang di pakai ini adalah operator "lebih besar dari" ($data[j] > data[j+1]$), pertukaran hanya terjadi kalau elemen di kiri itu benar benar lebih besar dari elemen kanan, jadi kalau bertemu kedua elemen yang nilainya itu sama, kondisi if nya false dan keduanya tidak ditukar, karena urutannya itu sudah relatif terjaga. Makanya algoritma ini bersifat stabil.
- Karena pengurutannya dilakukan langsung di dalam vektor aslinya tanpa membutuhkan alokasi memori tambahan yang bergantung pada ukuran data, algoritma ini adalah in place
- Karena mekanisme bekerja membandingkan dua elemen bersebelahan secara berulang lalu menukarnya kalau elemen kiri lebih besar, elemen terbesar akan "menggelembung" di tukar ke posisi kanan, jadi algoritma menjadi lebih efisien di kondisi data yang hampir terurut.

3. Operasi algoritma

- Jumlah comparison: Karena data terurut cuman butuh satu kali scan tanpa pertukaran jumlah comparison minimal itu $n-1$ ($n=100$: 99x, $n=1000$: 999x, $n=10000$: 9.999x) tapi dikarenakan datanya itu terbalik maka semua elemen itu harus dibandingkan terus menerus, jumlahnya melonjak maksimal sampai 49.995.000 kali di $n=10000$.
- Jumlah swap: Karena data terurut jadi tidak membutuhkan pertukaran sama sekali, pertukarannya 0 di semua ukuran data. Sebaliknya, karena data terbalik mengharuskan

setiap elemen berpindah tempat, maka maksimal pertukarnya sampai 49.995.000 kali di $n=10000$. Untuk data random di $n=10000$, pertukarannya ada di angka 25.118.148 kali.

- Jumlah Recursion Call: Karena kode ini pakai struktur perulangan iteratif bukan fungsi rekursif, recursion callnya ada di 0 di semua pengujian.
- Pengaruh bentuk data terhadap jumlah operasi: karena ada optimasi flag swapped, bentuk data yang menentukan jumlah operasi, data terurut langsung memicu best case dengan eksekusi super cepat, sementara data acak dan terbalik memicu lonjakan operasi yang besar. Data nearly sorted dan many duplicate menghasilkan performa yang sedang, seperti nearly sorted $n=10000$ di 259.2163 dan many duplicates $n=10000$ di 467.36.39

Selection Sort

A. Source Code

Table 17. Source Code *sorting_algorithms.cpp* (Selection Sort) Modul 4

1.	<code>void selection_sort(std::vector<int>& data, Metrics& m)</code>
2.	<code>{</code>
3.	<code> int n = data.size();</code>
4.	<code> auto start = Clock::now();</code>
5.	<code> for (int i = 0; i < n - 1; i++) {</code>
6.	<code> int min = i;</code>
7.	<code> for (int j = i + 1; j < n; j++) {</code>
8.	<code> m.comparisons++;</code>
9.	<code> if (data[j] < data[min]) {</code>
10.	<code> min = j;</code>
11.	<code> }</code>
12.	<code> }</code>
13.	<code> if (min != i) {</code>
14.	<code> std::swap(data[i], data[min]);</code>
15.	<code> m.swaps++;</code>
16.	<code> }</code>
17.	<code> }</code>
18.	<code> m.time_ms = std::chrono::duration<double,</code>
19.	<code>std::milli>(Clock::now() - start).count();</code>
	<code>}</code>

B. Pembahasan

`void selection_sort(std::vector<int>& data, Metrics& m) {` Baris ini mendeklarasikan fungsi utama bernama `selection_sort` dengan tipe pengembalian `void` (tidak mereturn nilai). Fungsi ini menerima dua argumen secara *pass-by-reference* (&), yaitu objek `std::vector` untuk manipulasi data integer secara langsung di memori, dan struktur data `Metrics` sebagai media perekam parameter performa algoritma.

`int n = data.size();` Program menyimpan total jumlah elemen yang terdapat di dalam objek data ke dalam variabel lokal `n` bertipe integer, yang nantinya digunakan sebagai batas atas perulangan.

`auto start = Clock::now();` Instruksi ini mengeksekusi metode statis untuk menangkap data waktu aktual dari sistem sesaat sebelum proses pengurutan dimulai, disimpan ke dalam variabel lokal `start` berbasis pustaka `std::chrono`.

`for (int i = 0; i < n - 1; i++) {` Program memulai perulangan luar yang berjalan sebanyak $n-1$ kali, di mana setiap putarannya mencari satu elemen minimum dari sisa data yang belum terurut untuk ditempatkan ke posisi yang tepat.

`int min = i;` Variabel `min` diinisialisasi dengan nilai indeks `i`, jadi sebagai posisi elemen terkecil sementara yang akan diperbarui jika ditemukan elemen yang lebih kecil di posisi berikutnya.

`for (int j = i + 1; j < n; j++) {` Program memulai perulangan dalam yang menelusuri seluruh sisa elemen mulai dari indeks `i+1` hingga akhir array untuk mencari posisi elemen minimum yang sebenarnya.

`m.comparisons++;` Program menaikkan nilai variabel `comparisons` pada objek `m` untuk mendokumentasikan setiap operasi perbandingan yang dilakukan selama pencarian elemen minimum.

`if (data[j] < data[min]) { min = j; }` Program membandingkan elemen pada indeks `j` dengan elemen pada posisi `min` saat ini. Jika ditemukan elemen yang lebih kecil, variabel `min` diperbarui ke indeks `j` sebagai kandidat posisi minimum terbaru.

`if (min != i) {` Setelah perulangan dalam selesai, program memeriksa apakah posisi minimum yang ditemukan berbeda dari posisi awal `i`. Pertukaran hanya dilakukan jika elemen minimum bukan berada di posisinya yang seharusnya, untuk menghindari operasi swap yang tidak perlu.

`std::swap(data[i], data[min]);` Program memanggil fungsi bawaan `std::swap` untuk menukar posisi elemen pada indeks `i` dengan elemen minimum yang ditemukan pada indeks `min`. `m.swaps++;` Setelah pertukaran terjadi, program menaikkan nilai variabel `swaps` pada objek `m` untuk mendokumentasikan total akumulasi operasi pertukaran.

`m.time_ms = std::chrono::duration<double, std::milli>(Clock::now() - start).count();` Setelah seluruh rangkaian algoritma selesai, program menangkap kembali timestamp akhir, mencari selisihnya dengan variabel `start`, lalu menyimpan hasilnya dalam satuan milidetik ke dalam properti `m.time_ms`.

C. Analisis

>> Selection Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.026500
Selection Sort	Sorted	100	4950	0	0	0	0	0.022800
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.049500
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.022800
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.025300
Selection Sort	Random	1000	499500	995	0	0	0	3.270200
Selection Sort	Sorted	1000	499500	0	0	0	0	2.207500
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	2.236300
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	2.196300
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	2.307500
Selection Sort	Random	10000	49995000	9992	0	0	0	226.683900
Selection Sort	Sorted	10000	49995000	0	0	0	0	221.533000
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	218.686500
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	221.460700
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	214.977600

Gambar 19. Output main.cpp (Selection Sort) Modul 4

1. Analisis Kompleksitas waktu

- Best case: Karena Selection Sort selalu mencari elemen minimum di setiap putaran tanpa harus peduli kondisi data, jumlah perbandingannya tetap sama di semua kondisi yaitu $n(n-1)/2$, ada dari data sorted $n=10000$ yang comparisonnya itu tetap di 49.995.000 kali. Namun karena data sudah terurut, pertukarannya 0 sehingga waktu eksekusinya sedikit lebih ringan di $n=10000$ di angka 216.4092 ms. Jadi kompleksitas waktunya itu tetap di $O(n^2)$.
- Average Case: Karena inputnya ini berupa data acak maupun data yang sama, jumlah perbandingannya tetap di angka $n(n-1)/2$ seperti pada $n=10000$ sebesar 49.995.000 kali, namun pertukarannya lebih banyak di banding best case seperti data random $n=10000$ sebesar 9.992 kali, sehingga waktu eksekusinya 214.1673. ms. Menghasilka kompleksitas waktu $O(n^2)$.
- Worst Case: Karena Selection Sort tidak mempunyai mekanisme penghentian dini seperti bubble sort, meskipun data terurut terbalik jumlah perbandingannya tetap 49.995.000 kali di $n=10000$ sama seperti kondisi yang lain. Bedanya pertukarannya itu maksimal di 5.000 kali di $n=10000$, sehingga waktu eksekusinya seiikit lebih tinggi di 221.3547 ms. Jadi kompleksitas waktunya $O(n^2)$

2. karakteristik algoritma

- Karena kode memakai operator “kurang dari” ($\text{data}[j] < \text{data}[\text{min}]$) untuk mencari elemen

minimum, kalau ketemu dua elemen bernilai sama kondisi if nya false sehingga posisi min

tidak perlu diperbarui, tetapi operasi pertukaran tetap bisa memindah elemen melewati elemen duplikat lainnya sehingga urutan relative elemen yang sama bisa berubah. Makanya algoritma ini bersifat tidak stabil

- Karena pengurutannya dilakukan langsung di dalam vektor asli tanpa membutuhkan alokasi memori tambahan yang bergantung pada ukuran data, algoritma ini masuk dalam kategori in place
- Karena mekanisme kerjanya adalah mencari elemen minimum dari sisa data yang belum terurut lalu menukarnya ke posisi paling kiri, setiap putaran hanya melakukan maksimal satu kali pertukaran meskipun perbandingannya itu tetap banyak. Makanya dari itu kenapa jumlah pertukarannya selection sort jauh lebih kecil di banding bubble sort di $n=10000$ data random, pertukarannya Cuma 9.992 kali dibanding bubble sort yang 25.118.148 kali.

3. Operasi Algoritma

- Jumlah comparison: karena Selection sort selalu men scan sisa data untuk mencari minimum tanpa bisa berhenti lebih awal, comparisonnya tetap $n(n-1)/2$ di semua kondisi input $n=100$: 4.950x, $n=1000$: 499.500x, $n=10000$: 49.995.000x.
- Jumlah Swap: Karena setiap putaran Cuma butuh maksimal satu kali pertukaran, jumlahnya jauh lebih kecil di banding bubble sort. Data sorted pertukarannya 0 di semua ukuran, data reverse sorted pertukarannya maksimal $n/2$ ($n=10000$: 5.000x), dan data random di $n=10000$ pertukarannya hanya 9992 kali.
- Jumlah Recursion Call: Karena kode ini pakai struktur perulangan iteratif bukan fungsi rekursif, recursion callnya ada di 0 di semua pengujian.
- Pengaruh Bentuk data terhadap jumlah operasi: Karena Selection Sort tidak mempunyai optimasi, bentuk data hampir tidak berpengaruh terhadap jumlah comparison seumanya tetap $n(n-1)/2$. Yang terpengaruh hanya jumlah pertukarannya, dimana data nearly sorted punya swap paling sedikit ($n=10000$: 50x) sedangkan data random mempunyai pertukaran paling banyak ($n=10000$: 9.992x). akibatnya perbedaan waktu eksekusi antar jenis input relatif kecil, seperti di $n=1000$ yang berkisar antara 209.6067 ms sampai 221.3546 ms.

Insertion Sort

A. Source Code

Table 18. Source Code *sorting_algorithms.cpp* (Insertion Sort) Modul 4

1.	<code>void insertion_sort(std::vector<int>& data, Metrics& m) {</code>
2.	<code> int n = data.size();</code>
3.	<code> auto start = Clock::now();</code>
4.	
5.	<code> for (int i = 1; i < n; i++) {</code>
6.	<code> int key = data[i];</code>
7.	<code> int j = i - 1;</code>
8.	<code> while (j >= 0) {</code>
9.	<code> m.comparisons++;</code>
10.	<code> if (data[j] > key) {</code>
11.	<code> data[j + 1] = data[j];</code>
12.	<code> m.shifts++;</code>
13.	<code> j--;</code>
14.	<code> } else {</code>
15.	<code> break;</code>
16.	<code> }</code>
17.	<code> }</code>
18.	<code> data[j + 1] = key;</code>
19.	<code> }</code>
20.	
21.	<code> m.time_ms = std::chrono::duration<double, std::milli>(Clock::now() - start).count();</code>
22.	<code>}</code>

B. Pembahasan

`void insertion_sort(std::vector<int>& data, Metrics& m) {` Baris ini mendeklarasikan fungsi utama bernama `insertion_sort` dengan tipe pengembalian `void` (tidak mereturn nilai). Fungsi ini menerima dua argumen secara `pass-by-reference` (&), yaitu objek `std::vector` untuk manipulasi data integer secara langsung di memori, dan struktur data `Metrics` sebagai media perekam parameter performa algoritma.

`int n = data.size();` Program menyimpan total jumlah elemen yang terdapat di dalam objek data ke dalam variabel lokal `n` bertipe integer sebagai batas atas perulangan.

`auto start = Clock::now();` Instruksi ini menangkap data waktu aktual dari sistem sesaat sebelum proses pengurutan dimulai, disimpan ke dalam variabel lokal `start` berbasis pustaka `std::chrono`.

`for (int i = 1; i < n; i++) {` Program memulai perulangan luar dari indeks 1 karena elemen pertama dianggap sudah terurut. Setiap iterasi mengambil satu elemen baru untuk disisipkan ke posisi yang tepat di bagian kiri yang sudah terurut.

`int key = data[i];` Nilai elemen pada indeks `i` disimpan ke dalam variabel lokal `key` sebagai elemen yang akan disisipkan ke posisi yang tepat di bagian kiri array.

`int j = i - 1;` Variabel `j` diinisialisasi dengan nilai `i-1` sebagai penunjuk indeks yang akan bergerak ke kiri untuk mencari posisi yang tepat bagi elemen `key`.

`while (j >= 0) {` Program memulai perulangan `while` yang bergerak dari kanan ke kiri selama indeks `j` masih valid (tidak melewati batas kiri array).

`m.comparisons++;` Program menaikkan nilai variabel `comparisons` pada objek `m` untuk mendokumentasikan setiap operasi perbandingan antara elemen `key` dengan elemen di sebelah kirinya.

`if (data[j] > key) {` Program membandingkan elemen di posisi `j` dengan nilai `key`. Jika elemen di kiri lebih besar, elemen tersebut perlu digeser ke kanan untuk memberi ruang bagi `key`.

`data[j + 1] = data[j];` Elemen pada indeks `j` digeser satu posisi ke kanan untuk membuka ruang bagi elemen `key` yang akan disisipkan.

`m.shifts++;` Setelah pergeseran terjadi, program menaikkan nilai variabel `shifts` pada objek `m` untuk mendokumentasikan total akumulasi operasi pergeseran.

`j--;` Indeks `j` dikurangi satu agar perulangan bergerak memeriksa elemen berikutnya di sebelah kiri.

`} else { break; }` Jika elemen di posisi `j` tidak lebih besar dari `key`, berarti posisi yang tepat untuk `key` sudah ditemukan dan perulangan langsung dihentikan tanpa memeriksa elemen-elemen lainnya.

`data[j + 1] = key;` Setelah posisi yang tepat ditemukan, nilai key disisipkan ke indeks `j+1` yang merupakan posisi finalnya di bagian array yang sudah terurut.

`m.time_ms = std::chrono::duration<double, std::milli>(Clock::now() - start).count();` Setelah seluruh rangkaian algoritma selesai, program menangkap kembali timestamp akhir, mencari selisihnya dengan variabel `start`, lalu menyimpan hasilnya dalam satuan milidetik ke dalam properti `m.time_ms`.

>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2656	0	2564	0	0	0.020400
Insertion Sort	Sorted	100	99	0	0	0	0	0.000900
Insertion Sort	Reverse Sorted	100	4950	0	4950	0	0	0.038200
Insertion Sort	Nearly Sorted	100	395	0	296	0	0	0.003400
Insertion Sort	Many Duplicates	100	2412	0	2315	0	0	0.028300
Insertion Sort	Random	1000	244181	0	243190	0	0	1.764400
Insertion Sort	Sorted	1000	999	0	0	0	0	0.008000
Insertion Sort	Reverse Sorted	1000	499500	0	499500	0	0	3.790600
Insertion Sort	Nearly Sorted	1000	26775	0	25778	0	0	0.202300
Insertion Sort	Many Duplicates	1000	241801	0	240808	0	0	2.462800
Insertion Sort	Random	10000	25128135	0	25118148	0	0	182.467000
Insertion Sort	Sorted	10000	9999	0	0	0	0	0.099100
Insertion Sort	Reverse Sorted	10000	49995000	0	49995000	0	0	354.059400
Insertion Sort	Nearly Sorted	10000	3129351	0	3119352	0	0	22.751800
Insertion Sort	Many Duplicates	10000	25103078	0	25093086	0	0	181.477100

Gambar 20. Output main.cpp (Insertion Sort) Modul 4

C. Analisis

1. Analisis Kompleksitas waktu

- **Best Case:** Karena data ini sudah terurut, loop while langsung break di iterasi pertama tanpa perlu menggeser elemen sama sekali, jadi comparisonnya ini hanya $n-1$ kali dan shiftnya 0 terbukti dari data sorted $n=10000$ yang comparisonnya cuman ada 9999 kali dengan waktu eksekusi yang singkat 0.0879 ms. Jadi kompleksitas waktunya $O(n)$.
- **Average Case:** Karena input berupa data acak maupun data duplikat/data sama, setiap elemen rata rata harus dibandingkan dan digeser setengah dari posisi sebelumnya, sehingga jumlah comparison dan shiftnya cukup tinggi, seperti data random $n=10000$ dengan comparison 25127135 kali dan shift 25118148 kali, jadi menghasilkan waktu eksekusi 183.2470 ms. Jadi kompleksitas waktunya itu $O(n^2)$
- **Worst Case:** Karena data terurut terbalik, setiap elemen harus dibandingkan dan digeser ke paling kiri melewati semua elemen sebelumnya tanpa bisa break lebih awal, sehingga comparison dan shiftnya ini maksimal terbukti di $n=10000$ keduanya mencapai 49995000 kali dengan waktu eksekusi 3855026 ms. Jadi kompleksitas waktunya $O(n^2)$.

2. Karakteristik Algoritma

- Karena kode memakai operator "lebih bsesar dari" ($\text{data}[j] > \text{key}$), oergeseran hanya terjadi kalau elemen sebelumnya benar benar lebih besar dari key, jadi kalau bertemu dua elemen bernilai kondisi if, false dan loop langsung break sehingga urutan elemennya itu tetap terjaga. Makanya algoritma ini termasuk bersifat stabil.
- Karena pengurutan dilakukan langsung di dalam vektor aslinya dengan cara menggeser elemen tanpa membutuhkan memori tambahan yang bergantung ke ukuran data, algoritma ini masuk ke kategori in place.
- Karena mekanisme kerjanya adalah mengambil satu elemen "key" lalu menggeser semua elemen sebelumnya yang lebihi besar ke kanan satu posisi hingga ketemu posisi yang tepat insert tidak memakai pertukaran melainkan memakai shift makanya kolom pertukarannya itu 0 di semua pengujian, sementara shifts yang tinggi. Ditambah ada optimasi break yang langsung menghentikan perbandingan begitu ketemu elemen yang lebih kecil atau sama makanya performa di data nearly sorted sangat baik seperti $n=10000$ yang hanya 223535 ms.

3. Operasi Algoritma

- Jumlah shift: karena insertion sort menggunakan mekanisme pergeseran bukan pertukaran, swapnya selalu 0 di semua pengujian shiftnya mengikuti pola yang sama dengan comparison 0 saat data terurut, maksimalnya itu 29995000 kali saat data terbalik, dan 25118148 kali untuk data random di $n=10000$.
- Jumlah Recursion Call: karena kode ini memakai sturuktur perulangan iteratif bukan fungsi rekursif, recursion callnya 0 di semuan pengujian.
- Pengaruh bentuk data terhadap jumlah operasi: karena ada optimasi break bentuk data sangat mempengaruhi performai insertion sort data yang terurut dan nearly sorted jauh lebih cepat dibanding data acak dan tebalik, perbedaannya jelas di $n=10000$ dimana nearly sorted hahnya butuh 22.3535 ms sementara reverse sorted mencapai 385.5026 ms, perbedaan yang terlihat.

Merge Sort

A. Source Code

Table 19. Source Code *sorting_algorithms.cpp* (Merge Sort) Modul 4

1	void merge_sort(std::vector<int>& data, Metrics& m) {
2	auto start = Clock::now();
3	auto recursive_mergesort = [&](auto self, int
	left, int right) -> void {
4	m.recursive_calls++;
5	if (left < right) {
6	int mid = left + (right - left) / 2;
7	
8	self(self, left, mid);
9	self(self, mid + 1, right);
10	
11	int n1 = mid - left + 1;
12	int n2 = right - mid;
13	
14	std::vector<int> L(n1);
15	std::vector<int> R(n2);
16	for (int i = 0; i < n1; i++) L[i] =
	data[left + i];
17	for (int j = 0; j < n2; j++) R[j] =
	data[mid + 1 + j];
18	
19	int i = 0;
20	int j = 0;
21	int k = left;
22	
23	while (i < n1 && j < n2) {
24	m.comparisons++;
25	if (L[i] <= R[j]) {
26	data[k] = L[i];
27	i++;
28	} else {
29	data[k] = R[j];
30	j++;
31	}
32	k++;
33	}
34	
35	while (i < n1) data[k++] = L[i++];
36	while (j < n2) data[k++] = R[j++];
37	}
38	};
39	
40	if (!data.empty()) {
41	recursive_mergesort(recursive_mergesort, 0,
	(int)data.size() - 1);
42	}
43	

44	<code>m.time_ms = std::chrono::duration<double, std::milli>(Clock::now() - start).count();</code>
45	<code>}</code>

B. Pembahasan

void merge_sort(std::vector<int>& data, Metrics& m) { Baris ini mendeklarasikan fungsi utama bernama `merge_sort` dengan tipe pengembalian `void` (tidak mereturn nilai). Fungsi ini menerima dua argumen secara *pass-by-reference* (&), yaitu objek `std::vector` untuk manipulasi data integer secara in-place di memori, dan struktur data `Metrics` yang dialokasikan sebagai media perekam parameter performa algoritma.

auto start = Clock::now(); Instruksi ini mengeksekusi metode statis untuk menangkap data waktu aktual dari sistem sesaat sebelum proses pengurutan dimulai. Objek waktu tersebut disimpan ke dalam variabel lokal `start` dengan penentu tipe otomatis (auto) berbasis pustaka `std::chrono`.

auto recursive_mergesort = [&](auto self, int left, int right) -> void { Di sini program mendefinisikan sebuah ekspresi fungsi lambda yang disimpan ke dalam variabel `recursive_mergesort`. Simbol [&] menandakan *reference capture* agar fungsi lambda dapat mengakses dan memodifikasi objek data serta `m` di luar cakupannya, sementara argumen `auto self` digunakan sebagai Teknik general lamda agar fungsi ini mampu memanggil dirinya sendiri secara rekursif, dengan batasan jangkauan indeks `left` dan `right`. **m.recursive_calls++;** Setiap kali fungsi lambda masuk ke dalam siklus eksekusi baru, program melakukan operasi penambahan pada variabel `recursive_calls` di dalam struktur objek `m` untuk mendokumentasikan total akumulasi panggilan rekursif.

if (left < right) { Baris ini mengimplementasikan struktur kontrol kondisional sebagai *base case* (kondisi batas) untuk menghentikan percabangan rekursi. Operasi di dalam blok ini hanya akan diproses jika sub-vektor yang dievaluasi masih memiliki panjang minimal dua elemen aktif (indeks kiri lebih kecil dari indeks kanan).

int mid = left + (right - left) / 2; Program menghitung posisi indeks titik tengah (`mid`) dari partisi array berjalan. Formulasi `left + (right - left) / 2` sengaja dipilih sebagai bentuk proteksi arsitektur memori guna menghindari risiko terjadinya kesalahan fatal akibat penambahan nilai (*integer overflow*) yang kerap muncul pada formula konvensional `(left + right) / 2`.

self(self, left, mid); Instruksi ini memicu panggilan rekursif pertama terhadap fungsi lambda itu sendiri untuk memproses, memecah, dan mengurutkan elemen pada partisi wilayah sebelah kiri, yang membentang dari indeks batas bawah `left` hingga batas atas titik tengah `mid`.

self(self, mid + 1, right); Tepat setelah sub-proses kiri selesai, baris ini memicu panggilan rekursif kedua untuk menangani partisi wilayah sebelah kanan, bergerak dari indeks pasca-titik tengah `mid + 1` hingga batas atas elemen `right`.

int n1 = mid - left + 1; Program melakukan kalkulasi matematika sederhana untuk menentukan besaran alokasi atau total kapasitas memori yang dibutuhkan untuk menampung elemen-elemen dari sub-vektor wilayah kiri.

int n2 = right - mid; Baris ini melakukan kalkulasi serupa untuk menentukan kapasitas ruang penyimpanan yang diperlukan bagi elemen-elemen dari sub-vektor wilayah kanan.

std::vector<int> L(n1); Program menginstansiasi objek array dinamis lokal baru bernama **L** bertipe **std::vector<int>** dengan dimensi ukuran sebesar variabel **n1** yang dialokasikan di memori heap sebagai tempat penyimpanan sementara data klaster kiri. **std::vector<int> R(n2);** Secara paralel, objek vektor temporer kedua bernama **R** diinstansiasi dengan kapasitas memori sebesar variabel **n2** sebagai media penyimpanan sementara data klaster kanan.

for (int i = 0; i < n1; i++) L[i] = data[left + i]; Struktur perulangan ini berfungsi melakukan replikasi data sekuensial, menyalin nilai elemen dari kontainer utama **data** (dimulai dari offset indeks **left**) ke dalam array temporer kiri **L** menggunakan operator subscript (**[]**).

for (int j = 0; j < n2; j++) R[j] = data[mid + 1 + j]; Perulangan *for* kedua ini menjalankan tugas replikasi serupa untuk menyalin nilai elemen dari sektor kanan kontainer utama **data** (dimulai dari offset indeks **mid + 1**) ke dalam array temporer kanan **R**. **int i = 0;** Variabel integer **i** diinisialisasi dengan nilai awal nol, yang diposisikan sebagai penunjuk indeks kontrol pergerakan elemen di dalam array temporer kiri **L**.

int j = 0; Variabel integer **j** diinisialisasi dengan nilai awal nol, yang diposisikan sebagai penunjuk indeks kontrol pergerakan elemen di dalam array temporer kanan **R**.

int k = left; Variabel integer **k** diinisialisasi dengan nilai awal dari parameter **left**, bertindak sebagai iterator utama penulisan ulang elemen hasil kombinasi pada objek **data** asli. **while (i < n1 && j < n2) {** Instruksi ini memulai struktur perulangan *while* yang akan terus berjalan secara kontinu selama indikator indeks pointer **i** dan **j** belum melampaui batas dimensi fisik dari kedua array temporer (**L** dan **R**).

m.comparisons++; Sebelum evaluasi elemen dilakukan, program memperbarui buku catatan statistik performa dengan menaikkan nilai variabel properti **comparisons** pada objek **m** sebanyak satu angka.

if (L[i] <= R[j]) { Program mengeksekusi operasi logika relasional untuk membandingkan nilai elemen pada indeks berjalan array kiri **L[i]** terhadap elemen array kanan **R[j]**. Penggunaan operator **<=** (kurang dari atau sama dengan) di sini sekaligus menjaga sifat kestabilan (*stability*) dari algoritma Merge Sort.

data[k] = L[i]; Jika hasil evaluasi kondisi **if** bernilai *true* (elemen kiri lebih kecil atau sama), maka nilai dari array temporer **L[i]** dipindahkan dan dituliskan ke posisi indeks aktif **k** pada array utama **data**.

i++; Indeks kontrol **i** dinaikkan sebanyak satu angka agar pada iterasi berikutnya program dapat mengevaluasi elemen selanjutnya yang berada di dalam array temporer kiri **L**.

} else { Blok percabangan alternatif ini akan dieksekusi apabila kondisi logika pada instruksi **if** menghasilkan nilai *false* (elemen kanan ternyata memiliki nilai yang lebih kecil).

data[k] = R[j]; Program memindahkan nilai dari array temporer kanan **R[j]** untuk dimasukkan dan dituliskan ke posisi indeks aktif **k** pada array utama **data**.

j++; Indeks kontrol **j** dinaikkan sebanyak satu angka agar program dapat bergeser memeriksa elemen berikutnya pada array temporer kanan **R**.

} Simbol kurung kurawal penutup untuk membatasi ruang lingkup dari blok percabangan *if-else*.

k++; Indeks utama **k** dinaikkan secara inkremental pada setiap akhir siklus perulangan *while* guna memastikan posisi penulisan data berikutnya di array utama bergeser ke kanan.

} Kurung kurawal penutup yang menandakan selesainya satu putaran evaluasi kombinasi dua sub-vektor temporer.

while (i < n1) data[k++] = L[i++]; Instruksi pasca-proses ini menangani skenario di mana array kanan **R** sudah kehabisan elemen terlebih dahulu. Sisa elemen yang masih tertinggal di array kiri **L** akan disalin secara massal langsung ke kontainer utama **data** menggunakan operasi post-increment (**k++** dan **i++**).

while (j < n2) data[k++] = R[j++]; Instruksi pasca-proses alternatif yang menangani skenario sebaliknya; jika array kiri **L** habis lebih dulu, seluruh sisa data pada array kanan **R** akan langsung dipindahkan ke dalam kontainer utama **data**.

} Kurung kurawal penutup untuk membatasi cakupan *scope* logika internal milik instruksi *base case if (left < right)*.

}; Tanda penutup yang secara resmi menandakan akhir dari deklarasi dan definisi blok fungsi **lambda recursive_mergesort**.

if (!data.empty()) { Sebelum memulai proses rekursi, program melakukan validasi pemeriksaan kondisi memori menggunakan fungsi bawaan **.empty()**. Blok kode di dalamnya hanya dieksekusi jika objek **data** tidak berada dalam kondisi kosong (*null/empty*).

recursive_mergesort(recursive_mergesort, 0, (int)data.size() - 1); Baris ini menginisialisasi jalannya algoritma Merge Sort untuk pertama kalinya dengan memanggil fungsi **lambda recursive_mergesort**, melemparkan parameter dirinya sendiri, indeks awal **0**, serta indeks akhir data yang dikonversi secara eksplisit ke tipe data **int**.

} Kurung kurawal penutup untuk membatasi ruang lingkup dari instruksi pengecekan data kosong **if (!data.empty())**.

m.time_ms = std::chrono::duration<double, std::milli>(Clock::now() - start).count(); Setelah seluruh rangkaian algoritma selesai, program menangkap kembali *timestamp* akhir via **Clock::now()**, mencari

selisihnya dengan variabel `start`, lalu melakukan operasi casting ke tipe `double` berskala milidetik (`std::milli`). Fungsi `.count()` kemudian mengambil nilai numerik murni tersebut untuk disimpan ke dalam properti `m.time_ms`.

} Kurung kurawal penutup yang menandai batas akhir dari seluruh ruang lingkup (*scope*) fungsionalitas fungsi `merge_sort`.

C. Analisis

>> Merge Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	535	0	0	199	0	0.027800
Merge Sort	Sorted	100	356	0	0	199	0	0.022700
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.037100
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.027000
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.029200
Merge Sort	Random	1000	8690	0	0	1999	0	0.294500
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.239300
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	0	0.300900
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.368800
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.320100
Merge Sort	Random	10000	120465	0	0	19999	0	4.113900
Merge Sort	Sorted	10000	69008	0	0	19999	0	2.828900
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	4.503500
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	3.291300
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	3.919600

Gambar 21. Output `main.cpp` (Merge Sort) Modul 4

1. Analisis Kompleksitas Waktu

- Best Case: Karena Merge Sort ini membagi array menjadi dua bagian secara rekursif lalu menggabungkannya kembali tanpa harus melihat kondisi data, jumlah comparison dan recursion callnya relatif stabil di semua kondisi dari data sorted $n=10000$ yang comparisonnya 69008 kali dengan recursion call 19999 kali dan waktu eksekusi 2.8115 ms. Jadi kompleksitas waktunya itu $O(n \log n)$.
- Average Case: Karena inputnya berupa data acak maupun data duplikat/datanya sama proses penggabungan membutuhkan lebih banyak perbandingan dibanding data terurut karena elemen dari dua sisi sering bergantian masuk menjadi comparisonnya lebih tinggi kayak data random $n=10000$ dengan comparison 120456 kali waktu eksekusinya itu 3.4643 ns. Jadi kompleksitas waktunya itu $O(n \log n)$.
- Worst Case: Karena data terurut terbalik memaksa proses merge selalu membandingkan elemen secara bergantian itu membuat salah satu sisi habis jumlah comparisonnya mendekati maksimal, di $n=10000$ dengan comparisonnya 64608 kali dan waktu eksekusinya itu 2.9093 ms. Jadi kompleksitas waktunya itu tetap $O(n \log n)$

2. Karakteristik Algoritma

- Karena Kode memakai operator "kurang dari atau sama dengan" (`l[i] <= r[j]`) saat menggabungkan dua bagian, elemen yang nilainya sama dari sisi kiri menjadi prioritas masuk lebih dulu sebelum elemen yang di kanan sehingga elemennya berurutan supaya

tetap terjaga elemennya, maka algoritma ini bersifat stabil

- Karena proses merge ini membutuhkan array tambahan L dan R untuk menyimpan elemen sementara di setiap pemanggilan rekursif, algoritmanya ini bukan in place dan membutuhkan memori tambahan $O(n)$

- Karena mekanismenya ini bekerja membagi array secara rekursif hingga ukurannya itu 1 lalu menggabungkan kembali menjadi dua bagian secara terurut dengan membandingkan elemen satu per satu, Merge Sort tidak menggunakan pertukaran sama sekali, makanya kolom Swap/pertukarannya itu 0 di semua pengujian, Rekursi berhenti saat $left \geq right$, dan jumlah recursion callnya selalu tetap di $2n-1$ untuk setiap ukuran datanya, kaya $n=10000$ yang selalu 19999 kali di semua kondisi input yang ada.

3. Operasi Algoritma

- Jumlah Comparison: Karena Proses Merge membandingkan elemen dari dua sisi secara bergantian jumlah comparison bergantung pada kondisi data ,paling sedikit saat data terurut ($n=10000$: 69008x) karena satu sisinya ini cepat habis dan paling banyak saat data random ($n=10000$: 120465x) jadi perbedaannya tidak terlalu jauh karena kompleksitasnya ini tetap $O(n \log n)$
- Jumlah swap: karena Merge Sort menggunakan mekanisme pemindahan elemen ke array sementara bukan melakukan pertukaran secara langsung, swapnya selalu 0 di semua pengujian.
- Jumlah recursion call: karena pemanggilan rekursifnya ini selalu membagi array menjadi dua hingga ukurannya menjadi 1, recursion callnya selalu tetap $2n-1$ di semua kondisi inputnya $n=100:199$, $n=1999$, $n=10000:19999$, itu tidak terpengaruh ke bentuk data sama sekali.
- Pengaruh bentuk data terhadap jumlah operasi: Karena Merge Sort ini selalu menyelesaikan seluruh prosesnya, bentuk data hanya mempengaruhi jumlah comparison sementara recursion call dan swap tetap sama, akibat perbedaan waktu eksekusi antar jenis itu inputnya relatif kecil, kaya $n=10000$ yang berkisaran 2.8115 ms sampai 3.4642 ms. Ini lebih stabil dibanding bubble sort maupun insertion sort.

Quick Sort

A. Source Code

Table 20. Source Code *sorting_algorithms.cpp* (Quick Sort) Modul 4

1	<code>void quick_sort(std::vector<int>& data, Metrics& m) {</code>
2	<code> auto start = Clock::now();</code>
3	<code> auto recursive_qsort = [&](auto self, int low, int high)</code>
	<code>-> void {</code>
4	<code> m.recursive_calls++;</code>
5	<code> if (low < high) {</code>
6	<code> int pivot = data[high];</code>
7	<code> int i = low - 1;</code>
8	
9	<code> for (int j = low; j < high; j++) {</code>
10	<code> m.comparisons++;</code>
11	<code> if (data[j] <= pivot) {</code>
12	<code> i++;</code>
13	<code> std::swap(data[i], data[j]);</code>
14	<code> m.swaps++;</code>
15	<code> }</code>
16	<code> }</code>
17	<code> std::swap(data[i + 1], data[high]);</code>
18	<code> m.swaps++;</code>
19	<code> int pi = i + 1;</code>
20	<code> self(self, low, pi - 1);</code>
21	<code> self(self, pi + 1, high);</code>
22	<code> }</code>
23	<code> };</code>
24	
25	<code> if (!data.empty()) {</code>
26	<code> recursive_qsort(recursive_qsort, 0, (int)data.size()</code>
	<code>- 1);</code>
27	<code> }</code>
28	
29	<code> m.time_ms = std::chrono::duration<double,</code>
	<code>std::milli>(Clock::now() - start).count();</code>
30	<code>}</code>

B. Pembahasan

`void quick_sort(std::vector<int>& data, Metrics& m) {` Baris ini merupakan deklarasi fungsi utama bernama `quick_sort` dengan tipe pengembalian `void`. Fungsi ini

menerima objek `std::vector` penampung elemen integer dan struktur data `Metrics` secara “&” agar proses manipulasi elemen dan perekaman data statistik langsung berdampak pada objek asli di memori tanpa prosedur duplikasi.

`auto start = Clock::now();` Instruksi ini berfungsi untuk menangkap waktu aktual dari sistem sesaat sebelum algoritma dijalankan. Objek waktu ini disimpan ke dalam variabel lokal `start` menggunakan penentu tipe otomatis (`auto`) sebagai basis perhitungan durasi eksekusi.

`auto recursive_qsort = [&](auto self, int low, int high) -> void {` Di sini program mendefinisikan fungsi lambda bernama `recursive_qsort`. Simbol `[&]` memungkinkan fungsi di dalam fungsi ini mengakses dan memodifikasi variabel dari *scope* luar, sementara parameter `auto self` diterapkan agar fungsi lambda ini memiliki kapabilitas untuk memanggil dirinya sendiri secara rekursif berdasarkan batas indeks `low` dan `high`.

`m.recursive_calls++;` Setiap kali fungsi lambda memasuki siklus eksekusi baru, program melakukan operasi penambahan pada properti `recursive_calls` di dalam objek metrik `m` untuk melacak total akumulasi panggilan rekursif yang terjadi.

`if (low < high) {` Baris ini mengimplementasikan struktur kontrol kondisional sebagai kondisi batas rekursi. Blok kode di dalamnya hanya akan dieksekusi jika sub-vektor yang sedang diperiksa masih memiliki skala minimal dua elemen aktif.

`int pivot = data[high];` Program menentukan strategi pemilihan elemen pembatas (*pivot*) dengan mengambil nilai dari elemen paling akhir pada sub-vektor aktif (`data[high]`). Nilai ini akan menjadi acuan sentral untuk mempartisi data menjadi dua kelompok skala nilai.

`int i = low - 1;` Variabel integer `i` diinisialisasi sebagai pointer statis dengan nilai indeks terkecil dikurangi satu. Variabel ini bertindak sebagai pembatas wilayah untuk menandai posisi akhir dari elemen-elemen yang nilainya terbukti lebih kecil atau sama dengan `pivot`.

`for (int j = low; j < high; j++) {` Baris ini memulai struktur *for loop* menggunakan variabel kontrol `j` sebagai iterator dinamis. Perulangan ini akan menelusuri setiap elemen di dalam sub-vektor, bergerak dari indeks `low` hingga satu elemen sebelum `pivot` (`high - 1`).

`m.comparisons++;` Di dalam perulangan, tepat sebelum operasi evaluasi logika dijalankan, program menaikkan nilai properti `comparisons` pada objek metrik `m` sebanyak satu angka untuk merekam jejak operasi perbandingan data.

`if (data[j] <= pivot) {` Program mengeksekusi instruksi kondisional untuk membandingkan nilai elemen pada indeks berjalan `data[j]` terhadap nilai `pivot`. Penggunaan operator `<=` memastikan elemen yang nilainya sama dengan `pivot` akan dikelompokkan ke sisi kiri.

`i++;` Jika kondisi `if` menghasilkan nilai *true*, pointer pembatas `i` digeser satu langkah ke kanan untuk memperluas area penampungan elemen berpoin kecil sekaligus menyiapkan slot memori baru.

`std::swap(data[i], data[j]);` Fungsi utilitas standar `std::swap` dipanggil untuk menukar nilai memori antara elemen pada indeks pembatas `i` dengan elemen pada indeks berjalan `j`. Operasi ini secara fisik memindahkan angka yang lebih kecil dari pivot ke sisi kiri array.

`m.swaps++;` Tepat setelah pertukaran nilai selesai di level memori, properti `swaps` pada objek metrik `m` di-inkrementasikan untuk mencatat total mutasi reposisi elemen.

`}` Kurung kurawal penutup untuk mengakhiri cakupan operasi dari blok instruksi kondisional `if`.

`}` Kurung kurawal penutup perulangan *for loop* (`j`).

`std::swap(data[i + 1], data[high]);` Setelah seluruh elemen selesai ditelusuri oleh perulangan `j`, elemen `pivot` yang berada di posisi `data[high]` ditukar dengan elemen di posisi `data[i + 1]`.

`m.swaps++;` Program kembali meningkatkan counter `m.swaps` sebanyak satu angka untuk mencatat aktivitas pemindahan posisi fisik elemen `pivot` tersebut.

`int pi = i + 1;` Program mencatat indeks lokasi final dari elemen `pivot` yang baru saja ditempatkan dan menyimpannya ke dalam variabel baru bernama `pi` (*pivot index*).

`self(self, low, pi - 1);` Fungsi lambda memicu panggilan rekursif pertama terhadap dirinya sendiri untuk memproses dan mengurutkan sub-vektor wilayah kiri, yang membentang dari indeks awal `low` hingga satu indeks sebelum `pivot` (`pi - 1`).

`self(self, pi + 1, high);` Fungsi lambda memicu panggilan rekursif kedua untuk menangani sub-vektor wilayah kanan, bergerak dari satu indeks setelah `pivot` (`pi + 1`) hingga batas akhir elemen `high`.

`}` Kurung kurawal penutup `if (low < high)`.

`};` Tanda penutupan yang secara formal menandai akhir dari deklarasi dan definisi blok fungsi lambda `recursive_qsort`.

`if (!data.empty()) {` Sebelum memulai rantai rekursi, fungsi utama melakukan validasi menggunakan fungsi `.empty()` guna memastikan bahwa objek vektor `data` tidak berada dalam kondisi kosong untuk menghindari ketidakjelasan definisi.
`recursive_qsort(recursive_qsort, 0, (int)data.size() - 1);` Baris ini menginisialisasi eksekusi perdana algoritma Quick Sort dengan memanggil fungsi lambda `recursive_qsort`, meloloskan fungsi itu sendiri sebagai argumen, indeks awal `0`, serta indeks akhir vektor yang dikonversi ke tipe data integer.

`}` Kurung kurawal penutup untuk `if (!data.empty())`.

`m.time_ms = std::chrono::duration<double, std::milli>(Clock::now() - start).count();` Setelah seluruh hierarki rekursi selesai, program menangkap waktu akhir, mengalkulasi selisihnya dengan waktu awal `start`, lalu melakukan operasi casting ke tipe data

double berskala milidetik (std::milli). Fungsi .count() kemudian mengekstrak nilai numerik durasi tersebut ke dalam variabel m.time_ms.

} Kurung kurawal penutup untuk fungsi quick_sort.

C. Analisis

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	608	353	0	129	0	0.008100
Quick Sort	Sorted	100	492	358	0	133	0	0.005600
Quick Sort	Reverse Sorted	100	487	328	0	133	0	0.005800
Quick Sort	Nearly Sorted	100	494	359	0	129	0	0.005700
Quick Sort	Many Duplicates	100	806	654	0	177	0	0.009600
Quick Sort	Random	1000	12393	7287	0	1319	0	0.121400
Quick Sort	Sorted	1000	8150	5174	0	1333	0	0.062000
Quick Sort	Reverse Sorted	1000	8067	4671	0	1139	0	0.058800
Quick Sort	Nearly Sorted	1000	9307	6208	0	1305	0	0.104000
Quick Sort	Many Duplicates	1000	15718	12174	0	1755	0	0.151000
Quick Sort	Random	10000	159305	91893	0	13175	0	1.620400
Quick Sort	Sorted	10000	114996	68914	0	13333	0	0.879900
Quick Sort	Reverse Sorted	10000	114214	64877	0	12287	0	0.784100
Quick Sort	Nearly Sorted	10000	157359	100776	0	12943	0	1.283400
Quick Sort	Many Duplicates	10000	173738	124136	0	17433	0	1.796000

Gambar 22. Output main.cpp (Quick Sort) Modul 4

1. Analisis Kompleksitas Waktu

- Best Case: jadi karena pivot yang ini dipilih selalu elemen terakhir jadi data random membuat pembagian array ini lebih sering seimbang sehingga kedalaman rekursinya minimal, dari data random $n=10000$ yang comparisonnya hanya 149797 kali dengan recursion call 13347 kali dan waktu eksekusi hanya 1.3518 ms. Jadi kompleksitas waktunya itu $O(n \log n)$.
- Average Case: Karena inputnya berupa data acak maupun data duplikat ataupun sama pembagian arraynya itu tidak selalu seimbang tapi masih cukup baik, comparison dan swapnya lebih tinggi dibandingkan best case, kayak data many duplicates $n=10000$ dengan comparisonnya 170629 kali dan waktu eksekusinya itu ada di 1.5083 ms. Jadi kompleksitas waktunya itu $O(n \log n)$
- Worst Case: Karena pivotnya ini diambil dari elemen terakhir jadi data terurut atau tidak terurut pun menyebabkan pembagian arraynya itu tidak seimbang, satu sisi selalu kosong dan sisi lain berisi $n-1$ elemen, sehingga recursion call dan kondisi comparisonnya yang cukup meningkat drastis mencapai 499995000 kali di $n=10000$ dengan waktu eksekusi 388.0812 ms untuk sorted dan 288.1437 ms. Jadi kompleksitas waktunya itu $O(n^2)$.

2. Karakteristik Algoritma

- Karena kode ini memakai pivot nya itu dari elemen terakhir dan proses swapnya bisa memindahkan elemen untuk melewati elemen lain yang bernilai sama urutan relatif elemen yang sama tidak selalu terjaga, makanya algoritmanya ini bersifat tidak stabil.
- Karena pengurutan dilakukan langsung di dalam vector yang aslinya menggunakan swap tanpa butuh memori tambahan yang bergantung pada ukuran data, algoritma ini mengurutkan data langsung di dalam array.
- Karena mekanisme kerjanya adalah memilih pivot dari elemen terakhir lalu mempartisi array sehingga elemen yang lebih kecil dari pivot itu ada di sebelah kiri dan yang lebih besar itu berada di sebelah kanan, proses ini diulang secara rekursif di kedua sisi. Performanya sangat bergantung pada pemilihan pivot kalau pivot selalu jadi elemen terkecil atau terbesar pada data sorted dan reverse sorted, rekursinya itu menjadi sangat lambat.

3. Operasi Algoritma

- Jumlah Comparison: Karena jumlah comparison sangat bergantung pada kualitas pivot yang dipilih, jadi hasilnya itu bervariasi, paling sedikit saat data random ($n=10000$: 149797) dan melonjak drastis saat data sorted atau reverse sorted ($n=10000$: 49.995.000x) jadi pembagian arraynya itu tidak seimbang.
- Jumlah Swap: Karena setiap elemen yang lebih kecil dari pivot itu langsung di tukar ke sisi kiri, swapnya itu juga berubah, paling sedikit saat data random itu ($n=10000$: 77749) dan paling banyak saat data sorted ($n=10000$: 50004999) karena hampir semua elemennya itu di tukar di setiap proses pembagiannya.
- Jumlah Recursion Call: Karena kedalam rekursinya itu bergantung pada seberapa seimbangnya partisinya, recursion callnya sangat bervariasi, paling sedikit itu saat data random ($n=10000$: 13347) dan paling banyak di saat data sorted dan reverse sorted ($n=10000$: 19999) karena rekursinya itu terus bercabang hingga batas maksimalnya.
- Pengaruh bentuk data terhadap jumlah operasi: karena pemilihan pivot dari elemen terakhir itu membuat quick sort sangat sensitif terhadap susunan data, data yang terurut dan terbalik itu membuat performa terburuk dengan waktu eksekusi menjadi sangat banyak, sementara data random dan nearly sorted justru menjadi kondisi terbaiknya kayak data random $n=10000$ yang hanya 1.3518 ms berbanding jauh dengan data sorted yang mencapai 388.0812 ms.

Radix Sort

A. Source Code

Table 21. Source Code *sorting_algorithms.cpp* (Radix Sort) Modul 4

1	<code>void radix_sort(std::vector<int>& data, Metrics& m) {</code>
2	<code> if (data.empty()) return;</code>
3	<code> auto start = Clock::now();</code>
4	
5	<code> int max_val = data[0];</code>
6	<code> for (size_t i = 1; i < data.size(); i++) {</code>
7	<code> if (data[i] > max_val) max_val = data[i];</code>
8	<code> }</code>
9	
10	<code> int n = data.size();</code>
11	<code> for (int exp = 1; max_val / exp > 0; exp *= 10) {</code>
12	<code> std::vector<int> output(n);</code>
13	<code> int count[10] = {0};</code>
14	
15	<code> for (int i = 0; i < n; i++) {</code>
16	<code> count[(data[i] / exp) % 10]++;</code>
17	<code> m.array_accesses++;</code>
18	<code> }</code>
19	
20	<code> for (int i = 1; i < 10; i++) {</code>
21	<code> count[i] += count[i - 1];</code>
22	<code> m.array_accesses += 2;</code>
23	<code> }</code>
24	
25	<code> for (int i = n - 1; i >= 0; i--) {</code>
26	<code> int digit = (data[i] / exp) % 10;</code>
27	<code> output[count[digit] - 1] = data[i];</code>
28	<code> m.array_accesses += 2;</code>
29	<code> count[digit]--;</code>
30	<code> m.array_accesses++;</code>
31	<code> }</code>
32	
33	<code> for (int i = 0; i < n; i++) {</code>
34	<code> data[i] = output[i];</code>
35	<code> m.array_accesses++;</code>
36	<code> }</code>
37	<code> }</code>
38	
39	<code> m.time_ms = std::chrono::duration<double,</code>
	<code>std::milli>(Clock::now() - start).count();</code>

B. Pembahasan

`void radix_sort(std::vector<int>& data, Metrics& m) {` Baris ini mendefinisikan fungsi utama bernama `radix_sort` yang tidak mereturn nilai (`void`). Fungsi ini menerima dua parameter secara `pass-by-reference` (`&`), yaitu `std::vector` berisi data integer yang akan diurutkan, dan objek `Metrics` sebagai pencatat performa algoritma.

`if (data.empty()) return;` Sebelum mulai, program mengecek dulu apakah datanya kosong. Kalau kosong, fungsi langsung berhenti tanpa melakukan apapun.

`auto start = Clock::now();` Program merekam waktu awal eksekusi dari sistem menggunakan pustaka `std::chrono`, disimpan ke variabel `start` untuk perhitungan durasi nantinya.

`int max_val = data[0];` Program asumsikan dulu elemen pertama sebagai nilai maksimum sementara, yang bakal diperbarui kalau ketemu nilai yang lebih besar.

`for (size_t i = 1; i < data.size(); i++) { if (data[i] > max_val) max_val = data[i]; }` Perulangan ini nyari nilai maksimum yang sebenarnya dari seluruh elemen array. Nilai ini penting karena menentukan berapa banyak digit yang perlu diproses.

`int n = data.size();` Total jumlah elemen disimpan ke variabel `n` sebagai batas atas perulangan berikutnya.

`for (int exp = 1; max_val / exp > 0; exp *= 10) {` Ini perulangan utamanya, yang jalan sebanyak jumlah digit dari nilai maksimum. Variabel `exp` mulai dari 1 (satuan), terus dikalikan 10 tiap iterasi (puluhan, ratusan, dst) sampai semua digit selesai diproses.

`std::vector<int> output(n);` Di tiap iterasi digit, program bikin array sementara bernama `output` berukuran `n` sebagai tempat nampung hasil pengurutan digit yang lagi diproses.

`int count[10] = {0};` Array count berukuran 10 (mewakili digit 0-9) dibuat dan diisi nol semua, fungsinya buat ngitung seberapa sering tiap digit muncul.

`for (int i = 0; i < n; i++) { count[(data[i] / exp) % 10]++; m.array_accesses++; }` Perulangan ini ngitung frekuensi kemunculan tiap digit di posisi yang lagi diproses. Ekspresi `(data[i] / exp) % 10` ngambil digit tertentu dari elemen, lalu count-nya dinaikkan satu. Tiap akses array dicatat ke `m.array_accesses`.

`for (int i = 1; i < 10; i++) { count[i] += count[i - 1]; m.array_accesses += 2; }` Perulangan ini ngubah count jadi array kumulatif (prefix sum) yang nentuin posisi akhir tiap elemen di output. Tiap iterasi mencatat dua akses array ke `m.array_accesses`.

`for (int i = n - 1; i >= 0; i--) {` Perulangan ini jalan dari kanan ke kiri supaya sifat stabil algoritmanya terjaga — elemen yang duluan muncul di array asli bakal duluan juga ditempatkan di output untuk digit yang sama.

`int digit = (data[i] / exp) % 10;` Program ngambil nilai digit di posisi yang lagi diproses dari elemen `data[i]`.

`output[count[digit] - 1] = data[i];` Elemen `data[i]` ditempatkan ke posisi yang udah ditentukan oleh `count[digit]-1` di dalam array output.

`m.array_accesses += 2;` Program nyatat dua akses array yang terjadi, yaitu baca `count[digit]` dan tulis ke output.

`Count[digit]--;` Nilai count pada digit tersebut dikurangi satu biar elemen berikutnya dengan digit yang sama ditempatkan di posisi sebelumnya.

`m.array_accesses++;` Program nyatat satu akses array tambahan untuk operasi pengurangan `count[digit]`.

`For (int i = 0; i < n; i++) { data[i] = output[i]; m.array_accesses++; }` Perulangan ini nyalin balik semua elemen dari output ke array data asli, menggantinya dengan hasil pengurutan digit yang baru selesai diproses. Tiap operasi penyalinan dicatat sebagai satu akses array.


```
m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
```

Setelah semua selesai, program mengambil timestamp akhir lewat `Clock::now()`, ngitung selisihnya sama variabel `start`, lalu nyimpan hasilnya dalam satuan milidetik ke properti `m.time_ms`.

C. Analisis

>> Radix Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Radix Sort	Random	100	0	0	0	0	1554	0.007900
Radix Sort	Sorted	100	0	0	0	0	1554	0.007700
Radix Sort	Reverse Sorted	100	0	0	0	0	1554	0.007800
Radix Sort	Nearly Sorted	100	0	0	0	0	1554	0.007000
Radix Sort	Many Duplicates	100	0	0	0	0	1036	0.006800
Radix Sort	Random	1000	0	0	0	0	20072	0.069300
Radix Sort	Sorted	1000	0	0	0	0	20072	0.086400
Radix Sort	Reverse Sorted	1000	0	0	0	0	20072	0.069400
Radix Sort	Nearly Sorted	1000	0	0	0	0	20072	0.070000
Radix Sort	Many Duplicates	1000	0	0	0	0	15054	0.052700
Radix Sort	Random	10000	0	0	0	0	250090	0.865700
Radix Sort	Sorted	10000	0	0	0	0	250090	0.908100
Radix Sort	Reverse Sorted	10000	0	0	0	0	250090	0.873400
Radix Sort	Nearly Sorted	10000	0	0	0	0	250090	1.277400
Radix Sort	Many Duplicates	10000	0	0	0	0	200072	0.683200

Gambar 23 Output main.cpp (Radix Sort) Modul 4

1. Analisis Kompleksitas Waktu

- Best Case: jadi karena Radix Sort ini memproses data perdigit tanpa melakukan perbandingan antara elemen, jadi jumlah iterasinya itu bergantung pada jumlah digit nilai maksimum (d) atau ukuran data (n) bukan susunan datanya, dari data many duplicates $n=10000$ yang arr.accessnya lebih sedikit 2000072 kali dengan waktu eksekusi 0.7938 ms karena nilai maksimumnya lebih kecil dibanding kondisi lainnya. Jadi $O(d \times n)$
- Average Case: Karena inputnya berupa data acak maupun nearly sorted jumlah digit maksimalnya sama dengan kondisi lainnya sehingga arr.accessnya tetap di angka 250090 kali di $n=10000$ menghasilkan eksekusi yang stabil di kisaran 0.7.939 smapai 0.8167 ms. Kompleksitas waktunya Adalah $O(d \times n)$
- Worst Case: Karena many duplicates $n=10000$ justru memiliki arr.access lebih sedikit, worst case di sini Adalah kondisi Dimana nilai maksimum data paling besar sehingga jumlah digitnya itu paling banyak seperti data sorted $n=10000$ yang arr.accessnya 250.090 kali dengan waktu eksekusi tertinggi yaitu 0.1867 ms. Kompleksitas waktunya itu tetap sama $O(d \times n)$

2. Karakteristik algoritma

- Karena proses pengisian ke array ouputnya dilakukan dari kanan ke kiri ($i = n-1$ sampai 0)

Dan menggunakan counting array yang bersifat kumulatif, elemen dengan digit yang sama akan tetap mempertahankan urutan relatifnya. Jadi algoritmanya ini bersifat stabil.

- Karena proses pengurutannya membutuhkan array tambahan output berukuran n dan array count berukuran 10 di setiap iterasi digitnya, jadi algoritma ini bukan in place dan membutuhkan memori tambahan sebesar $O(n+k)$ dimana k ini adalah bilangan 10.
- Selanjutnya karena mekanisme kerjanya adalah mengelompokkan elemen berdasarkan nilai digit di setiap posisi menggunakan counting sort di setiap iterasinya, radix sort tidak menggunakan comparison, swap, shift, dan rekursi sama sekali makanya kolom comparison, swap, shift, dan recursion callnya itu di angka 0 di semua pengujian. Satu satunya metrik yang tercatat adalah arr.access yang merepresentasikan total akses array selama proses pengelompokan dan penulisan ulang data.

3. Operasi Algoritma

- Jumlah comparison: Karena Radix Sort tidak membandingkan elemen satu sama lain melainkan hanya membaca nilai digitnya dan comparisonnya 0 di semua pengujian.
- Jumlah swap dan shift: karena mekanisme pengurutannya menggunakan penulisan ke array output bukan pertukaran atau pergeseran langsung, swap dan shiftnya 0 di semua pengujian.
- Jumlah recursion call: karena satu satunya metrik yang diukur adalah akses array, jumlahnya bergantung pada jumlah digit nilai maksimum dan ukuran data $n=100:1554$, $n=1000: 20072$, $n=10000: 250090$ kecuali many duplicate jadi perbedaannya itu terjadi karena nilai maksimal lebih kecil sampai jumlah digitnya itu yang diproses lebih sedikit.
- Pengaruh bentuk data terhadap jumlah operasi: karena Radix sort tidak terpengaruh oleh susunan data melainkan hanya oleh nilai maksimum dan ukuran datanya, perbedaan waktu eksekusi antar jenis sangat kecil di $n=10000$ itu antara 0.7939 ms hingga 0.8167 ms untuk input biasa. Kecuali many duplicate yang sedikit lebih cepat karena nilai duplikatnya lebih kecil sehingga jumlah digitnya lebih sedikit.

ANALISIS PERFORMA

1. Perbandingan Waktu Eksekusi Antar Algoritma

Bubble Sort dan insertion sort itu menunjukkan waktu eksekusi paling tinggi di kondisi worst case masing masing mencapai 533.4661 ms dan 385.5026 ms, kalau selection sort itu lebih ke stabil tapi tetap lambat di sekitar 209 sampai 221 ms karena melakukan $n(n-1)/2$ comparison tanpa optimasi apapun. Merge sort dan Quick sort ini jauh lebih baik di banding dari sorting sebelumnya, kayak merge sort konsisten di 2.8ms sampai 3.5 ms dan quick sort bisa 1.3518 ms untuk data randomnya, tapi quick sort lumayan turun di data sorted dan reverse sorted sampai 388 ms itu karena pivotnya diambil dari elemen terakhir. Terakhir ada radix sort ini paling konsisten di semua kondisi dengan waktu Cuma 0.79 sampai 1.09 di $n=10000$ karena radix ini kda tapi peduli dengan susunan data.

2. Algoritma Plain Cepat Untuk Dataset Kecil dan Besar

Buat dataset kecil " $n=100$ " hampir semua algoritma itu di bawah 0.05 ms jadi bedanya itu kda begitu terlihat, tapi untuk di data random itu quick sort lebih unggul dikit 0.0077 ms, sama juga dengan radix sort di sekitar 0.0047 sampai 0.0074 ms. Tetapi insertion ini juga cukup oke buat data sorted di $n=100$ dengan 0.0031 ms karena optimasi breaknya kerja di dataset kecil yang hampir terurut

3. Pengaruh kondisi data terhadap performa algoritma

Kondisi data ini memberi pengaruh yang berbeda ke setiap algoritma, bubble sort dan insertion sort sangat unggul kalau datanya itu hampir terurut atau nearly sorted karena optimasi flag swap dan breaknya bisa membuat prosesnya mendekati $O(n)$, tapi keduanya langsung menjadi lambat kalau datanya itu terbalik karena harus mengerjakan operasinya secara maksimal di tiap iterasi, selection sort juga hampir tidak terpengaruh oleh kondisi data karena comparisonnya selalu sama aja, yang berbeda itu cuman di swapnya, Quick sort ini paling tricky karena pemilihan pivotnya itu dari elemen terakhir itu menjadikan data randomnya itu menjadi best casenya sementara data sorted menjadi worst case, merge sort ini relatif stabil karena selalu menjalankan proses pembelahan dan penggabungan. Radix sort sama sekali tidak terpengaruh susunan data yang menentukan dari radix ini hanya nilai maksimal dan ukuran datanya.

4. Hubungan Antara Teori kompleksitas dan hasil eksperimen

Hasilnya secara keseluruhan teori kompleksitas itu bubble sort dan insertion sort itu adalah $O(n^2)$ di worst case dengan waktu eksekusinya naik drastis dari $n=100$ ke $n=10000$ namun keduanya mendapatkan $O(n)$ di best case optimasi masing masing. Selection sort itu konsisten di $O(n^2)$ di semua kondisi karena tidak mempunyai optimasi apapun. Merge sort ini $O(n \log n)$ dengan waktu

eksekusi yang stabil dan pertumbuhan jauh lebih lambat dibanding algoritma $O(n^2)$, quick sort membuktikan bahwa kompleksitas waktunya itu $O(n \log n)$ itu hanya berlaku di kondisi data yang baus, sementara di worst casenya itu naik kembali ke $O(n^2)$ seperti yang ada pada data sorted $n=10000$ yang mencapai 388 ms. Terakhir itu radix sort menunjukkan kalau $O(d \times n)$ itu memang tidak berpengaruh terhadap kondisi data, makanya waktunya itu kecil dan stabil.

MODUL 5: SORTING

Linear Searching

A. Source Code

Table 22. Source Code searching_algorithms (Linear Searching) Modul 5

1.	<code>void linear_search(const std::vector<int>& data, int</code>
2.	<code>target, Metrics& m) {</code>
3.	<code> auto start = Clock::now();</code>
4.	<code> for (int i = 0; i < (int)data.size(); i++) {</code>
5.	<code> m.comparisons++;</code>
6.	<code> if (data[i] == target) {</code>
7.	<code> m.found_index = i;</code>
8.	<code> break;</code>
9.	<code> }</code>
10.	<code> }</code>
11.	
12.	<code> m.time_us = std::chrono::duration<double,</code>
13.	<code>std::micro>(Clock::now() - start).count();</code>
	<code>}</code>

B. Pembahasan

Fungsi `linear_search(const std::vector<int>& data, int target, Metrics& m)` digunakan untuk mencari suatu nilai pada sebuah **vector** menggunakan algoritma **Linear Search**. Fungsi ini menerima tiga parameter, yaitu `data` yang berisi kumpulan elemen bertipe integer, `target` yang merupakan nilai yang akan dicari, serta objek `Metrics` yang digunakan untuk mencatat performa algoritma, seperti jumlah perbandingan, indeks data yang ditemukan, dan waktu eksekusi. Parameter `data` diterima sebagai *const reference* sehingga isi vektor tidak dapat diubah selama proses pencarian dan tidak perlu membuat salinan data.

Pada awal fungsi, program mencatat waktu mulai eksekusi menggunakan `Clock::now()`. Nilai waktu ini disimpan ke dalam variabel `start` agar nantinya dapat digunakan untuk menghitung lama proses pencarian setelah algoritma selesai dijalankan.

Proses pencarian dilakukan menggunakan sebuah perulangan yang dimulai dari indeks pertama hingga elemen terakhir pada vektor. Pada setiap iterasi, nilai `m.comparisons` ditambah satu untuk mencatat jumlah perbandingan yang dilakukan selama proses pencarian. Selanjutnya, elemen pada indeks saat ini dibandingkan dengan nilai `target`.

Apabila nilai elemen sama dengan nilai yang dicari, maka indeks tempat elemen tersebut ditemukan disimpan ke dalam atribut `m.found_index`. Setelah itu, perulangan langsung dihentikan menggunakan perintah `break` karena `target` telah ditemukan sehingga tidak diperlukan lagi pemeriksaan terhadap elemen-elemen

berikutnya. Sebaliknya, jika nilai yang dibandingkan tidak sama, proses pencarian akan dilanjutkan ke elemen berikutnya hingga seluruh data selesai diperiksa atau target berhasil ditemukan.

Setelah proses pencarian selesai, program menghitung waktu eksekusi dengan mencari selisih antara waktu saat ini dan waktu awal yang telah dicatat sebelumnya. Selisih tersebut kemudian dikonversi ke satuan **mikrodetik (µs)** dan disimpan ke dalam atribut `m.time_us`. Dengan demikian, fungsi ini tidak hanya menentukan apakah nilai target ditemukan, tetapi juga mencatat jumlah perbandingan yang dilakukan serta waktu yang dibutuhkan selama proses pencarian.

Berdasarkan analisis algoritma, **kompleksitas waktu (Time Complexity)** pada kondisi terbaik adalah **O(1)**, yaitu ketika nilai target berada pada elemen pertama sehingga pencarian langsung berhenti. Pada kondisi rata-rata dan kondisi terburuk, kompleksitas waktunya adalah **O(n)** karena algoritma dapat memeriksa sebagian besar atau seluruh elemen di dalam vektor sebelum menemukan target atau menyimpulkan bahwa target tidak ada. Adapun **kompleksitas ruang (Space Complexity)** adalah **O(1)** karena algoritma hanya menggunakan beberapa variabel tambahan tanpa memerlukan struktur data baru.

C. Analisis

```
>> Linear Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Linear Search	Best	1000	1	0	0.000
Linear Search	Average	1000	375	374	1.400
Linear Search	Worst	1000	1000	Not Found	7.000
Linear Search	Best	10000	1	0	0.100
Linear Search	Average	10000	3746	3745	13.300
Linear Search	Worst	10000	10000	Not Found	34.300
Linear Search	Best	100000	1	0	0.100
Linear Search	Average	100000	37455	37454	126.700
Linear Search	Worst	100000	100000	Not Found	336.600
Linear Search	Best	1000000	1	0	0.000
Linear Search	Average	1000000	374541	374540	1380.400
Linear Search	Worst	1000000	1000000	Not Found	3562.000

Gambar 24. Ouput Main.cpp(linear Search) Modul 5

1. Kompleksitas Waktu

Best Case: karena targetnya ini diletakkan di index pertama (`data[0]`) loopnya itu langsung break di iterasi pertama setelah melakukan satu kali comparison jadi ukuran data yang bagian comparisonnya Cuma 1 kali dengan waktu eksekusinya itu dekat 0 us, jadi kompleksitas waktunya Adalah O(1)

Average Case: Karena targetnya ini diambil secara acak di dalam data jadi rata rata loopnya menelusuri setengah dari total elemen sebelum ketemu, maka dari itu dari comparisonnya selalu di sekitar $n/2$ di semua ukuran, jadi kompleksitas waktunya itu $O(n)$.

Worst Case: jadi karena worst case ini diset ke nilai yang memang tidak ada di data loopnya itu terpaksa mencek semua elemen dari awal sampai habis tapi tidak ketemu, makanya comparisonnya itu cocok dengan n dan hasilnya itu Not Found. Jadi kompleksitasnya $O(n)$

2. karakteristik Algoritma

Linear Search tidak memerlukan data terurut karena mekanismenya itu hanya menelusuri elemennya satu persatu dari kiri ke kanan tanpa harus memanfaatkan informasi data.

Algoritma ini bersifat iteratif karena menggunakan struktur for loop biasanya tanpa ada pemanggilan fungsi secara rekursif.

Jadi mekanisme utamanya itu membandingkan setiap elemen sama target secara berurutan dari indeks 0, jadi ketemu langsung stop, kalau tidak ketemu itu terus sampai habis, simpel tapi tidak efisien buat data yang besar karena tidak bisa ngeskip elemen.

3. analisis program

Jumlah Comparison: Jumlah comparison ini bergantung sama posisi target kalau ada di awal 1x, kalau di Tengah itu sekitar $n/2$, kalau tidak sama kelai ya ditempatkan di n , polanya ini konsisten semua ukuran data.

Nilai Found Index: Best casenya selalu di index 0, Average case ketemu di sekitar indeks $n/2$
 $n=1000$: 374, $n=10000$: 374, sampai seterusnya dan worst case selalu Not Found karena memang targetnya memang tidak ada di data.

Pengaruh posisi target: Posisi target menentukan berapa banyak comparison yang terjadi, kalau makin jauh target dari indeks awal makin banyak comparisonnya karena linear search tidak mempunyai cara langsung buat lompat ke posisi yang dicari

Pengaruh ukuran data: Karena Kompleksitasnya $O(n)$, jumlah comparison di worst case naik lurus sama ukuran datanya, jadi waktu eksekusinya itu naik melonjak dari 5.2 us di $n=1000$ sampai 3641 di $n=1000000$.

Binary Searching

A. Source Code

Table 23. Source Code *searching_algorithms* (Binary Searching) Modul 5

1.	<code>void binary_search(const std::vector<int>& data, int</code>
2.	<code>target, Metrics& m) {</code>
3.	<code> auto start = Clock::now();</code>
4.	<code> int left = 0;</code>
5.	<code> int right = (int)data.size() - 1;</code>
6.	
7.	<code> while (left <= right) {</code>
8.	<code> m.comparisons++;</code>
9.	<code> int mid = left + (right - left) / 2;</code>
10.	
11.	<code> if (data[mid] == target) {</code>
12.	<code> m.found_index = mid;</code>
13.	<code> break;</code>
14.	<code> } else if (data[mid] < target) {</code>
15.	<code> left = mid + 1;</code>
16.	<code> } else {</code>
17.	<code> right = mid - 1;</code>
18.	<code> }</code>
19.	<code> }</code>
20.	
21.	<code> m.time_us = std::chrono::duration<double,</code>
22.	<code>std::micro>(Clock::now() - start).count();</code>
	<code>}</code>

B. Pembahasan

Fungsi `binary_search(const std::vector<int>& data, int target, Metrics& m)` digunakan untuk mencari suatu nilai pada sebuah vector yang telah terurut menggunakan algoritma Binary Search. Fungsi ini menerima tiga parameter, yaitu `data` yang berisi kumpulan elemen bertipe integer, `target` yang merupakan nilai yang akan dicari, serta objek `Metrics` yang digunakan untuk mencatat performa algoritma, seperti jumlah perbandingan, indeks data yang ditemukan, dan waktu eksekusi. Parameter `data` diterima sebagai *const reference* sehingga isi vektor tidak dapat diubah selama proses pencarian dan tidak perlu membuat salinan data.

Pada awal fungsi, program mencatat waktu mulai eksekusi menggunakan `Clock::now()` dan menyimpannya ke dalam variabel `start`. Selanjutnya, dideklarasikan dua buah variabel, yaitu `left` yang diinisialisasi dengan nilai 0 sebagai batas kiri pencarian dan `right` yang diinisialisasi dengan indeks elemen terakhir pada vektor sebagai batas kanan pencarian. Kedua variabel ini digunakan untuk menentukan rentang data yang masih

mungkin mengandung nilai target.

Proses pencarian dilakukan menggunakan perulangan `while` yang akan terus berjalan selama nilai `left` masih lebih kecil atau sama dengan `right`. Pada setiap iterasi, nilai `m.comparisons` ditambah satu untuk mencatat jumlah perbandingan yang dilakukan algoritma. Setelah itu, program menghitung posisi elemen tengah menggunakan rumus $left + (right - left) / 2$. Bentuk perhitungan ini dipilih karena lebih aman dibandingkan $(left + right) / 2$, sehingga dapat menghindari kemungkinan terjadinya *integer overflow* pada data berukuran besar.

Selanjutnya, elemen pada posisi tengah dibandingkan dengan nilai `target`. Apabila kedua nilai sama, berarti target telah ditemukan sehingga indeks tersebut disimpan ke dalam atribut `m.found_index`, kemudian perulangan dihentikan menggunakan perintah `break`. Jika nilai elemen tengah lebih kecil daripada target, berarti target hanya mungkin berada pada bagian kanan vektor sehingga batas kiri (`left`) digeser ke posisi setelah elemen tengah. Sebaliknya, jika nilai elemen tengah lebih besar daripada target, maka target hanya mungkin berada pada bagian kiri sehingga batas kanan (`right`) digeser ke posisi sebelum elemen tengah. Proses ini terus dilakukan hingga target ditemukan atau rentang pencarian menjadi tidak valid.

Setelah proses pencarian selesai, program menghitung waktu eksekusi dengan mencari selisih antara waktu saat ini dan waktu awal yang telah dicatat sebelumnya. Selisih tersebut kemudian dikonversi ke satuan mikrodetik (μs) dan disimpan ke dalam atribut `m.time_us`. Dengan demikian, fungsi ini tidak hanya menentukan posisi target di dalam vektor, tetapi juga mencatat jumlah perbandingan yang dilakukan serta waktu yang dibutuhkan selama proses pencarian.

Berdasarkan analisis algoritma, kompleksitas waktu (Time Complexity) pada kondisi terbaik adalah $O(1)$, yaitu ketika nilai target langsung ditemukan pada elemen tengah saat iterasi pertama. Pada kondisi rata-rata maupun kondisi terburuk, kompleksitas waktunya adalah $O(\log n)$ karena pada setiap iterasi ruang pencarian selalu dibagi menjadi dua bagian. Adapun kompleksitas ruang (Space Complexity) adalah $O(1)$ karena algoritma hanya menggunakan beberapa variabel tambahan tanpa memerlukan struktur data baru.

C. Analisis

```
>> Binary Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Binary Search	Best	1000	9	500	0.200
Binary Search	Average	1000	3	374	0.100
Binary Search	Worst	1000	10	Not Found	0.400
Binary Search	Best	10000	13	5000	0.200
Binary Search	Average	10000	13	3745	0.300
Binary Search	Worst	10000	14	Not Found	0.500
Binary Search	Best	100000	16	50000	0.300
Binary Search	Average	100000	16	37454	0.500
Binary Search	Worst	100000	17	Not Found	0.600
Binary Search	Best	1000000	19	500000	0.300
Binary Search	Average	1000000	15	374540	0.600
Binary Search	Worst	1000000	20	Not Found	0.600

Gambar 25. Output main.cpp(Binary Searching) Modul 5

1. Kompleksitas waktu

Best Case: Karena targetnya diset ke elemen tengah ($\text{data}[\text{n}/2]$), harusnya loopnya ketemu di iterasi pertama setelah mengecek posisi midnya, tapi comparisonnya bukan 1 karena posisi mid bergeser setiap ada pembagian, makanya dari $n=1000$ yang comparisonnya itu 9x $n=10000$ 13x, $n=100000$ 16x, $n=1000000$ 19x, jadi terlihat pertumbuhannya yang sangat lambat kompleksitasnya itu $O(\log n)$

Average Case: Karena targetnya ini diambil random dari dalam data, posisinya tidak terlalu di tengah persis jadi butuh beberapa langkah pembagian sebelum ketemu comparisonnya tetap kecil di kisaran $\log n$ ($n=1000$: 3x, $n=10000$: 13x, dan seterusnya) jadi kompleksitasnya $O(\log n)$

Worst case: karena targetnya memang tidak ada di data, pencarian terus jakan memotong data menjadi dua terus sampai rentangnya habis, jadi comparisonnya maksimal sekitar $\log_2(n)+1$ ($n=1000$: 10x, $n=10000$: 14x, $n=100000$: 17x, $n=1000000$: 20x) dan hasilnya itu Not Found. Jadi kompleksitasnya itu $O(\log n)$.

2. Karakteristik algoritma

Binary search ini membutuhkan data yang terurut karena cara kerjanya mengandalkan perbandingan nilai mid sama target untuk menentukan harus lanjut ke kiri atau ke kanan, kalau datanya acak maka hasilnya bakal error/salah

Algoritma iteratif karena memakai while loop biasa, jadi tidak memakai rekursi.

Mekanisme utamanya adalah selalu mengecek elemen di posisi tengah dari rentang yang terisia, kalau target lebih kecil pencarian pindah ke kiri, kalau lebih besar pindah ke kanan, dan kalau sama langsung stop setiap iterasi memotong setengah dari sisa data yang perlu dicek, jadi lebih efisien dibandingkan linear search untuk ukuran data yang besar.

3. analisis program

Jumlah comparison: Karena setiap iterasi selalu memotong setengahnya data, jumlah comparison menjadi lambat dari $n=1000$ ke $n=1000000$ naiknya lumayan signifikan, comparisonnya Cuma naik dari 9-10x jadi 19x-20x. Jadi

Nilai found index: Best case selalu ketemu di indeks $n/2$ ($n=1000$: 500, $n=10000$: 5000, $n=100000$: 50000, sampai seterusnya), average case ketemu di indeks $n/2$ juga karena targetnya itu random dan worst case itu Not Found

Pengaruh posisi target: Posisi target mempengaruhi jumlah iterasi tapi tidak banyak seperti linear search bahkan di average case comparisonnya lebih sedikit dari best case karena yang random bisa lebih cepat ketemunya tergantung posisi relatif terhadap pembagian yang terjadi.

Pengaruh ukuran data: Karena kompleksitasnya ini $O(\log n)$ kenaikan ukuran data tidak terlalu mempengaruhi ke jumlah comparison, walaupun datanya naik 10000x dari $n=1000$ ke $n=1000000$, comparisonnya hanya naik sekitar 2 kali lipatnya aja waktu eksekusinya juga kecil di kisaran 0.1 sampai 0.7 us di semua ukuran data

ANALISIS PERFORMA

1. Perbandingan waktu eksekusi antar algoritma

Perbedaan waktu eksekusi antara linear search dan binary search ini terlihat pas datanya itu besar. Di $n=1000000$ linear search untuk worst casenya membutuhkan 3641 us sementara binary search worst casenya itu hanya 0.5 us, jadi perbedaannya itu sangat banyak, bahkan di best case sekalipun linear search sudah mulai ketinggalan karena perbandingan Cuma 1 tapi binary search dengan perbandingan 19x tetap lebih cepat karena kemungkinan overhead itu kecil.

2. Algoritma mana yang paling cepat untuk dataset kecil dan besar

Untuk dataset kecil keduanya masih cukup cepat dan bedanya tidak begitu terasa, linear search best case 0 us dan binary search best case 0.1 us. Tapi buat dataset besar binary search jauh lebih bagus di semua kondisi dengan waktu eksekusinya itu konsisten di 0.2-0.5 us, sementara di linear search average casenya sudah menyentuh 1503.7 us dan worst casenya 3641. Jadi binary search lebih bagus untuk data besar.

3. Pengaruh distribusi data terhadap performa algoritma

Kondisi data ini berdampak ke linear search tapi hampir tidak berdampak ke binary search. Linear search sangat bergantung sama posisi target, best casenya bisa 0 us tapi worst case bisa 3641 us di $n=1000000$, perbedaannya itu jauh, kalau binary search di sisi lain tetap stabil di semua kondisi karena setiap iterasi selalu membuang setengah data, jadi mau targetnya ada di manapun atau tidak ada sama sekali, perbandingan tetap di sekitaran $\log n$.

4. Hubungan antara teori kompleksitas dan hasil eksperimen

Hasil eksperimen membuktikan perbedaan antara $O(n)$ dan $O(\log n)$, linear search ini $O(n)$ karena perbandingan naik lurus sama ukuran datanya dari $n=1000$ ke $n=1000000$ (naik 1000x), perbandingan worst case juga naik cukup signifikan 1000x dari 1000 menjadi 1000000. Nah beda kalo binary search itu $O(\log n)$ dari kenaikan data yang sama aja 1000x, perbandingan naik 2x lipat dari 10 menjadi 20. Nah jadi kalau buat data besar perbedaan kompleksitas $O(n)$ sama $O(\log n)$ berdampak sangat signifikan ke performanya

MODUL 6: Graph and Tree

Soal 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN
M
U1 V1 W1
U2 V2 W2
...
UM VM WM
```

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

```
V1 V2 ... VN
V1 a11 a12 ... a1N
V2 a21 a22 ... a2N
```

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris

B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Table 24 Source Code Soal1.cpp Modul 6

1.	#include <iostream>	
2.	#include <vector>	
3.		
4.	int main() {	
5.	int N, M;	
6.		
7.	if (!(std::cin >> N)) return 0;	
8.	std::vector<char> labels(N);	
9.	for (int i = 0; i < N; ++i) {	
10.	std::cin >> labels[i];	
11.	}	
12.	std::vector<std::vector<int>>	matrix(N,
13.	std::vector<int>(N,	
14.	0));	
15.		
16.	if (!(std::cin >> M)) return 0;	
17.	for (int i = 0; i < M; ++i) {	
18.	char u, v;	
19.	int w;	
20.	if (std::cin >> u >> v >> w) {	
21.	int u_idx = -1, v_idx = -1;	
22.	for (int k = 0; k < N; ++k) {	
23.	if (labels[k] == u) u_idx = k;	
24.	if (labels[k] == v) v_idx = k;	
25.	}	
26.	if (u_idx != -1 && v_idx != -1) {	
27.	matrix[u_idx][v_idx] = w;	
28.	}	
29.	}	
30.	}	
31.	std::cout << "Adjacency Matrix:\n";	
32.	for (int i = 0; i < N; ++i) std::cout << " " << labels[i];	
33.	std::cout << std::endl;	
34.		
35.	for (int i = 0; i < N; ++i) {	
36.	std::cout << labels[i];	
37.	for (int j = 0; j < N; ++j) {	
38.	std::cout << " " << matrix[i][j];	
39.	}	
40.	std::cout << std::endl;	
41.	}	

42.	return 0;
43.	}

B. Output Program

```
PS D:\programming stuff\tugas\ALGO\PRAK\modul 6> cd "d:\programming stuff\tugas\ALGO\PRAK\modul 6\" ; if ($?) { g++ Soal1.cpp -o Soal1 } ; if ($?) { .\Soal1 }
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
```

Gambar 26 Output Soal1.cpp Modul 6

C. Pembahasan

Program ini digunakan untuk membentuk dan menampilkan **adjacency matrix** dari sebuah graf berdasarkan data simpul (*vertex*) dan sisi (*edge*) yang dimasukkan oleh pengguna. Pada bagian awal program digunakan `#include <iostream>` untuk mendukung proses input dan output menggunakan `std::cin` dan `std::cout`, sedangkan `#include <vector>` digunakan agar program dapat memanfaatkan struktur data **vector** yang bersifat dinamis.

Eksekusi program dimulai dari fungsi `main()`. Pertama, dideklarasikan dua buah variabel bertipe integer, yaitu `N` untuk menyimpan jumlah simpul dan `M` untuk menyimpan jumlah sisi pada graf. Program kemudian membaca nilai `N` dari masukan pengguna. Apabila proses pembacaan gagal, program langsung dihentikan menggunakan `return 0` sehingga tidak ada proses selanjutnya yang dijalankan.

Setelah itu dibuat sebuah `vector<char>` bernama `labels` dengan ukuran sebanyak `N` elemen. Vector ini digunakan untuk menyimpan label dari setiap simpul. Program kemudian melakukan perulangan sebanyak `N` kali untuk membaca setiap label yang dimasukkan pengguna dan menyimpannya ke dalam vector sesuai urutannya.

Selanjutnya program membuat sebuah matriks dua dimensi menggunakan `std::vector<std::vector<int>>` berukuran $N \times N$. Seluruh elemen matriks diinisialisasi dengan nilai 0 sebagai penanda bahwa pada awalnya belum terdapat hubungan antar simpul. Setelah matriks selesai dibuat, program membaca nilai `M` yang menunjukkan jumlah sisi pada graf. Jika proses pembacaan gagal, program kembali dihentikan menggunakan `return 0`.

Program kemudian melakukan perulangan sebanyak M kali untuk memproses setiap sisi. Pada setiap iterasi dibaca tiga buah data, yaitu label simpul asal (u), label simpul tujuan (v), dan bobot sisi (w). Jika ketiga data berhasil dibaca, program membuat dua variabel, yaitu u_idx dan v_idx , yang diinisialisasi dengan nilai -1 sebagai penanda bahwa posisi kedua simpul belum ditemukan.

Selanjutnya dilakukan pencarian posisi masing-masing label dengan menelusuri seluruh isi vector `labels`. Apabila label yang sedang diperiksa sama dengan u , maka indeksnya disimpan ke dalam u_idx . Begitu pula jika label tersebut sama dengan v , maka indeksnya disimpan ke dalam v_idx . Setelah proses pencarian selesai, program memeriksa apakah kedua indeks telah ditemukan. Jika u_idx dan v_idx tidak lagi bernilai -1 , maka bobot sisi w disimpan ke dalam matriks pada posisi `matrix[u_idx][v_idx]`. Dengan cara ini, setiap sisi pada graf direpresentasikan sebagai nilai bobot pada adjacency matrix.

Setelah seluruh data sisi selesai diproses, program menampilkan tulisan "**Adjacency Matrix:**" sebagai judul hasil keluaran. Selanjutnya program mencetak seluruh label simpul sebagai judul kolom matriks. Setelah itu dilakukan perulangan untuk mencetak isi matriks baris demi baris. Pada setiap baris, label simpul dicetak terlebih dahulu sebagai penanda baris, kemudian seluruh nilai pada kolom dicetak secara berurutan sehingga membentuk tampilan adjacency matrix yang lengkap. Setelah satu baris selesai dicetak, program berpindah ke baris berikutnya hingga seluruh matriks berhasil ditampilkan.

Program diakhiri dengan perintah `return 0`, yang menandakan bahwa seluruh proses telah selesai dijalankan tanpa terjadi kesalahan.

Berdasarkan analisis algoritma, proses pembacaan label simpul membutuhkan waktu $O(N)$. Untuk setiap sisi, program melakukan pencarian indeks kedua simpul dengan menelusuri seluruh label sehingga memerlukan waktu $O(N)$. Karena proses tersebut dilakukan sebanyak M kali, maka total waktunya menjadi $O(M \times N)$. Selain itu, proses pencetakan adjacency matrix memerlukan waktu $O(N^2)$ karena seluruh elemen matriks dicetak satu per satu. Dengan demikian, **kompleksitas waktu (Time Complexity)** keseluruhan program adalah $O(M \times N + N^2)$. Adapun **kompleksitas ruang (Space Complexity)** adalah $O(N^2)$ karena program menyimpan adjacency matrix berukuran $N \times N$, sedangkan vector label hanya membutuhkan ruang tambahan sebesar $O(N)$.

D. Analisis

Algoritma ini membangun adjacency matrix melalui tiga tahap utama: pemetaan, translasi, dan konstruksi. Pertama, program menyimpan label vertex ke dalam vektor sebagai referensi indeks. Kedua, saat membaca sisi, program melakukan pencarian linear untuk menerjemahkan label ke koordinat matriks (baris, kolom). Ketiga, bobot sisi ditempatkan pada koordinat tersebut dalam matriks $N \times N$ yang telah diinisialisasi dengan nol. Hasil akhirnya adalah tabel numerik yang merepresentasikan struktur graf secara terarah. Kompleksitas waktu dari algoritma ini adalah $O(M \times N)$, di mana M adalah jumlah sisi dan N adalah jumlah vertex. Hal ini disebabkan oleh proses pencarian label, untuk setiap M sisi yang dibaca, program melakukan perulangan for hingga N kali guna menemukan posisi indeks dari dua vertex yang terlibat. adanya deklarasi matriks `matrix(N, std::vector(N, 0))` menyebabkan matriks ini menjadi variabel dominan yang menentukan batas atas penggunaan ruang memori. Sehingga penggunaan memori sekaligus kompleksitas ruangnya sebesar $O(N^2)$.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

```
N
V1 V2 ... VN A11 A12 ... A1N A21 A22 ... A2N
...
AN1 AN2 ... ANN
```

Keterangan:

N adalah jumlah vertex.

V_i adalah label vertex ke- i .

A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .

Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .

Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

```
Adjacency List:
V1 -> (X1,w1), (X2,w2), ...
V2 -> (X1,w1), (X2,w2), ...
...
VN -> ...
```

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

$$2 \leq N \leq 10$$

$$0 \leq A_{ij} \leq 1000$$

Label vertex berupa karakter alfabet kapital A sampai Z .

Semua label vertex dijamin unik.

Nilai diagonal utama matrix dijamin 0.

Nilai 0 berarti tidak ada edge.

Nilai positif berarti bobot edge.

Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5	Adjacency List:
A B C D E	A -> (B,4), (C,2)
0 4 2 0 0	B -> (D,7)
0 0 0 7 0	C -> (B,1), (E,6)
0 1 0 0 6	D -> (C,3)
0 0 3 0 0	E -> -
0 0 0 0 0	

Penjelasan Contoh

Pada baris vertex A , nilai positif muncul pada kolom B dan C . Karena itu, adjacency list untuk

A adalah $(B, 4), (C, 2)$.

Pada baris vertex E , semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah $E \rightarrow -$.

A. Source Code

Table 25. Source Code Soal2.cpp Modul 6

1.	#include <iostream>
2.	#include <vector>
3.	
4.	int main() {
5.	int N;
6.	if (!(std::cin >> N) N <= 0) return 0;
7.	
8.	std::vector<char> labels(N);
9.	for (int i = 0; i < N; ++i) {
10.	std::cin >> labels[i];
11.	}
12.	
13.	std::vector<std::vector<int>> matrix(N,
	std::vector<int>(N));
14.	for (int i = 0; i < N; ++i) {
15.	for (int j = 0; j < N; ++j) {
16.	std::cin >> matrix[i][j];
17.	}
18.	}
19.	
20.	std::cout << "Adjacency List:\n";
21.	for (int i = 0; i < N; ++i) {
22.	std::cout << labels[i] << " -> ";
23.	bool first = true;
24.	bool Edge = false;
25.	for (int j = 0; j < N; ++j) {
26.	if (matrix[i][j] != 0) {
27.	if (!first) std::cout << ", ";
28.	std::cout << "(" << labels[j] << ", " <<
	matrix[i][j] << ")";
29.	first = false;
30.	Edge = true;
31.	}
32.	}
33.	if (!Edge) {
34.	std::cout << "-";
35.	}
36.	std::cout << std::endl;
37.	}
38.	
39.	return 0;
40.	}

B. Ouput Program

```
PS D:\programming stuff\tugas> cd "d:\programming stuff\tugas\ALGO\PRAK\modul 6\" ; if ($?) { g++ soal2.cpp -o soal2 } ; if ($?) { .\soal2 }
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 27. Ouput Soal2.cpp Modul 6

C. Pembahasan

`#include <iostream>` dan `#include <vector>` ini dipakai untuk membuat pustaka standar C++ supaya program bisa melakukan input dan output, untuk menggunakan struktur data vector yang fleksibel.

`int main()` { titik awal program ini mulai dijalankan.

`int N;` deklarasi variabel integer buat nyimpan jumlah vertex.

`if (!(std::cin >> N) || N <= 0) return 0;` baris ini baca nilai N dari user, dan langsung berhenti kalau inputnya gagal **atau** kalau N-nya nol/negatif jadi ada validasi tambahan dibanding kode sebelumnya, supaya nggak ada kasus N negatif yang bikin program error pas alokasi vector.

`std::vector<char> labels(N);` bikin vector berukuran N buat nampung karakter label tiap vertex.

`for (int i = 0; i < N; ++i) { std::cin >> labels[i]; }` perulangan buat ngisi tiap posisi di vector labels dengan karakter yang diinput user.

`std::vector<std::vector<int>> matrix(N, std::vector<int>(N));` bikin matriks dua dimensi ukuran N x N. Beda dengan kode sebelumnya, di sini matriksnya nggak diinisialisasi 0 secara eksplisit, soalnya isinya bakal langsung diisi penuh dari input user di baris selanjutnya jadi nggak perlu nilai default.

`for (int i = 0; i < N; ++i) { for (int j = 0; j < N; ++j) { std::cin >> matrix[i][j]; } }` ini dua loop bersarang yang baca seluruh isi matriks dari input user, baris demi baris, kolom demi kolom — jadi user di sini langsung kasih adjacency matrix yang

utuh, bukan dalam bentuk list sisi (u, v, w) kayak soal sebelumnya.

`std::cout << "Adjacency List:\n";` nyetak judul ke layar sebagai info hasil programnya.

`for (int i = 0; i < N; ++i) {` mulai perulangan buat mroses tiap vertex satu per satu, baris ini bakal nentuin vertex mana yang jadi titik asal.

`std::cout << labels[i] << " -> ";` nyetak label vertex yang sedang diproses, diikuti tanda panah buat nunjukin awal dari daftar tujuannya.

`bool first = true; bool Edge = false;` dua variabel boolean disiapin di sini — `first` buat ngatur kapan koma pemisah perlu dicetak, dan `Edge` buat nandain apakah vertex ini punya minimal satu sisi keluar atau nggak.

`for (int j = 0; j < N; ++j) {` loop dalam buat ngecek satu-satu kolom di baris ke-`i`, nyari semua vertex tujuan yang terhubung dari vertex `i`.

`if (matrix[i][j] != 0) {` kalau nilai di sel itu bukan nol, artinya ada sisi dari vertex `i` ke vertex `j` dengan bobot sebesar nilai itu.

`if (!first) std::cout << ", ";` kalau ini bukan sisi pertama yang ditemukan, cetak koma dulu sebagai pemisah sebelum nyetak sisi berikutnya.

`std::cout << "(" << labels[j] << ", " << matrix[i][j] << ")";` nyetak pasangan vertex tujuan dan bobotnya dalam format (`label, bobot`).

`first = false; Edge = true;` update status, nandain bahwa udah ada minimal satu sisi yang ketemu dan dicetak.

`if (!Edge) { std::cout << "-"; }` setelah loop dalam selesai, kalau ternyata nggak ada satu pun sisi yang ketemu buat vertex `i` (`Edge` masih `false`), cetak tanda "-" sebagai penanda vertex itu nggak punya tujuan keluar.

`std::cout << std::endl; }` pindah baris baru setelah satu vertex selesai diproses, lalu lanjut ke vertex berikutnya.

`return 0; }` fungsi main selesai.

D. Analisis

Algoritma ini menggunakan metode Breadth First (BFS) buat mencari jalur terpendek dalam grid dua dimensi, dengan bekerja secara sistematis menyebar dari titik awal ke arah adjacent vertices terdekat secara berlapis. Proses diawali dengan memasukkan titik awal ke dalam antrian, lalu secara berulang mengambil vertex dari antrian tersebut untuk diperiksa adjacent verticesnya ke empat arah mata angin. Algoritma memanfaatkan matriks visited untuk mencatat sel yang sudah pernah dilewati, sehingga mencegah pengulangan proses pada sel yang sama dan menghindari looping yang tidak perlu. Karena sifat BFS yang menjelajahi setiap sel berdasarkan jarak langkah dari titik awal, vertex tujuan yang ditemukan pertama kali dipastikan merupakan jalur terpendek yang mungkin, dengan total kompleksitas waktu sebesar $O(R \times C)$ karena setiap sel pada grid akan dikunjungi maksimal satu kali. semua struktur data utama yang digunakan skalanya berbanding lurus dengan jumlah sel dalam grid ($N = R \times C$), maka total kompleksitas ruangnya adalah $O(R \times C)$.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

0 menyatakan jalan yang dapat dilewati.

1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C G21 G22 ... G2C
...
GR1 GR2 ... GRC SR SC
FR FC
```

Keterangan:

R adalah jumlah baris grid.

C adalah jumlah kolom grid.

G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .

(S_R, S_C) adalah posisi awal.

(F_R, F_C) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.

Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

$$1 \leq R \leq 100$$

$$1 \leq C \leq 100$$

G_{ij} hanya bernilai 0 atau 1.

$$0 \leq S_R < R$$

$$0 \leq S_C < C$$

$$0 \leq F_R < R$$

$$0 \leq F_C < C$$

Sel start dan finish dijamin bernilai 0.

Pergerakan hanya boleh dilakukan ke sel bernilai 0.

Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada $(0, 0)$ dan posisi tujuan berada pada $(7, 9)$. Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel $(2, 1)$.

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari

$(0, 0)$ ke $(7, 9)$ adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Table 26. Source Code Soal3.cpp Modul 6

```
1.  #include <iostream>
2.  #include <vector>
3.
4.  using namespace std;
5.  struct Node {
6.      int r, c, dist;
7.  };
8.
9.  int main() {
10.     int R, C;
11.     if (!(cin >> R >> C) || R <= 0 || C <= 0) return 0;
12.
13.     vector<vector<int>> grid(R, vector<int>(C));
14.     for (int i = 0; i < R; ++i) {
15.         for (int j = 0; j < C; ++j) {
16.             cin >> grid[i][j];
17.         }
18.     }
19.
20.     int SR, SC, FR, FC;
21.     if (!(cin >> SR >> SC >> FR >> FC)) return 0;
22.     if (SR < 0 || SR >= R || SC < 0 || SC >= C || FR < 0 ||
FR >= R || FC < 0 || FC >= C) {
23.         return 0;
24.     }
25.
26.     vector<Node> q;
27.     int head = 0;
28.
29.     vector<vector<bool>> visited(R, vector<bool>(C, false));
30.
31.     q.push_back({SR, SC, 0});
32.     visited[SR][SC] = true;
33.
34.     int dr[] = {-1, 1, 0, 0};
35.     int dc[] = {0, 0, -1, 1};
36.
37.     while (head < q.size()) {
38.         Node curr = q[head];
39.         head++;
40.
41.         if (curr.r == FR && curr.c == FC) {
42.             cout << curr.dist << endl;
43.             return 0;
44.         }
45.
46.         for (int i = 0; i < 4; ++i) {
47.             int nr = curr.r + dr[i];
48.             int nc = curr.c + dc[i];
```

```

49.
50.     if (nr >= 0 && nr < R && nc >= 0 && nc < C
51.         && grid[nr][nc] == 0 && !visited[nr][nc]) {
52.         visited[nr][nc] = true;
53.         q.push_back({nr, nc, curr.dist + 1});
54.     }
55. }
56. }
57. cout << -1 << endl;
58. return 0;
59. }

```

B. Ouput Program

```

PS D:\programming stuff\tugas\ALGO\PRAK\modul 6> cd "d:\programming stuff\tugas\ALGO\PRAK\modul 6\" ; if ($?) { g++ soal3.cpp -o soal3 } ; if
($?) { .\soal3 }
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16

```

Gambar 28. Ouput Soal3.cpp Modul 6

C. Pembahasan

`#include <iostream>` dan `#include <vector>` ini dipakai untuk membuat pustaka standar C++ supaya program bisa melakukan input output dan menggunakan struktur data vector yang fleksibel.

`using namespace std;` ini bikin kita kda perlu nulis `std::` di depan setiap fungsi/objek standar library.

`struct Node { int r, c, dist; };` di sini didefinisikan struct Node yang isinya tiga atribut: `r` dan `c` buat nyimpen posisi baris dan kolom suatu sel di grid, sama `dist` buat nyimpen jarak (jumlah langkah) dari titik awal sampai ke sel itu. Struct ini nantinya dipakai sebagai elemen di antrean BFS.

`int main() {` titik awal program mulai dijalankan.

`int R, C;` deklarasi variabel buat nyimpen jumlah baris (R) dan kolom (C) dari grid.

`if (!(cin >> R >> C) || R <= 0 || C <= 0) return 0;` baca nilai R dan C, lalu langsung berhenti kalau pembacaannya gagal atau kalau salah satu nilainya nol/negatif

soalnya ukuran grid yang nggak valid bisa bikin program error pas alokasi memori.

`vector<vector<int>> grid(R, vector<int>(C));` bikin grid dua dimensi ukuran $R \times C$ buat nyimpen data petanya (misalnya 0 buat jalan yang bisa dilewati, dan nilai lain buat penghalang).

`for (int i = 0; i < R; ++i) { for (int j = 0; j < C; ++j) { cin >> grid[i][j]; } }` dua loop bersarang buat baca isi grid dari input user, baris demi baris, kolom demi kolom.

`int SR, SC, FR, FC;` deklarasi variabel buat nyimpen koordinat titik awal (SR, SC) dan titik tujuan (FR, FC).

`if (!(cin >> SR >> SC >> FR >> FC)) return 0;` baca keempat koordinat itu, langsung berhenti kalau pembacaannya gagal.

`if (SR < 0 || SR >= R || SC < 0 || SC >= C || FR < 0 || FR >= R || FC < 0 || FC >= C) { return 0; }` ini validasi tambahan, mastiin koordinat titik awal dan tujuan yang diinput user masih dalam batas ukuran grid (nggak boleh negatif atau lebih besar dari R/C) — kalau salah satu koordinatnya di luar batas, program berhenti.

`vector<Node> q;` bikin vector `q` yang berfungsi sebagai antrean (queue) buat nyimpen sel-sel yang bakal diperiksa selama proses BFS.

`int head = 0;` variabel `head` ini dipakai sebagai penanda posisi "depan" antrean jadi alih-alih pakai `std::queue` beneran (yang ngehapus elemen depan tiap kali di-pop), di sini antreannya disimulasikan pakai vector biasa, dan `head` nunjukin elemen mana yang lagi diproses sekarang.

`vector<vector<bool>> visited(R, vector<bool>(C, false));` bikin matriks boolean ukuran $R \times C$, semua diinisialisasi `false`, buat nyatet sel mana aja yang udah pernah dikunjungi selama BFS.

`q.push_back({SR, SC, 0});` titik awal dimasukin ke antrean sebagai elemen pertama, dengan jarak awal 0.

`visited[SR][SC] = true;` titik awal langsung ditandai sebagai udah dikunjungi, supaya nggak diproses ulang nanti.

`int dr[] = {-1, 1, 0, 0}; int dc[] = {0, 0, -1, 1};` dua array ini menyimpan pola pergeseran koordinat buat ngecek empat arah tetangga (atas, bawah, kiri, kanan) dari suatu sel — pasangan `dr[i]` dan `dc[i]` di indeks yang sama ngewakilin satu arah gerakan.

`while (head < q.size())` { loop utama BFS ini berjalan terus menerus selama masih ada elemen di antrean yang belum diproses (ditandai dengan `head` belum nyampe ukuran `q`).

`Node curr = q[head]; head++;` ambil elemen di posisi depan antrean (sesuai `head`), simpan ke variabel `curr`, lalu majukan `head` supaya elemen ini dianggap udah diproses dan nggak diulang.

`if (curr.r == FR && curr.c == FC)` { `cout << curr.dist << endl;` `return 0;` } kalau sel yang sedang diproses ternyata sama dengan titik tujuan, langsung cetak jaraknya dan program berhenti — soalnya BFS jamin kalau titik tujuan ditemukan, jaraknya itu udah pasti yang paling pendek.

`for (int i = 0; i < 4; ++i)` { perulangan buat ngecek keempat arah tetangga dari sel yang sedang diproses.

`int nr = curr.r + dr[i]; int nc = curr.c + dc[i];` ngitung koordinat baris dan kolom dari tetangga ke arah ke-`i`, berdasarkan posisi sekarang ditambah pola pergeseran yang udah disiapin di array `dr` dan `dc`.

`if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] == 0 && !visited[nr][nc])` { ini pengecekan empat syarat sekaligus sebelum tetangga itu boleh diproses: koordinatnya harus masih dalam batas grid (nggak keluar baris/kolom), selnya harus berupa jalan yang bisa dilewati (nilai grid-nya 0), dan selnya belum pernah dikunjungi sebelumnya.

`visited[nr][nc] = true;` kalau semua syarat terpenuhi, sel tetangga itu langsung ditandai udah dikunjungi.

`q.push_back({nr, nc, curr.dist + 1});` sel tetangga itu dimasukin ke antrean buat diproses nanti, dengan jaraknya ditambah satu dari jarak sel sekarang.

} } dua kurung kurawal ini nutup blok `if` pengecekan syarat tetangga, lalu nutup loop `for` pengecekan empat arah.

} kurung ini nutup loop `while` utama, menandakan satu iterasi pemrosesan satu sel BFS udah

selesai.

`cout << -1 << endl;` baris ini cuma bakal dieksekusi kalau loop `while` udah selesai total (antrean kosong) tapi titik tujuan nggak pernah ketemu — jadi -1 dicetak sebagai tanda kalau jalur ke tujuan nggak ada.

`return 0; }` fungsi main selesai.

D. Analisis

algoritma yang digunakan adalah DFS dengan backtracking yang berfungsi menghitung seluruh jumlah rute dari titik awal ke tujuan. Cara kerjanya adalah dengan menjelajahi setiap rute sedalam mungkin, saat mencapai tujuan, algoritma menambah jumlah jalur, lalu melakukan backtracking yaitu membatalkan status kunjungan pada sel tersebut agar rute alternatif lain dapat menggunakan sel yang sama. Karena pendekatan ini mencoba semua kombinasi langkah tanpa membatasi kunjungan sel, kompleksitas waktunya menjadi $O(4^{R \times C})$, yakni eksponensial, di mana setiap sel di dalam grid memiliki hingga empat potensi arah percabangan yang harus dievaluasi. Akibatnya, durasi eksekusi akan meningkat drastis seiring bertambahnya ukuran grid. Secara keseluruhan, karena semua struktur data yang digunakan matriks grid, matriks visited, dan stack skalanya berbanding lurus dengan total jumlah sel ($R \times C$), maka kompleksitas ruang totalnya adalah $O(R \times C)$.

Soal 4 Menghitung Semua Jalur Keluar Labirin

Time Limit: 2.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

```
R C
G11 G12 ... G1C G21 G22 ... G2C
...
GR1 GR2 ... GRC SR SC
FR FC
```

Keterangan:

R adalah jumlah baris grid.

C adalah jumlah kolom grid.

G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .

(S_R, S_C) adalah posisi awal.

(F_R, F_C) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.

Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

$$1 \leq R \leq 7$$

$$1 \leq C \leq 7$$

G_{ij} hanya bernilai 0 atau 1.

$$0 \leq S_R < R$$

$$0 \leq S_C < C$$

$$0 \leq F_R < R$$

$$0 \leq F_C < C$$

Sel start dan finish dijamin bernilai 0.

Pergerakan hanya boleh dilakukan ke sel bernilai 0.

Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.

Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.

Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada $(0,0)$ dan posisi tujuan berada pada $(4,5)$. Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel $(0,5)$.

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Table 27. Source Code Soal4.cpp Modul 6

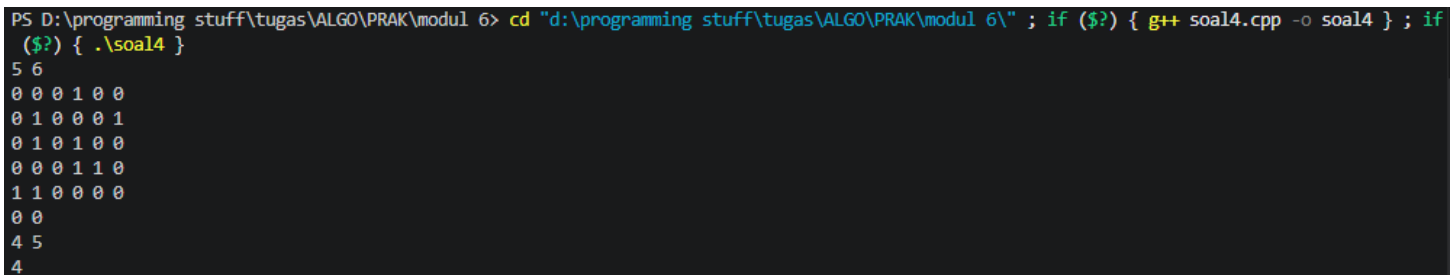
1.	#include <iostream>
2.	#include <vector>
3.	
4.	using namespace std;
5.	
6.	struct DataPosisi {
7.	int r, c;
8.	int arahKe;
9.	};
10.	
11.	int main() {
12.	int R, C;
13.	cin >> R >> C;
14.	
15.	vector<vector<int>> grid(R, vector<int>(C));
16.	for (int i = 0; i < R; ++i) {
17.	for (int j = 0; j < C; ++j) {
18.	cin >> grid[i][j];
19.	}
20.	}
21.	
22.	int SR, SC, FR, FC;
23.	cin >> SR >> SC >> FR >> FC;
24.	
25.	vector<vector<bool>> visited(R, vector<bool>(C,
26.	false));
27.	vector<DataPosisi> tumpukan;
28.	int jumlahJalur = 0;
29.	
30.	int dr[] = {-1, 1, 0, 0};
31.	int dc[] = {0, 0, -1, 1};
32.	
33.	tumpukan.push_back({SR, SC, 0});
34.	visited[SR][SC] = true;
35.	
36.	while (!tumpukan.empty()) {
37.	int r = tumpukan.back().r;
38.	int c = tumpukan.back().c;
39.	int d = tumpukan.back().arahKe;
40.	
41.	if (r == FR && c == FC) {
42.	jumlahJalur++;
43.	visited[r][c] = false;
44.	tumpukan.pop_back();
45.	continue;
46.	}
47.	
48.	if (tumpukan.back().arahKe < 4) {
	int arahSekarang = tumpukan.back().arahKe;

```

49.         tumpukan.back().arahKe++;
50.
51.         int barisBaru = r + dr[arahSekarang];
52.         int kolomBaru = c + dc[arahSekarang];
53.
54.         if (barisBaru >= 0 && barisBaru < R &&
55.             kolomBaru >= 0 && kolomBaru < C &&
56.             grid[barisBaru][kolomBaru] == 0 &&
57.             !visited[barisBaru][kolomBaru]) {
58.
59.             visited[barisBaru][kolomBaru] = true;
60.             tumpukan.push_back({barisBaru, kolomBaru,
0});
61.         }
62.     } else {
63.         visited[r][c] = false;
64.         tumpukan.pop_back();
65.     }
66. }
67.
68. cout << jumlahJalur << endl;
69.
70. return 0;
71. }

```

B. Ouput Program



```

PS D:\programming stuff\tugas\ALGO\PRAK\modul 6> cd "d:\programming stuff\tugas\ALGO\PRAK\modul 6\" ; if ($?) { g++ soal4.cpp -o soal4 } ; if ($?) { .\soal4 }
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

Gambar 29. Ouput Soal4.cpp Modul 6

C. Pembahasan

`#include <iostream>` dan `#include <vector>` ini dipakai untuk membuat pustaka standar C++ supaya program bisa melakukan input output dan menggunakan struktur data vector yang fleksibel.

`using namespace std;` ini bikin kita kda perlu nulis `std::` di depan setiap fungsi/objek standar library.

`struct DataPosisi { int r, c; int arahKe; };` di sini didefinisikan struct `DataPosisi` yang isinya tiga atribut: `r` dan `c` buat nyimpen posisi baris dan kolom suatu sel,

sama arahKe buat nyimpen arah mana yang udah/akan dicoba dari sel itu (nilainya 0-4). Atribut arahKe ini yang bikin struct ini beda dari Node di kode sebelumnya — soalnya di sini kita perlu nginget "udah nyoba arah ke berapa" buat tiap sel yang ada di stack.

```
int main() { titik awal program mulai dijalankan.
```

```
int R, C; cin >> R >> C; deklarasi dan baca jumlah baris (R) dan kolom (C) dari grid.
```

Beda dari kode BFS sebelumnya, di sini nggak ada validasi input (nggak dicek apakah pembacaan gagal atau R/C-nya negatif).

```
vector<vector<int>> grid(R, vector<int>(C)); bikin grid dua dimensi ukuran R x C buat nyimpen data petanya.
```

```
for (int i = 0; i < R; ++i) { for (int j = 0; j < C; ++j) { cin >>
```

```
grid[i][j]; } } dua loop bersarang buat baca isi grid dari input user, baris demi baris, kolom
```

```
demi kolom.
```

```
int SR, SC, FR, FC; cin >> SR >> SC >> FR >> FC; deklarasi dan baca
```

koordinat titik awal (SR, SC) dan titik tujuan (FR, FC). Sama seperti baris sebelumnya, di sini juga nggak ada validasi apakah koordinatnya masih dalam batas grid.

```
vector<vector<bool>> visited(R, vector<bool>(C, false)); bikin matriks
```

boolean ukuran R x C, semua diinisialisasi false, buat nyatet sel mana yang sedang dilewati di jalur yang lagi dijalanin sekarang.

```
vector<DataPosisi> tumpukan; bikin vector tumpukan yang berfungsi sebagai stack
```

— di sinilah algoritmanya beda fundamental dari BFS, soalnya stack ini diakses dari ujung belakang (LIFO), bukan dari depan kayak antrean (FIFO).

```
int jumlahJalur = 0; variabel buat ngitung berapa banyak jalur yang berhasil nyampe
```

dari titik awal ke titik tujuan.

```
int dr[] = {-1, 1, 0, 0}; int dc[] = {0, 0, -1, 1}; dua array ini nyimpen
```

pola pergeseran koordinat buat ngecek empat arah tetangga (atas, bawah, kiri, kanan), sama persis kayak di kode BFS.

```
tumpukan.push_back({SR, SC, 0}); titik awal dimasukin ke stack, dengan arahKe
```

diset 0 — artinya belum ada arah yang dicoba dari sel ini.

`visited[SR][SC] = true;` titik awal ditandai sebagai sedang dilewati.

`while (!tumpukan.empty())` { loop utama jalan terus selama stack belum kosong.

`int r = tumpukan.back().r; int c = tumpukan.back().c; int d = tumpukan.back().arahKe;` ambil data posisi dan arah dari elemen paling atas (paling belakang) di stack — ini adalah sel yang sedang "aktif" diproses sekarang. Catatan: variabel `d` di sini sebenarnya nggak dipakai lagi di baris-baris berikutnya, soalnya nilai `arahKe` dibaca ulang langsung dari `tumpukan.back()` di kondisi `if` selanjutnya.

`if (r == FR && c == FC)` { kalau sel yang sedang aktif itu sama dengan titik tujuan, berarti satu jalur lengkap udah ketemu.

`jumlahJalur++;` jalur yang ketemu ini dihitung, nambahin total jumlah jalur.

`visited[r][c] = false; tumpukan.pop_back(); continue;` sel tujuan ini ditandai balik jadi `false` (di-unmark) dan dikeluarkan dari stack — ini bagian penting dari proses **backtracking**, soalnya sel ini harus "dilepas" lagi supaya bisa dilewati oleh jalur lain yang berbeda nantinya. Setelah itu `continue` lompat ke awal loop `while` buat lanjut proses elemen stack selanjutnya.

`if (tumpukan.back().arahKe < 4)` { kalau sel yang sekarang aktif masih punya arah yang belum dicoba (dari 4 arah yang mungkin), program lanjut nyoba arah berikutnya.

`int arahSekarang = tumpukan.back().arahKe;`
`tumpukan.back().arahKe++;` simpan arah yang mau dicoba sekarang ke `arahSekarang`, lalu langsung naikkan `arahKe` di elemen stack itu — jadi kalau nanti balik lagi ke sel ini, dia bakal coba arah selanjutnya, bukan ngulang arah yang sama.

`int barisBaru = r + dr[arahSekarang]; int kolomBaru = c + dc[arahSekarang];` ngitung koordinat tetangga ke arah yang sedang dicoba, berdasarkan posisi sekarang ditambah pola pergeseran dari array `dr` dan `dc`.

`if (barisBaru >= 0 && barisBaru < R && kolomBaru >= 0 && kolomBaru < C && grid[barisBaru][kolomBaru] == 0 && !visited[barisBaru][kolomBaru])` { pengecekan empat syarat sebelum tetangga itu

boleh dilewati: koordinatnya harus dalam batas grid, selnya harus jalan yang bisa dilewati (nilai grid 0), dan selnya belum sedang dilewati di jalur sekarang.

`visited[barisBaru][kolomBaru] = true;` kalau syaratnya terpenuhi, sel tetangga itu ditandai sebagai sedang dilewati.

`tumpukan.push_back({barisBaru, kolomBaru, 0});` sel tetangga itu dimasukan ke stack buat diproses, dengan `arahKe` diset 0 lagi karena belum ada arah yang dicoba dari sel baru ini.

`} }` dua kurung ini nutup blok `if` pengecekan syarat tetangga, lalu nutup blok `if` (`tumpukan.back().arahKe < 4`).

`} else { visited[r][c] = false; tumpukan.pop_back(); }` kalau ternyata sel yang sekarang aktif udah nyoba keempat arah (`arahKe` udah nyampe 4) dan nggak ada lagi yang bisa dieksplor, sel ini di-unmark dan dikeluarkan dari stack — ini juga bagian dari backtracking, mundur ke sel sebelumnya karena jalur dari sel ini udah buntu.

`}` kurung ini nutup loop `while` utama.

`cout << jumlahJalur << endl;` cetak total jumlah jalur yang ditemukan dari titik awal ke titik tujuan.

`return 0; }` fungsi main selesai.

D. Analisis

Algoritma yang dipakai adalah DFS dengan backtracking yang berfungsi menghitung semua jumlah dari titik awal ke tujuan. Berbeda dari BFS yang akan berhenti ketika tujuannya itu ketemu pertama kali, kalo DFS ini programnya itu harus tetap melanjutkan eksplorasi sampai semua kemungkinan rute habis dicoba. Karena tujuannya bukan jalur terpendek tapi total jumlah jalur yang sudah ada

Cara kerjanya setiap sel yang masuk ke stack di siman bersama atribut `arahke` yang menyimpan progres arah mana yang sudah di coba dari sel itu. Selama sebuah sel masih mempunyai arah yang belum dicoba (`arahKe < 4`), program bakal terus nyoba arah berikutnya satu per satu. Begitu sel itu nyampe ke titik tujuan, `jumlahJalur` ditambah, lalu sel itu **di-unmark** (`visited` diset balik ke `false`) dan dikeluarkan dari stack — proses unmark inilah yang disebut backtracking, soalnya sel itu harus

"dilepas" supaya bisa dipakai lagi oleh rute lain yang berbeda. Backtracking yang sama juga terjadi kalau sebuah sel udah nyoba keempat arah dan nggak ada lagi yang bisa dieksplor (jalur buntu).

Untuk kompleksitas ruang, karena semua struktur data utama yang dipakai — grid, matriks visited, dan stack tumpukan — skalanya berbanding lurus dengan total jumlah sel ($R \times C$), maka kompleksitas ruang totalnya adalah **$O(R \times C)$** .

Soal 5: Tree Traversal

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTREE terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.

INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.

POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.

ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

```
N
A1 A2 ... AN Q
T1 T2
... TQ
```

Keterangan:

N adalah banyaknya bilangan yang akan dimasukkan ke RBTREE.

A_i adalah bilangan ke- i .

Q adalah banyaknya query traversal.

T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

```
[Preorder] : daftar_angka [Inorder] : daftar_angka [Postorder] : daftar_angka
```

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

```
Tree kosong. Tidak ada yang bisa ditampilkan.
```

Constraints

$$0 \leq N \leq 1000$$

$$-10000000000 \leq A_i \leq 10000000000$$

$$1 \leq Q \leq 20$$

T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER INORDER POSTORDER ALL	[Preorder] : 50 30 20 40 70 60 80
	[Inorder] : 20 30 40 50 60 70 80
	[Postorder] : 20 40 30 60 80 70 50

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Table 28. Source Code Soal5.cpp Modul 6

1.	#include <iostream>
2.	#include <string>
3.	#include "RedBlackTree.h"
4.	
5.	using namespace std;
6.	
7.	void cetakPreorder(const RedBlackTree::Node* node, bool &first)
	{
8.	if (node->isNil) return;
9.	
10.	if (!first) cout << " ";
11.	cout << node->key;
12.	first = false;
13.	
14.	cetakPreorder(node->left, first);
15.	cetakPreorder(node->right, first);
16.	}
17.	
18.	void cetakInorder(const RedBlackTree::Node* node, bool &first) {
19.	if (node->isNil) return;
20.	
21.	cetakInorder(node->left, first);
22.	
23.	if (!first) cout << " ";
24.	cout << node->key;
25.	first = false;
26.	
27.	cetakInorder(node->right, first);
28.	}
29.	
30.	void cetakPostorder(const RedBlackTree::Node* node, bool &first)
	{
31.	if (node->isNil) return;
32.	
33.	cetakPostorder(node->left, first);
34.	cetakPostorder(node->right, first);
35.	
36.	if (!first) cout << " ";
37.	cout << node->key;
38.	first = false;
39.	}
40.	
41.	int main() {

```

42.     RedBlackTree rbt;
43.     int N;
44.     if (!(cin >> N)) return 0;
45.
46.     for (int i = 0; i < N; i++) {
47.         int angka;
48.         cin >> angka;
49.         if (!rbt.contains(angka)) {
50.             rbt.insert(angka);
51.         }
52.     }
53.
54.     int Q;
55.     cin >> Q;
56.     for (int i = 0; i < Q; i++) {
57.         string tipeQuery;
58.         cin >> tipeQuery;
59.
60.         bool first = true;
61.         if (tipeQuery == "PREORDER") {
62.             cout << "[Preorder] : ";
63.             cetakPreorder(rbt.root(), first);
64.             cout << endl;
65.         } else if (tipeQuery == "INORDER") {
66.             cout << "[Inorder] : ";
67.             cetakInorder(rbt.root(), first);
68.             cout << endl;
69.         } else if (tipeQuery == "POSTORDER") {
70.             cout << "[Postorder] : ";
71.             cetakPostorder(rbt.root(), first);
72.             cout << endl;
73.         } else if (tipeQuery == "ALL") {
74.             first = true;
75.             cout << "[Preorder] : ";
76.             cetakPreorder(rbt.root(), first);
77.             cout << endl;
78.
79.             first = true;
80.             cout << "[Inorder] : ";
81.             cetakInorder(rbt.root(), first);
82.             cout << endl;
83.
84.             first = true;
85.             cout << "[Postorder] : ";
86.             cetakPostorder(rbt.root(), first);
87.             cout << endl;
88.         }

```

89.	}
90.	
91.	return 0;
92.	}

B. Output Program

```
PS D:\programming stuff\tugas\ALGO\PRAK\modul 6> cd "d:\programming stuff\tugas\ALGO\PRAK\modul 6\" ; if ($?) { g++ soal5.cpp -o soal5 } ; if ($?) { .\soal5 }
7
50 30 70 20 40 60 80
4
PREORDER INORDER POSTORDER ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
```

Gambar 30. Output Soal5.cpp Modul 6

C. Pembahasan

`#include <iostream>`, `#include <string>`, dan `#include "RedBlackTree.h"` dipakai buat memasukan pustaka standar C++ untuk input output dan tipe data string, sekaligus header custom `RedBlackTree.h` yang isinya implementasi struktur data Red-Black Tree-nya sendiri (kelas `RedBlackTree` dengan fungsi-fungsi kayak `insert`, `contains`, dan `root`).

`using namespace std;` supaya tidak perlu nulis `std::` di depan tiap fungsi/objek standar library.

`void cetakPreorder(const RedBlackTree::Node* node, bool &first)` { deklarasi fungsi rekursif buat nyetak isi tree dengan urutan preorder (root dulu, baru kiri, baru kanan). Parameternya berupa pointer ke node yang sedang diproses, dan referensi ke boolean `first` yang dipakai buat ngatur spasi antar angka pas dicetak.

`if (node->isNil) return;` ini base case dari rekursi — kalau node yang sekarang itu node "nil" (semacam node penanda kosong yang biasa dipakai di implementasi Red-Black Tree, gantinya `nullptr`), fungsi langsung berhenti tanpa nyetak apa-apa.

`if (!first) cout << " ";` kalau ini bukan angka pertama yang dicetak, kasih spasi dulu sebelum nyetak key dari node sekarang, lalu tandain `first` jadi false biar angka selanjutnya juga dikasih spasi pemisah.

`cetakPreorder(node->left, first); cetakPreorder(node->right, first);` dua pemanggilan rekursif ke anak kiri dan anak kanan — sesuai urutan preorder, root udah dicetak duluan di atas, baru sekarang lanjut ke subtree kiri, lalu subtree kanan.

`void cetakInorder(const RedBlackTree::Node* node, bool &first) {` fungsi rekursif buat nyetak isi tree dengan urutan inorder (kiri dulu, baru root, baru kanan) — strukturnya mirip `cetakPreorder`, tapi posisi pencetakan key-nya beda.

`if (node->isNil) return;` base case yang sama, berhenti kalau ketemu node nil.

`cetakInorder(node->left, first);` rekursi ke subtree kiri duluan, sebelum nyetak node sekarang — ini yang bikin hasilnya keluar dalam urutan terurut (khususnya buat BST/RBT yang terurut berdasarkan key).

`if (!first) cout << " "; cout << node->key; first = false;` setelah subtree kiri selesai diproses semua, baru key dari node sekarang dicetak.

`cetakInorder(node->right, first);` terakhir, rekursi ke subtree kanan.

`void cetakPostorder(const RedBlackTree::Node* node, bool &first) {` fungsi rekursif buat nyetak isi tree dengan urutan postorder (kiri dulu, kanan, baru root paling akhir).

`if (node->isNil) return;` base case yang sama.

`cetakPostorder(node->left, first); cetakPostorder(node->right, first);` rekursi ke subtree kiri dan kanan duluan, sebelum nyetak node sekarang.

`if (!first) cout << " "; cout << node->key; first = false;` key dari node sekarang baru dicetak paling akhir, setelah kedua subtree-nya selesai diproses semua.

`int main() {` titik awal program mulai dijalankan.

`RedBlackTree rbt;` bikin objek `rbt` dari kelas `RedBlackTree` yang struktur internalnya didefinisikan di file header di sinilah treenya bakal dibangun.

`int N; if (!(cin >> N)) return 0;` baca jumlah angka yang mau dimasukin ke tree, langsung berhenti kalau pembacaannya gagal.

`for (int i = 0; i < N; i++) {` perulangan buat masukin N angka satu per satu ke dalam tree.

`int angka; cin >> angka;` baca satu angka dari input.

`if (!rbt.contains(angka)) { rbt.insert(angka); }` sebelum dimasukin, dicek dulu apakah angka itu udah ada di tree (pakai fungsi `contains`) — kalau belum ada, baru di-insert. Ini buat mencegah duplikat key di dalam tree.

`int Q; if (!(cin >> Q)) return 0;` baca jumlah query yang mau diproses, langsung berhenti kalau pembacaannya gagal.

`for (int i = 0; i < Q; i++) {` perulangan buat memproses Q query satu per satu.

`string tipeQuery; cin >> tipeQuery;` baca jenis query yang diminta, dalam bentuk string (misalnya "PREORDER", "INORDER", dst).

`bool first = true;` variabel `first` direset ke `true` di tiap awal query baru, supaya format spasi pas pencetakan tree mulai dari kosong lagi setiap kali ada query baru.

`if (tipeQuery == "PREORDER") { cout << "[Preorder] : "; cetakPreorder(rbt.root(), first); cout << endl; }` kalau query-nya "PREORDER", cetak label dulu, lalu panggil fungsi `cetakPreorder` dari root tree, terus pindah baris baru.

`} else if (tipeQuery == "INORDER") { cout << "[Inorder] : "; cetakInorder(rbt.root(), first); cout << endl; }` mirip di atas, tapi buat query "INORDER".

`} else if (tipeQuery == "POSTORDER") { cout << "[Postorder] : "; cetakPostorder(rbt.root(), first); cout << endl; }` mirip juga, buat query "POSTORDER".

`} else if (tipeQuery == "ALL") {` kalau query-nya "ALL", program bakal nyetak ketiga jenis traversal sekaligus dalam satu kali query.

`first = true; cout << "[Preorder] : "; cetakPreorder(rbt.root(), first); cout << endl;` cetak preorder dulu — `first` di-reset lagi ke `true` di sini supaya format spasinya benar buat traversal yang baru ini, soalnya kalau nggak di-reset, status `first` bisa dibawa dari traversal sebelumnya.

`first = true; cout << "[Inorder] : "; cetakInorder(rbt.root(), first); cout << endl;` lanjut cetak inorder, dengan `first` di-reset lagi.

`first = true; cout << "[Postorder] : "; cetakPostorder(rbt.root(), first); cout << endl;` terakhir cetak postorder, juga dengan `first` direset lagi.

`} else { cout << "Query tidak dikenali: " << tipeQuery << endl; }` kalau tipe query yang dibaca nggak cocok sama salah satu dari empat pilihan di atas, program nyetak pesan error yang nunjukin query apa yang nggak dikenali.

`}` kurung ini nutup loop `for` pemrosesan query.

`return 0; }` menandakan fungsi `main` selesai dan program berhasil dijalankan sampai akhir.

`} else if (tipeQuery == "ALL") {` kalau query-nya "ALL", program bakal nyetak ketiga jenis traversal sekaligus dalam satu kali query.

`first = true; cout << "[Preorder] : "; cetakPreorder(rbt.root(), first); cout << endl;` cetak preorder dulu — `first` di-reset lagi ke `true` di sini supaya format spasinya benar buat traversal yang baru ini, soalnya kalau nggak di-reset, status `first` bisa dibawa dari traversal sebelumnya.

`first = true; cout << "[Inorder] : "; cetakInorder(rbt.root(), first); cout << endl;` lanjut cetak inorder, dengan `first` di-reset lagi.

`first = true; cout << "[Postorder] : "; cetakPostorder(rbt.root(), first); cout << endl;` terakhir cetak postorder, juga dengan `first` direset lagi.

```
} else { cout << "Query tidak dikenali: " << tipeQuery << endl; } kalau tipe query yang dibaca  
nggak cocok sama salah satu dari empat pilihan di atas, program nyetak pesan error yang nunjukin  
query apa yang nggak dikenali.
```

```
} kurung ini nutup loop for pemrosesan query.
```

```
return 0; } fungsi main selesai.
```

D. Analisis

cara yang dipakai adalah tree traversal pada Red-Black Tree dengan menggunakan rekursi untuk menelusuri setiap node, di mana setiap metode (Preorder, Inorder, dan Postorder) memiliki urutan kunjungan yang berbeda berdasarkan prioritas kunjungan antara akar, cabang kiri, dan cabang kanan. Kompleksitas waktunya adalah $O(N)$ karena algoritma wajib mengunjungi seluruh node tree sebanyak satu kali untuk keperluan print di layar, sementara kompleksitas ruangnya adalah $O(\log N)$ karena sifat Red-Black Tree yang selalu menjaga keseimbangan tinggi pohon melalui rotasi otomatis. Pada fungsi main, program mengelola alur dengan membaca N data yang disaring melalui metode contains, kemudian merespons Q perintah query secara dinamis, di mana variabel bool &first diterapkan sebagai pengendali pemformatan spasi agar setiap output tetap rapi dan tidak menyertakan spasi di awal atau akhir baris.

TAUTAN GITHUB

<https://github.com/Fath-wan/DSA-all-module.git>